# Fraus

**Medição de satisfação sem LLM em runtime**

*João Pedro Tavares Vicente*

# Table of contents

1. O cliente *diz*. O Fraus estima. 6
1.1 Escolha seu percurso 7
1.2 Sem LLM em runtime 7
1.3 Baixar 7
2. Banca e metodologia 8
2.1 A tese 8
2.1.1 Pergunta de pesquisa 8
2.1.2 Hipótese e alcance 8
2.1.3 Unidade de análise e método 8
2.1.4 Origem dos dados e avaliação 9
2.1.5 Interpretação e ameaças à validade 9
2.1.6 O que a tese não promete 9
2.2 Arquitetura 10
2.2.1 Dentro de uma cabeça Transformer 11
2.2.2 A decisão que explica o desenho 12
2.2.3 As sete famílias que pontuam 13
2.2.4 Onde roda, e com que motor 13
2.2.5 A ironia continua carregada, e não pontua 13
2.2.6 Régua: o que pontuou uma conversa não é fixo no tempo 14
2.2.7 Repontuar é um trabalho, não uma requisição 14
2.2.8 A explicação da nota 15
2.2.9 Confiança: conjunto conformal e abstenção 16
2.2.10 Sinalizadores por regra 16
2.2.11 Impacto por tema e deterioração 16
2.2.12 Onde cada coisa mora 17
2.3 Treinamento dos modelos do Fraus 18
2.3.1 Receita vigente de pesquisa — 26/10/2026 18
2.3.2 Diário histórico — não é a receita de execução vigente 20
2.3.3 Como abrir no Colab 20
2.3.4 Notebook 01 — classificador de texto 21
2.3.5 Notebook 02 — fusor dos tres sinais 22
2.3.6 Contrato de features 31
2.3.7 Notebook 03 — classificador de emocao 31
2.3.8 Notebook 04 — classificador de ironia 34
2.3.9 Sinal lexico — SentiLex-PT02 39

2.3.10 Sinal de estilo 39
2.3.11 Calibração — a temperatura mora no artefato (desde 19/10/2026) 40
2.3.12 Notebook 02 — a seção 10 é uma pendência, não uma etapa 40
2.4 Generalização e evidências 43
2.4.1 Estado da investigação — 27/10/2026 43
2.4.2 Matriz de evidências 43
2.4.3 Auditoria integral dos corpora 48
2.4.4 Protocolo de auditoria 48
2.4.5 Resultados da reprodução — 26–27/10/2026 49
2.4.6 Protocolo da campanha fechada 55
2.4.7 Como interpretar as conclusões 56
2.5 Cartão do modelo 57
2.5.1 Uso pretendido e uso proibido 57
2.5.2 As peças — métricas históricas de 15/09/2026 58
2.5.3 Onde o fusor erra — avaliação histórica por fatia 58
2.5.4 Conversa só de cortesia — sem sinal desde 15/09/2026 60
2.5.5 A cabeça de ironia — taxa de erro histórica publicada 60
2.5.6 Calibração e conformal do fusor 60
2.5.7 Limitações que valem para tudo acima 62
2.6 Limitações conhecidas 63
2.6.1 Generalização e proveniência — atualização de 27/10/2026 63
2.6.2 Limitações operacionais do deploy gratuito 63
2.6.3 O NPS é inferido, não perguntado 64
2.6.4 O sinal de tempo é treinado em dados sintéticos 65
2.6.5 A cabeça de ironia é pouco confiável 66
2.6.6 Desprezo não é uma classe treinada 66
2.6.7 Nenhuma feature agregada reverte uma probabilidade saturada 66
2.6.8 A explicação é do modelo, não do cliente 67
2.6.9 O que não dá para consertar sem corpus de atendimento real 67
2.7 Conformidade regulatória — o Fraus e o EU AI Act 68
2.7.1 1. Por que este documento existe 68
2.7.2 2. Escopo do sistema 68
2.7.3 3. As três obrigações que já valem, e a que vem 68
2.7.4 4. Limitações que importam para quem lê um número do Fraus 70
2.7.5 5. Fontes 70
2.8 Ironia: corpus e Laya 72
2.8.1 Resumo: consertar o corpus antes de treinar o Laya 72
2.8.2 O IDPT entrega o rótulo, e nenhum encoder escapa disso 72

2.8.3 Treinar só ironia exige mexer em sete células e num rótulo de produção 73
2.8.4 Consertar o negativo vale mais que trocar o encoder 76
2.8.5 Três treinos e uma régua de 300 pares separam as causas 76
2.8.6 O ganho esperado é de poucos pontos, e vem do dado 78
2.8.7 Régua de pares mínimos 79
2.8.8 Conclusão 79
2.9 Diferenciais 81
2.9.1 Diferenciais — relatório de entrega (14/10/2026) 81
2.9.2 Diferenciais — o que os concorrentes fazem (fontes de 14/10/2026) 88
2.9.3 Diferenciais — roteiro da demonstração 93
3. Usuários 97
3.1 Guia de uso 97
3.1.1 Acesso e permissões 97
3.1.2 Analisar um arquivo 97
3.1.3 Confirmar o que foi salvo 98
3.1.4 Encontrar o período e os atendimentos 98
3.1.5 Interpretar os números sem inventar certeza 99
3.1.6 Quando algo não responde 99
3.1.7 Receber conversas do Tars 99
3.1.8 Escopo deste guia 99
3.2 Integração com o Tars 100
3.2.1 Conectar, em dois passos 100
3.2.2 O que é um atendimento 100
3.2.3 Quando o envio acontece 100
3.2.4 O pedido 101
3.2.5 Privacidade — o que vale e o que falta 101
3.2.6 Limpeza 101
4. Desenvolvedores 102
4.1 Desenvolvimento local 102
4.1.1 Ambiente Python e extras 102
4.1.2 Artefatos obrigatórios 103
4.1.3 Executar API e dashboard 103
4.1.4 Verificar uma alteração 104
4.1.5 Trabalhar na documentação existente 105
4.1.6 Contratos para consultar antes de editar 105
4.2 As dez invariantes 107
4.2.1 1. Sem LLM em runtime 107
4.2.2 2. Ausência de dado não é insatisfação 107

4.2.3 3. Score, nota e categoria são derivados no servidor 107
4.2.4 4. Faixas de NPS: 0–6 detrator, 7–8 neutro, 9–10 promotor 108
4.2.5 5. Latência nunca é persistida 108
4.2.6 6. Timestamps timezone-aware 108
4.2.7 7. Modelo ausente é falha alta e explícita 109
4.2.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito 109
4.2.9 9. As 39 chaves de feature batem exatamente com NOMES_FEATURES 109
4.2.10 10. O corpus de treino não pode entregar o rótulo 110
4.3 Design da interface — Contexto, não certeza 111
4.3.1 Direção 111
4.3.2 Cor e significado 113
4.3.3 Tipografia e ícones 113
4.3.4 LP 114
4.3.5 Dashboard 116
4.3.6 Acesso, acessibilidade e estados 119
4.3.7 Verificação e manutenção 120
4.4 LP e dashboard — antes e depois 121
4.4.1 Condições da comparação 121
4.4.2 Landing page 122
4.4.3 Dashboard 0
4.4.4 Otimização da LP aprovada 0
4.4.5 Verificação e limites 0
4.5 Encolher os modelos — o que foi medido, e por que a int8 foi recusada 0
4.5.1 O problema 0
4.5.2 O que foi testado 0
4.5.3 O resultado da melhor variante 0
4.5.4 E é aqui que ela é recusada 0
4.5.5 A variante mista, medida em 11/09/2026 0
4.5.6 O backend ONNX em produção, medido em 15/09/2026 0
4.5.7 O que sobra como caminho 0
4.5.8 Reproduzir 0
4.5.9 Variante experimental: um encoder, três cabeças 0
4.6 Referência de código 0
4.6.1 Indicadores 0
4.6.2 Modelo canônico 0
4.6.3 O fusor e as 39 features 0
4.6.4 Os sinais 0
4.6.5 Evidência e deriva 0

FRAUS — DOCUMENTAÇÃO

# 1. O cliente diz.
# O Fraus estima.

Ferramenta que lê atendimento por chatbot e estima satisfação **sem perguntar nada ao cliente**. Um encerramento como "*ok, obrigado* 🙂" não prova satisfação nem insatisfação: o contexto traz sinais, não acesso ao estado real de quem escreveu.

LER AS INVARIANTES | A TESE | ARQUITETURA

---

| FAMÍLIAS DE SINAL | FEATURES NO FUSOR | BERTIMBAU FINE-TUNADOS | LLMS NA INFERÊNCIA |
|---|---|---|---|
| 08 | 39 | 03 | 00 |

> ⚠ NPS INFERIDO, NÃO DECLARADO
>
> O NPS do Fraus **não é perguntado**: ele é derivado do texto do atendimento. Toda exibição carrega a etiqueta de estimativa, e apresentá-lo como NPS declarado seria falso. Ver Limitações e Conformidade.

## 1.1 ESCOLHA SEU PERCURSO

### BANCA E METODOLOGIA — A TESE

Entenda a pergunta, as unidades de análise e o que sustenta as conclusões. Siga por arquitetura, treinamento, generalização e evidências e cartão do modelo. Leia os limites antes de interpretar uma métrica como prova em atendimentos reais.

### USUÁRIOS — GUIA DE USO

Leve um arquivo à análise, confira a gravação e encontre os atendimentos no período certo. Aprenda a distinguir estimativa, ausência de sinal e indisponibilidade. Para receber conversas automaticamente, siga a integração com o Tars.

### DESENVOLVEDORES — DESENVOLVIMENTO LOCAL

Prepare os extras e artefatos, execute API e dashboard e encontre os comandos de testes e documentação. Comece pelas invariantes; consulte o design e a referência de código antes de alterar contratos.

## 1.2 SEM LLM EM RUNTIME

Três BERTimbau fine-tunados (satisfação, emoção, ironia) e um fusor `LogisticRegression` sobre 39 features de sete famílias de sinal. Inferência local, em CPU, sem nenhuma chamada de rede no caminho de predição.

Isso é **requisito**, não otimização. As probabilidades por mensagem e a decomposição do fusor permitem inspecionar a leitura do sistema; não são ativações internas do Transformer nem prova causal sobre palavras.

## 1.3 BAIXAR

O PDF com o site inteiro está em pdf/fraus.pdf, gerado no mesmo build que publica estas páginas — então ele nunca está mais velho que o site.

# 2. Banca e metodologia

## 2.1 A TESE

### 2.1.1 Pergunta de pesquisa

**Como combinar sinais observáveis de uma conversa de chatbot para estimar a satisfação do cliente, sem pesquisa obrigatória e sem LLM em runtime?**

Pesquisas declaradas de NPS e CSAT dependem da participação do cliente e podem sofrer viés de não resposta. Já a contenção informa se houve transferência para um humano, mas não distingue, sozinha, resolução de desistência. São medidas complementares; o Fraus não propõe substituir uma pela outra nem tratar quem não respondeu à pesquisa como insatisfeito.

### 2.1.2 Hipótese e alcance

A hipótese de trabalho é que **texto, emoji e estrutura temporal contêm pistas complementares** para uma estimativa de satisfação. Essas pistas podem apoiar a revisão humana e a identificação de tendências, inclusive em atendimentos sem resposta a uma pesquisa.

Uma frase motivadora é:

> "*que atendimento maravilhoso, só esperei 3 horas*"

Ela ilustra a possível divergência entre elogio literal e situação relatada. Não é prova de que o sistema reconheça ironia, nem de que uma espera relatada no texto tenha sido medida pelos timestamps. O comportamento nesse caso depende dos classificadores e das features; falhas são declaradas em Limitações.

### 2.1.3 Unidade de análise e método

- **Mensagem:** as três cabeças neurais estimam satisfação textual, emoção e ironia. Emoção tem sete classes treinadas; desprezo é derivado de raiva e nojo, não uma oitava classe aprendida.
- **Conversa:** os sinais das mensagens e da estrutura são agregados em **39 features de sete famílias**. O fusor aplica `StandardScaler` e `LogisticRegression` e produz um score de 0 a 100. Ironia é a oitava família observada, exibida por mensagem, mas não participa desse vetor.
- **Recorte de conversas:** o servidor deriva nota, categoria e indicadores. O **NPS é inferido**, não a resposta declarada de clientes. Conversa sem evidência suficiente, incluindo cortesia isolada, é “sem sinal”, não zero.

O método usa classificadores BERTimbau fine-tunados e sinais determinísticos, com inferência local em CPU e **nenhuma chamada a LLM em runtime**. A Arquitetura detalha o processamento; o Treinamento separa a receita vigente do diário histórico.

### 2.1.4 Origem dos dados e avaliação

O texto de satisfação vem de resenhas B2W; emoção usa GoEmotions traduzido para português; a campanha diagnóstica de ironia usa MultiPICo em português. ConsumerBR e XED são referências de transferência **já consultadas**, não testes virgens. As fontes, divisões, artefatos e resultados efetivamente obtidos estão centralizados em Generalização e evidências, sem repetir suas tabelas nesta página.

O fusor é avaliado em conversas sintéticas: texto real é combinado com tempo, emoji e estrutura gerados. Isso mede comportamento nesse domínio, não acurácia em atendimentos reais rotulados. A recomendação e as estrelas de uma resenha também não são o mesmo alvo, nem equivalem automaticamente a NPS declarado.

### 2.1.5 Interpretação e ameaças à validade

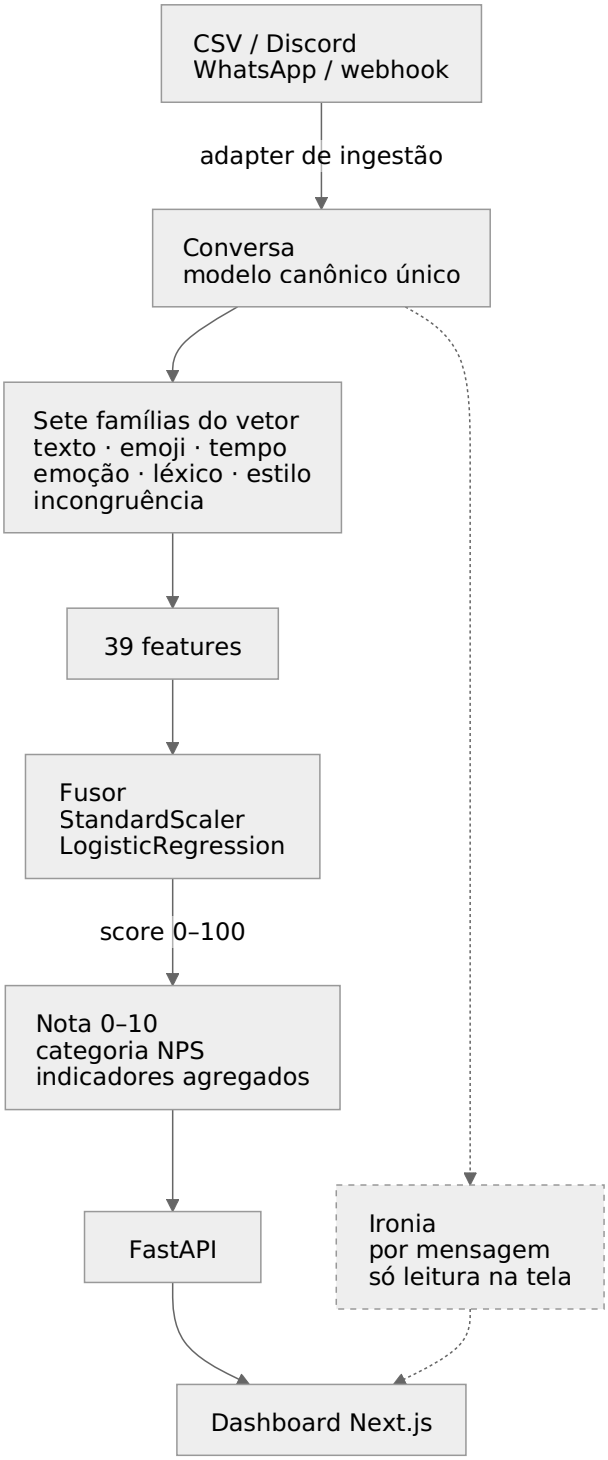
O **falso containment** cruza ausência de transferência com classificação detratora inferida. É um sinal para investigar, não comprovação de que o cliente desistiu ou de que o bot causou insatisfação. Esse cruzamento pode ser feito por outras abordagens; não se reivindica exclusividade metodológica.

As principais ameaças são mudança de domínio (resenha e rede social não são suporte), tradução e ruído dos rótulos, dependência entre produtos/autores, atalhos do simulador, seleção histórica com consulta ao teste e proveniência incompleta dos pesos antigos. Sem curvas comparáveis e manifestos históricos não se conclui presença nem ausência de overfitting atual. A campanha de três sementes por cabeça está **pendente de autenticação do Colab**, não concluída.

### 2.1.6 O que a tese não promete

- Não mede diretamente o estado interno do cliente nem produz diagnóstico.
- Não atribui causalidade a palavras, tempo de espera ou contribuições do fusor.
- Não decide automaticamente sobre clientes e **não pontua atendentes** — ver Conformidade.
- Não transforma contenção em qualidade, nem ausência de dado em insatisfação.
- Não apresenta resultados históricos ou testes sintéticos como validação de produção. O Cartão do modelo preserva as medições históricas e explicita seus limites.

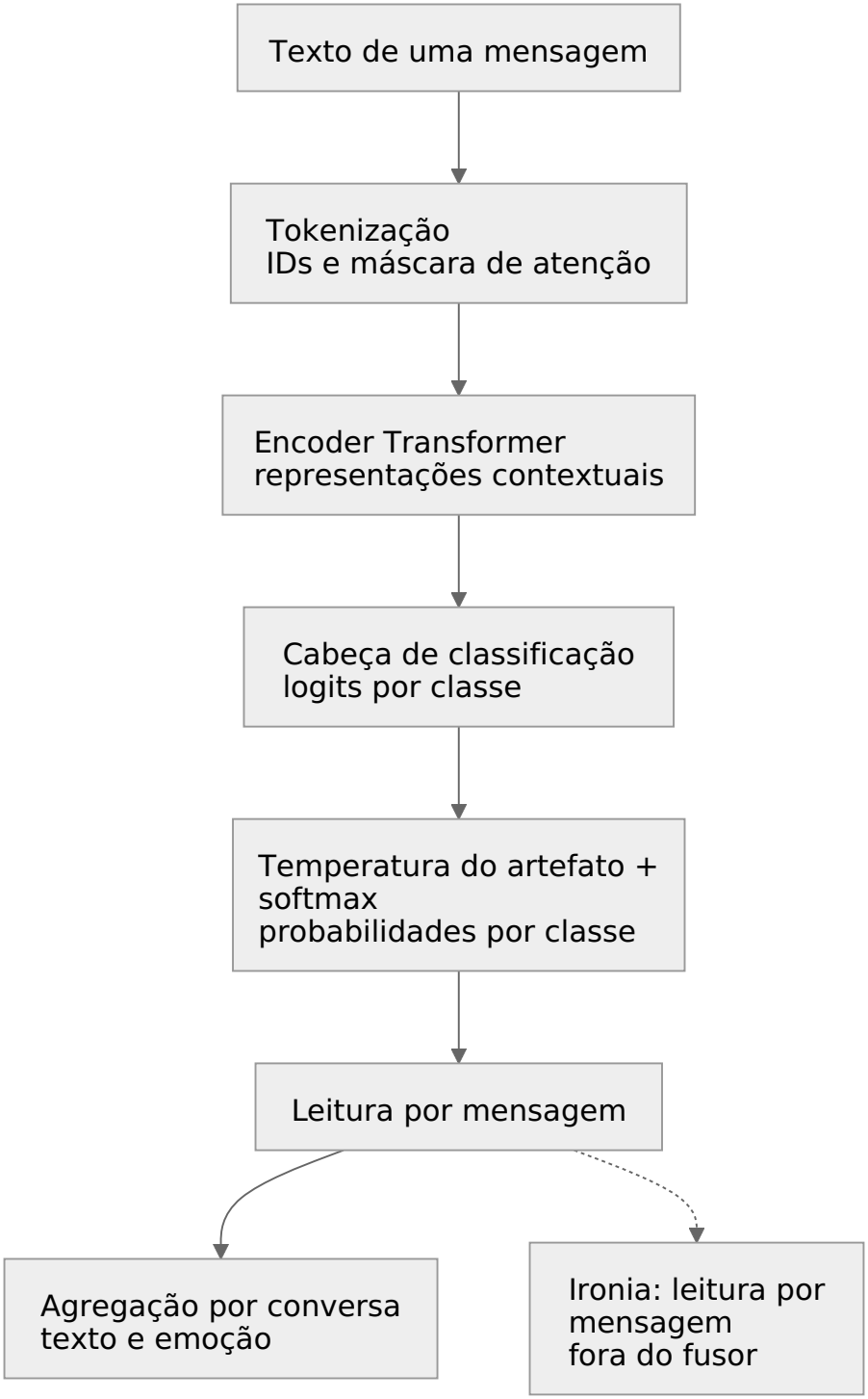
## 2.2 ARQUITETURA




** a cabeça de ironia é lida por mensagem e mostrada na tela, mas não entra nas 39 features do Fusor — ver a seção abaixo.*

São **oito famílias de sinais**, não oito cabeças neurais: satisfação textual, emoção e ironia usam três classificadores; as demais famílias são calculadas. Sete famílias alimentam o fusor. A ausência de fala informativa do cliente produz score nulo, nunca zero.

### 2.2.1 Dentro de uma cabeça Transformer




**Equivalente textual:** o tokenizador converte o texto em identificadores de tokens e indica quais posições são válidas. O encoder constrói representações contextuais; a cabeça de classificação as transforma em logits das classes. A temperatura, quando registrada no artefato, ajusta a escala desses logits antes do softmax. A saída é

uma distribuição de probabilidades, não uma nota da conversa. Texto e emoção entram em agregações de suas famílias; ironia segue para leitura por mensagem. Emoção tem sete classes treinadas e desprezo derivado da díade raiva + nojo. Não se fixa aqui número de camadas ou dimensões sem metadados do checkpoint.

O explorador da tela **Modelo** é um esquema didático desse fluxo. A simulação de uma frase mostra somente os sinais devolvidos pela classificação; não mede tempo, não processa uma conversa completa e não calcula o score do fusor. Selecionar uma etapa não é executar inferência.

QUATRO COISAS DIFERENTES QUE A VISUALIZAÇÃO NÃO DEVE CONFUNDIR

| GRANDEZA | SIGNIFICADO | LIMITE |
|---|---|---|
| Probabilidade por classe | saída de uma cabeça para a mensagem, ou do fusor para suas classes | confiança do modelo, não verdade sobre o cliente |
| Peso global | coeficiente aprendido pelo fusor para uma feature padronizada | não é a importância causal de uma palavra ou o efeito fixo em pontos |
| Contribuição local em logits | coeficiente multiplicado pela feature padronizada daquela conversa | soma com o intercepto em cada logit; não soma diretamente o score |
| Ativações e atenção internas | estados intermediários do Transformer | não são expostos pela API nem observados pelo esquema didático |

O fusor **não é uma rede densa ilustrativa**: as 39 features passam pelo `StandardScaler` e pela regressão logística. O softmax e a projeção em score são não lineares; contribuições por família não demonstram causalidade. Esquema e saídas reais não autorizam desenhar “palavras que causaram a nota”.

## 2.2.2 A decisão que explica o desenho

**Um modelo canônico, muitos adaptadores.** CSV, Totalk, transcrição de `.docx`/`.pdf` e webhook desembocam todos em `Conversa` — e o resto do sistema não sabe de onde o dado veio. Adicionar uma fonte é escrever um adaptador, não tocar em sinal, fusor ou indicador.

### 2.2.3 As sete famílias que pontuam

| FAMÍLIA | O QUE MEDE | POR QUE EXISTE |
|---|---|---|
| **texto** | P(insatisfeito/neutro/satisfeito) por mensagem | o sinal principal — o único fine-tunado para a tarefa |
| **emoji** | polaridade e **posição relativa** | emoji no fim da conversa não vale o mesmo que no começo |
| **tempo** | latências, escalação, abandono | espera longa degrada satisfação (literatura de live chat) |
| **emoção** | 6 emoções básicas de Ekman + neutro, e desprezo derivado de raiva + nojo | raiva e frustração distinguem *tipos* de insatisfação |
| **léxico** | SentiLex-PT02 com escopo de negação | âncora determinística, auditável, imune a retreino |
| **estilo** | caixa alta, alongamento, palavrão, censura | intensidade que o classificador de sentença perde |
| **incongruência** | elogio convivendo com situação negativa | é o que alcança a frase canônica do projeto |

### 2.2.4 Onde roda, e com que motor

| | |
|---|---|
| **Backend dos modelos** | `FRAUS_BACKEND=torch` (padrão), `onnx` — os mesmos checkpoints, sem torch em runtime — ou `onnx-multitarefa` (candidato, não promovido); valor desconhecido é erro de boot |
| **Cabeça de ironia** | `FRAUS_IRONIA_BACKEND`: `padrao` (BERTimbau), `laya` ou `laya-onnx` — troca só a leitura por mensagem; a ironia não entra no score |
| **Motor** | carregado por um provedor preguiçoso (`ProvedorDeMotor`): `frio` → `carregando` → `pronto` ou `erro`, sem fallback para dublê |
| **Aquecimento** | a carga começa em segundo plano no boot e a cada `GET /saude`; `GET /saude/prontidao` só dá 200 com o motor pronto |
| **Produção** | dashboard e API em dois projetos Vercel, estado no Supabase Postgres; a API é serverless e **esfria** |

O aquecimento reduz o cold start e não o elimina. Para eliminá-lo, a API precisa de um container sempre ligado (ver Limitações).

### 2.2.5 A ironia continua carregada, e não pontua

A família de ironia usa uma das **três cabeças neurais** e é obrigatória no **motor de inferência**, não para a API subir. A API pode estar no estado frio, carregando ou erro; predição exige todos os artefatos válidos, sem fallback. A ironia aparece na tela **por mensagem**, mas **saiu do vetor** em 04/09/2026.

Em medição histórica no corpus de treino do fusor (B2W-Reviews01), ela marcou **74% das resenhas satisfeitas** como irônicas contra **9% das insatisfeitas**: naquele corpus ela é um detector de sentimento **positivo**, não de ironia. O fusor aprendeu peso **+0,77** para `ironia_prob_media`, empurrando ironia para *satisfeito* — o inverso do que o nome promete.

> **Feature que mede outra coisa que não o nome dela é pior que feature ausente.** Ela some do vetor sem que ninguém perceba a inversão, porque a acurácia global continua ótima.

Ver Treinamento para a medição completa.

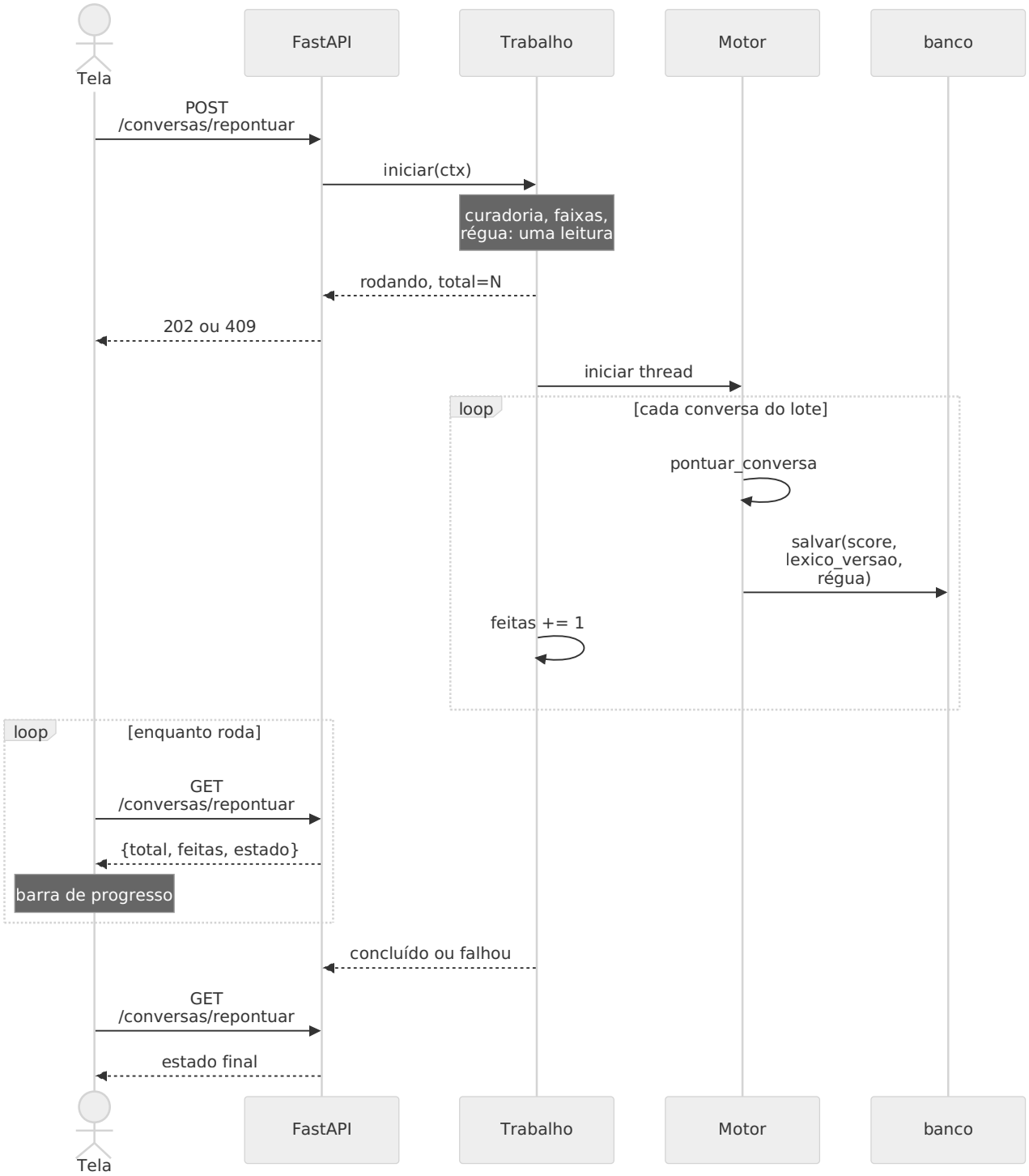
### 2.2.6 Régua: o que pontuou uma conversa não é fixo no tempo

Cada conversa é gravada com a **régua** que a pontuou: a assinatura do motor servindo (`Motor.regua`, `None` no motor dublê) mais a versão da curadoria (`lexico_versao`). Trocar de checkpoint, trocar de backend (torch/onnx) ou o analista ensinar um termo novo ao léxico muda a régua — e o banco passa a ter conversas antigas pontuadas por uma régua que não é mais a vigente.

O sistema não recalcula sozinho: ele **avisa**. `GET /indicadores` expõe `pontuadas_com_regua_antiga`, contando contra a régua vigente lida a cada requisição (nunca guardada em memória, pelo mesmo motivo de `faixas_vigentes` e `curadoria_vigente` — sem cópia de estado envelhecendo). A tela mostra o aviso com o motivo (checkpoint novo, léxico curado, ou os dois), nunca um número solto sem explicação.

### 2.2.7 Repontuar é um trabalho, não uma requisição

Zerar o aviso de régua misturada significa rodar os três BERTimbau de novo em cada conversa do banco — em milhares de atendimentos isso estoura qualquer timeout HTTP. `POST /conversas/repontuar` (`fraus/api/repontuacao.py`) só dispara uma thread em segundo plano; `GET /conversas/repontuar` lê o progresso (`total`, `feitas`, `estado`, `erro`) que a tela transforma em barra. Não é fila — é **um trabalho por vez**: uma segunda chamada enquanto a primeira roda recebe 409, porque duas repontuações em paralelo gravariam o mesmo banco com duas curadorias diferentes. O estado mora na memória do processo (reiniciar a API no meio perde o progresso, não o que já foi gravado — cada conversa salva já carrega sua própria `lexico_versao` e régua).




## 2.2.8 A explicação da nota

`GET /conversas/{id}/atribuicao` devolve também o campo `explicacao`: a decomposição exata do fusor (intercepto e contribuições por feature, somando no logit), o efeito por família e frases geradas por template — nenhum LLM. Duas rotas completam o E1: `POST /conversas/{id}/contrafactual` refaz a nota mudando só o relógio (e recusa o que sai de `|z| ≤ 4`) e `GET /indicadores/drivers` agrega os efeitos numa amostra com semente. Nenhuma das três grava; o score, a nota e a categoria gravados não mudam. A lógica de texto fica em `fraus/explicacao.py`; os limites estão em Limitações.

### 2.2.9 Confiança: conjunto conformal e abstenção

Duas rotas somente leitura, que não gravam e não alteram score, nota nem categoria: `GET /conversas/{id}/confianca` (conjunto de classes e o marcador de abstenção da conversa) e `GET /indicadores/confianca` (agregado, só pelo critério do conjunto; conversas de outra régua ficam fora e vêm contadas em `defasadas`). A lógica pura está em `fraus/conformal.py`; as métricas de calibração, em `fraus/calibracao.py`.

O artefato é `fraus/dados/conformal_fusor.json`, versionado, fora de `modelos/`, chaveado pela régua (`regua_do_fusor`: fusor mais regra de cortesia). Contrato de falha: sem artefato, artefato malformado ou régua diferente da vigente, as duas rotas respondem **503 com o motivo**; no detalhe, uma conversa pontuada por outra régua responde 409. Conjunto vazio conta como abstenção. Conversa sem score tem conjunto `null` e não entra na contagem. Indicadores e demais rotas não dependem do artefato. Limites em Limitações.

### 2.2.10 Sinalizadores por regra

`sinalizadores` é derivado na leitura, em `_com_derivacoes`, por `fraus/sinalizadores.py`: cinco regras de risco sobre a estrutura da conversa, sem gravar coluna nem chave de payload e sem ler o texto ou o relógio da fala humana. Conversa sem fala do cliente devolve `null` (não avaliada), nunca lista vazia. `GET /conversas?sinalizador=<regra>` filtra e só então pagina. Limites em Limitações.

### 2.2.11 Impacto por tema e deterioração

`GET /operacao/radar` traz, em cada tema, o impacto no NPS inferido com IC por bootstrap (`fraus/impacto.py`): associação, não causa, nulo abaixo de 30 conversas com sinal em cada lado. `GET /operacao/replay/{id}` traz o início retrospectivo da deterioração (`fraus/deterioracao.py`, campo `indice_virada`) sobre os scores por prefixo que o replay já calcula: derivada na leitura, nada gravado, nunca alerta ao vivo. Limites em Limitações.

## 2.2.12 Onde cada coisa mora

| CAMINHO | RESPONSABILIDADE |
|---|---|
| `fraus/modelos.py` | `Conversa` / `Mensagem` — o modelo canônico |
| `fraus/ingest/` | adaptadores de entrada, um por formato |
| `fraus/sinais/` | as oito famílias de sinal |
| `fraus/sinais/curadoria.py` | o que o analista ensinou ao léxico — vence o SentiLex |
| `fraus/fusor.py` | `NOMES_FEATURES` (39) e o `Fusor` |
| `fraus/indicadores.py` | NPS, CSAT, contenção, falso containment, série |
| `fraus/explicacao.py` | frases por template a partir da decomposição do fusor, e os drivers agregados |
| `fraus/evidencia.py` | evidência fraca — a cabeça tracejada |
| `fraus/deriva.py` | deriva de distribuição — a invariante 10 em runtime |
| `fraus/cortesia.py` | "ok, obrigado" sozinho é sem sinal, não promotor |
| `fraus/api/` | FastAPI: `criar_app` (fábrica) e as rotas por domínio |
| `fraus/api/contexto.py` | faixas, curadoria e régua vigentes, lidas por requisição |
| `fraus/api/repontuacao.py` | repontuar em segundo plano, com progresso e 409 se já rodando |
| `fraus/api/vazao.py` | teto de `/auth/*` por IP e de `/ingestao` por fonte |
| `dashboard/` | Next.js — ver Design |

## 2.3 TREINAMENTO DOS MODELOS DO FRAUS

### 2.3.1 Receita vigente de pesquisa — 26/10/2026

> **CAMPANHA DE 26–27/10/2026 CONCLUÍDA, SEM PROMOÇÃO**
>
> As nove rodadas (três cabeças × três sementes) foram executadas em T4 pela CLI oficial do Google Colab, com nove bundles recuperados. O candidato do fusor e as ablações também foram avaliados. A sessão própria foi encerrada; nenhum peso ativo foi substituído. Publicar site/API é uma operação separada. Generalização e evidências centraliza resultados, curvas e limites: há sobreajuste nesta reprodução; os pesos antigos continuam sem diagnóstico conclusivo.

O contrato de inferência continua sendo **39 features de sete famílias**, quatro esperas em `log1p`, `StandardScaler` e `LogisticRegression`. Ironia é obrigatória na leitura por mensagem do motor, mas não entra no vetor. As três cabeças e um fusor compatível são necessários para predizer; a API pode subir antes da carga, sem oferecer fallback silencioso.

#### SELEÇÃO SEM ABRIR O TESTE

`fraus/receita_treino.py` centraliza a receita dos notebooks 01, 03 e 04:

1. Preparar divisão agrupada, deduplicada e auditada, com semente 42: treino/validação/teste em 75/10/15; MultiPICo em 60/15/25.
2. Executar sementes **13, 42 e 77**, até cinco épocas, LR `2e-5`, lote 32, `max_length=192`, warmup 10%, weight decay `0,01` e early stopping de paciência 2. A satisfação usa o treino B2W completo após a divisão, sem teto arbitrário de exemplos e sem ConsumerBR auxiliar.
3. `treinar_uma_semente` ajusta modelo e temperatura usando **treino e validação apenas**. A seleção de checkpoint é por F1-macro de validação; não há inferência nem métrica de teste nessa fase.
4. Selecionar a semente mediana por `f1_macro_validacao` e gravar configuração e seleção congeladas **antes** de abrir o teste.
5. Só então chamar `avaliar_teste_final` para as três sementes. O fingerprint deve coincidir com o manifesto esperado; divergência bloqueia a avaliação. Reportar média e desvio amostral sem trocar a semente escolhida na validação.

O `fraus_receita.json` preserva argumentos efetivos, melhor época, manifestos e histórico por época. A curva diagnóstica usa até 2.000 exemplos fixos de treino, estratificados com seed 42, em modo de avaliação, com a mesma perda ponderada da validação. Não se interpreta loss de minibatches com dropout como se fosse a mesma medida. `avaliacao_final.json` registra separadamente teste e hashes; exportar pesos, tokenizador, curvas, receitas e manifestos antes de descartar checkpoints.

NOTEBOOKS E ESCOPO DA CAMPANHA

| # | NOTEBOOK | PAPEL ATUAL | SITUAÇÃO |
|---|---|---|---|
| 1 | `notebooks/01_treino_bertimbau.ipynb` | candidato de satisfação, três classes, BERTimbau sobre B2W por estrelas | três sementes GPU concluídas; escolhida 42 pela validação |
| 2 | `notebooks/02_treino_fusor.ipynb` | candidato de fusor, manifesto explícito e caches vinculados aos artefatos | experimento via CLI local concluído; completo/texto/tempo, sem promoção |
| 3 | `notebooks/03_treino_emocao.ipynb` | candidato de emoção, sete classes, GoEmotions-PT | três sementes GPU concluídas; escolhida 13 pela validação |
| 4 | `notebooks/04_treino_ironia.ipynb` | candidato binário de ironia, **MultiPICo PT, sem IDPT auxiliar** | três sementes GPU concluídas; escolhida 42 pela validação |
| 5 | `notebooks/05_treino_multitarefa.ipynb` | candidato ONNX com encoder compartilhado e três cabeças | fora desta campanha; não promovido |
| 6 | `notebooks/06_exportar_laya_onnx.ipynb` | exportação de Laya para ONNX INT8 | fora desta campanha; não implica uso em produção |
| 7 | `notebooks/07_treino_laya.ipynb` | comparação histórica de Laya com BERTimbau | histórico de 02/10/2026; não promovido |

01, 03 e 04 são independentes entre si; o 02 usa as cabeças disponíveis para gerar sinais, mas somente texto e emoção são exigidos por `montar_features`. O motor completo também exige ironia. O léxico é recurso determinístico e versionado, não uma quarta cabeça treinada.

Na execução pela CLI oficial, os notebooks aceitam `FRAUS_RAIZ`, `FRAUS_REPO` e `FRAUS_DADOS` para caminhos explícitos, sem montagem interativa do Drive. O modo interativo continua disponível. Todos os destinos de pesquisa são diretórios de **candidatos**, separados dos pesos ativos; executar a receita não autoriza promoção ou publicação.

A fonte da campanha foi congelada em `03f91770ec92dec934d794cbad50ea3224f4cb8a`. Como o upload não levou `.git`, o commit interno dos notebooks é nulo; `manifesto-bundles.json` externo vincula os três manifestos de código, comparados integralmente ao `git archive` dessa fonte. Bundles, receitas individuais, logs, corpora e pesos não são publicados no Git/site. As médias/desvios amostrais, ECE bruto/calibrado e evidências por época permanecem em uma única tabela de referência em Generalização, sem confundir a mediana do teste com a semente escolhida pela validação.

AUDITORIA E FUSOR COM MANIFESTO

```
# Caminhos de pesquisa escolhidos pelo operador; saída fora do site público.
uv run python scripts/auditar_generalizacao.py \
  --corpora dados/corpora --modelos modelos --saida resultados/auditoria.json
```

O relatório agregado e o manifesto privado registram normalização, grupos, rótulos, hashes e colisões, sem incorporar texto bruto às docs. Reconstruir splits hoje não comprova a origem de pesos antigos.

O treino local do fusor exige `--manifesto`: não há mais divisão posicional implícita. O cache é vinculado ao manifesto, checkpoints e contrato de features; um nome de arquivo igual não autoriza reutilização. As ablações completo, somente texto e somente tempo ajustam seus scalers no treino e são avaliações experimentais, não modelos de runtime. O protocolo completo, incluindo a independência supervisionada das cabeças, está em Generalização.

Reserve o manifesto **antes do notebook 01**, que o recebe em `FRAUS_MANIFESTO_FUSOR`. As variáveis abaixo representam o CSV exato (`B2W`), um diretório externo ao repositório/checkpoints (`EXPERIMENTO`) e a raiz dos checkpoints candidatos (`CABECAS`). A segunda chamada só acontece após o treino e o congelamento dos candidatos:

```
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" --gerar-manifesto
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" \
  --modelos "$CABECAS" --backend torch \
  --cache "$EXPERIMENTO/features.json" --saida "$EXPERIMENTO/candidato"
```

O manifesto versão 1 registra hash do CSV, contagem aceita pelo leitor, divisão agrupada 42 e, por lado, índices aceitos base zero, hashes de texto/grupo e rótulos de satisfação. O alvo do fusor continua sendo derivado da recomendação com precedência do neutro de três estrelas, não desses rótulos da cabeça.

## 2.3.2 Diário histórico — não é a receita de execução vigente

O restante desta página preserva decisões, medições e procedimentos de suas respectivas datas. Menções a 16, 35, 38 ou 40 features, IDPT como padrão, cópia para diretório ativo, seleção antiga e métricas de holdouts antigos **não substituem o protocolo acima**. A acurácia histórica 0,947 não identifica o checkpoint publicado hoje. Pesos globais e sanidades isoladas não provam ausência de vazamento ou de overfitting. Os números do Cartão do modelo continuam identificados como históricos.

## 2.3.3 Como abrir no Colab

1. Va em colab.research.google.com.
2. `Arquivo > Fazer upload de notebook` e selecione o notebook (ou abra direto do GitHub se o repositorio estiver acessivel a partir da sua conta).

## 2.3.4 Notebook 01 — classificador de texto

### EXIGENCIA DE GPU

O notebook so funciona com acelerador de GPU habilitado. Antes de rodar qualquer celula:

`Ambiente de execucao > Alterar tipo de ambiente de execucao > GPU`

A primeira celula do notebook faz `assert torch.cuda.is_available()` e falha com uma mensagem explicita se o ambiente de execucao estiver sem GPU.

### ONDE O CHECKPOINT FICA NO DRIVE

O notebook monta o Google Drive (`drive.mount('/content/drive')`) e usa dois diretorios dentro dele, para sobreviver a uma queda de sessao do Colab — o mesmo padrao ja usado no notebook de RVC do projeto Neuro-ai:

- `/content/drive/MyDrive/fraus/checkpoints` — checkpoints intermediarios salvos pelo `Trainer` a cada epoca (`save_strategy="epoch"`, `save_total_limit=2`). Se a sessao cair, o proximo `Trainer.train()` pode retomar a partir do ultimo checkpoint salvo aqui.
- `/content/drive/MyDrive/fraus/modelos/bertimbau-satisfacao` — destino final do modelo treinado, do tokenizador e do `metricas.json`, escritos na ultima celula do notebook.

### DESTINO DO ARTEFATO — PROCEDIMENTO HISTÓRICO

A receita antiga orientava copiar o resultado para `modelos/bertimbau-satisfacao/`, o diretório consumido pela inferência. Isso **não é instrução para a campanha atual**: manter os resultados em diretório de candidatos e preservar pesos, tokenizador, `fraus_receita.json`, histórico, manifestos e `avaliacao_final.json` como um bundle identificado. `modelos/` não é versionado no Git; ter arquivos locais não comprova a cadeia de proveniência de um checkpoint antigo nem autoriza substituir o ativo.

### MAPEAMENTO DE ROTULO

O rotulo de satisfacao vem do campo `overall_rating` (nota de 1 a 5) do B2W-Reviews01:

| `OVERALL_RATING` | ROTULO | INDICE |
|---|---|---|
| 1–2 | insatisfeito | 0 |
| 3 | neutro | 1 |
| 4–5 | satisfeito | 2 |

O corpus e balanceado por subamostragem da classe majoritaria antes do split de treino/teste, porque as notas 4-5 dominam o corpus e enviesariam o classificador.

## 2.3.5 Notebook 02 — fusor dos tres sinais

Roda DEPOIS do 01, sem GPU. O que ele faz, em sequencia:

1. monta o Drive e confere `modelos/bertimbau-satisfacao` (falha alto apontando o notebook 01 se faltar);
2. clona este repositorio e instala o pacote `fraus` — a extracao de features usa o MESMO codigo da API (`fraus.fusor.montar_features`), nunca uma reimplementacao;
3. carrega o B2W-Reviews01 e rotula por `recommend_to_a_friend` (ver abaixo);
4. costura as frases em conversas sinteticas com `fraus.ingest.simulador.gerar_lote`, deterministico por semente, com latencia log-normal e emoji calibrados por rotulo;
5. extrai as features de cada conversa com o BERTimbau do notebook 01 carregado — hoje sao **39** (ver Contrato de features);
6. treina o `Fusor` (`treinar(exemplos, rotulos)`);
7. avalia num conjunto de teste separado — conversas geradas com outra semente e a partir de frases disjuntas — imprimindo acuracia e F1-macro;
8. exporta `fusor.joblib` (via `Fusor.salvar`) e `importancias.json` (o retorno de `Fusor.importancias()`, que vira o grafico "qual sinal pesou mais" da apresentacao).

O destino no Drive descrito nesta seção era o procedimento histórico. Na campanha vigente, exportar para um bundle de candidato, sem copiar para o diretório ativo e sem promoção.

### ROTULO DO FUSOR: `RECOMMEND_TO_A_FRIEND`

Esta e a etapa em que o campo `recommend_to_a_friend` do B2W-Reviews01 entra. A spec (secao 5.1) o declara como a **ancora do NPS inferido** — "sem ele, o NPS da dashboard seria metrica inventada" — e o alvo do fusor e justamente a categoria de NPS que a dashboard exibe. O notebook 01 usou `overall_rating` porque treinava sentimento de texto; o 02 usa a recomendacao porque treina NPS.

| CAMPO | VALOR | CLASSE | INDICE |
|---|---|---|---|
| `recommend_to_a_friend` | `Yes` | satisfeito | 2 |
| `recommend_to_a_friend` | `No` | insatisfeito | 0 |
| `overall_rating` | `3` | neutro | 1 |

**Por que a linha do neutro existe.** `recommend_to_a_friend` e binario, e as classes do projeto sao tres. Um fusor de duas classes zeraria o termo do neutro no score (`100 * (P(satisfeito) + 0.5 * P(neutro))`) e empurraria todo atendimento morno para uma das pontas, inflando o NPS nos dois sentidos. A alternativa seria treinar em duas classes e recortar o neutro por uma faixa de score — mas isso trocaria um rotulo observado por um limiar arbitrado a mao depois do treino. Aqui o neutro e `overall_rating == 3`, um rotulo do corpus, e a nota 3 tem precedencia sobre a recomendacao para que a mesma resenha nao caia em duas classes. As duas pontas — as unicas que contam no NPS, ja que neutros nao entram na formula — seguem ancoradas em `recommend_to_a_friend`.

As tres classes sao subamostradas ao tamanho da menor (a nota 3 e minoria no corpus), o mesmo balanceamento do notebook 01.

## LIMITACAO METODOLOGICA A DECLARAR NO RELATORIO

O fusor e treinado sobre conversas SINTETICAS: nenhum corpus publico de resenha PT-BR tem timestamps de dialogo, entao a latencia, a escalacao e o abandono saem de distribuicoes calibradas pela literatura de live chat, nao de atendimento observado. O texto e real (B2W), o tempo nao e.

As metricas do notebook 02 continuam OTIMISTAS mesmo depois da correcao descrita abaixo: elas medem que o fusor aprendeu a combinar os sinais no dominio sintetico, nao que ele acertaria essa taxa em atendimento real.

## O VAZAMENTO DO PRIMEIRO FUSOR — 13/08/2026

O primeiro fusor treinado marcou **99,3% de acuracia e F1-macro 0,993**, e era inutil. Vale registrar porque o modo de falha e silencioso e pode voltar.

**O sintoma.** Carregado no repositorio local e alimentado com conversas escritas a mao, o fusor devolvia praticamente a mesma nota para tudo:

| | LATENCIA 5S | 30S | 200S |
|---|---|---|---|
| texto otimo (BERTimbau: 0,96 satisfeito) | 52,48 | 51,32 | 50,02 |
| texto pessimo (BERTimbau: 0,98 insatisfeito) | 49,94 | 49,92 | 49,84 |

Dois textos que o classificador de texto separa com mais de 95% de confianca saiam com **2,5 pontos** de diferenca, ambos colados em 50 e ambos classificados como neutro.

**A causa.** O `PERFIL_POR_ROTULO` do simulador amostrava latencia de faixas **disjuntas** — insatisfeito 60-400s, neutro 15-60s, satisfeito 3-15s. Sem sobreposicao nenhuma, a latencia *era* o rotulo: dava para acertar a classe olhando so o relogio, sem ler uma letra do texto. A regressao logistica fez exatamente isso. Os quatro maiores pesos aprendidos eram todos de tempo (`latencia_p90_s` 2,06 · `latencia_mediana_s` 2,03 · `latencia_primeira_resposta_s` 1,91 · `duracao_total_s` 1,52), contra 1,39 e 1,12 das features de texto. Fora da variedade que ele decorou, o modelo respondia "neutro" para tudo.

Os 99,3% nao mediam fusao. Mediam que o simulador vazava o gabarito.

**Um segundo achado, do mesmo diagnostico.** As features `emoji_score_medio`, `emoji_frac_positivos` e `emoji_frac_negativos` tinham coeficiente **0,0000**: o simulador nunca emitia emoji, entao elas eram constantes no treino e um dos tres sinais da arquitetura nao contribuia nada.

**A correcao** (`fraus/ingest/simulador.py`, coberta por `tests/test_simulador.py`):

- a latencia passou a ser **log-normal** com caudas que se cruzam. As medianas continuam ordenadas, como manda a literatura de live chat, mas ~11% das conversas satisfeitas sao mais lentas que a mediana das insatisfeitas, e vice-versa. Existe atendimento rapido que termina mal;
- as falas do cliente passaram a levar **emoji do lexicon**, com cruzamento deliberado — cliente irritado manda 🙏 (+0,418), cliente satisfeito manda 😩 (-0,368) — para o emoji nao virar o proximo gabarito.

**Como saber se voltou.** Acuracia alta e sintoma que pede investigacao, nao veredito. Depois da correcao o notebook 02 marcou **0,96**, e desta vez o numero se sustentou: os pesos aprendidos mudaram de lugar.

| FEATURE | FUSOR VAZADO | FUSOR CORRIGIDO |
|---|---|---|
| `emoji_score_medio` | **0,0000** (morto) | **1,6662** (1º) |
| `texto_prob_satisfeito_media` | 1,3858 | 1,6026 (2º) |
| `texto_prob_insatisfeito_media` | 1,1212 | 1,4722 (3º) |
| `latencia_p90_s` | **2,0575** (1º) | 0,0674 |
| `latencia_mediana_s` | 2,0329 | 0,1621 |
| `latencia_primeira_resposta_s` | 1,9068 | 0,0138 |

O mesmo teste de sanidade que expos o vazamento — mesmo texto, so mudando o relogio — passou de 2,5 pontos de separacao para **99,6**: texto otimo pontua 99,98 e texto pessimo 0,35, com a latencia mexendo fracoes de ponto. O sinal de tempo virou o modificador fraco que ele deve ser.

**O criterio, entao, nao e o numero sozinho.** Diante de acuracia alta, olhe os PESOS: se as features que lideram forem as que carregam o conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais (tempo, contagem de turnos), procure o vazamento.

## O FUSOR DE 35 FEATURES — 24/08/2026

O contrato subiu de 16 para 35 features em 21/08 (emocao, lexico, ironia e estilo entraram no vetor) e o **artefato ficou tres dias para tras**. Nesse intervalo a API real nao subia: o `StandardScaler` de 16 rejeitava o vetor de 35 na carga. Falha alta e explicita, que e o comportamento que as invariantes 7 e 9 pedem — mas ainda assim tres dias de API parada por artefato desatualizado.

O notebook 02 rodou de novo e o artefato daquele momento passou a ter `n_features_in_ = 35`, na ordem exata de `NOMES_FEATURES`. As seções posteriores registram a passagem por 38 e pelo contrato vigente de 39 features.

**A acuracia caiu de 0,96 para 0,93, e a queda e o resultado saudavel.** O fusor de 16 features media um problema mais facil. Dezenove features novas entraram, tres delas vindas de uma cabeca de ironia com

vazamento de corpus conhecido e nao consertado — um numero MENOR e o esperado. Pela regra desta secao, o que decide nao e o numero e sim onde os pesos foram parar:

| FEATURE | FUSOR DE 16 | FUSOR DE 35 |
|---|---|---|
| `texto_prob_satisfeito_media` | 1,6026 (2º) | **1,6572 (1º)** |
| `texto_prob_insatisfeito_media` | 1,4722 (3º) | 1,3867 (2º) |
| `emoji_score_medio` | **1,6662 (1º)** | 1,3354 (3º) |
| `emoji_frac_positivos` | 0,8430 | 0,8098 |
| `emocao_surpresa_media` | — | 0,6655 |
| `emocao_desprezo_derivado` | — | 0,4757 |
| `latencia_mediana_s` | 0,1621 | 0,1200 |
| `latencia_p90_s` | 0,0674 | 0,1419 |
| `duracao_total_s` | 0,4964 | 0,3397 |

O topo continua sendo **texto e emoji**, e as features de tempo continuam no rodape, onde devem estar. As de emocao entram no meio da tabela com peso real sem deslocar o conteudo — `desprezo_derivado`, que e a diade calculada e nao uma classe do modelo, puxa 0,48. Nao ha sinal de vazamento novo.

**A divida que este fusor assume.** Ele foi treinado com a cabeca de ironia vazando (ver a pendencia 1 do README: 6 em 10 frases sinceras saem marcadas como ironicas). `ironia_prob_media` e `ironia_prob_max` pesam 0,30 e 0,31 — nao e desprezivel. **Retreinar a ironia obriga a retreinar o fusor de novo**, e essa ordem esta registrada aqui para nao se perder.

**Teste de sanidade ponta a ponta**, com os tres BERTimbau carregados, sobre conversas do simulador: rotulo insatisfeito pontua ~0, neutro ~73–75, satisfeito ~99, e as tres categorias de NPS saem certas em 9 de 9. **Isso nao e evidencia de qualidade** — sao conversas do proprio gerador sintetico, o dominio em que o fusor foi treinado. O numero honesto continua sendo 0,93 no conjunto de teste separado, e a ressalva do topo desta secao continua valendo: as metricas do notebook 02 medem fusao no dominio sintetico, nao acerto em atendimento real.

## CONSEQUENCIA NO NPS: A CLASSE NEUTRA CAI EM DETRATOR — HISTORICO (FUSOR DE 16 FEATURES, ATE 24/08/2026)

**Esta secao mede o fusor de 16 features, NAO o vigente.** Ela ficou aqui sem data enquanto o artefato foi trocado, e passou a ler como se descrevesse o fusor atual — que e o defeito mais caro deste projeto. Fica registrada porque o raciocinio sobre o score continua valendo, e porque a medicao dela e o contraponto do que o fusor de 35 features entrega.

**A consequencia DEIXOU de valer com o fusor de 35 features.** Base: o teste de sanidade ponta a ponta da secao anterior, com os tres BERTimbau carregados — rotulo neutro pontua **~73–75**, cai na faixa 7–8 e as tres categorias de NPS saem certas em 9 de 9. Ou seja, o neutro nao cai mais em detrator. Ressalva do mesmo

tamanho: aquele teste corre sobre conversas do proprio simulador, o dominio em que o fusor foi treinado, entao ele desmente a medicao abaixo no dominio sintetico e nao promete nada sobre atendimento real.

Medido em 90 conversas do simulador, 30 por classe, o fusor corrigido separa bem — medianas **0,11 / 50,99 / 99,03** por rotulo verdadeiro. Mas a categoria de NPS derivada sai **67% detrator · 29% promotor · 4% neutro**, com **NPS -38** num lote equilibrado por construcao.

Nao e falha do treino. O score e `100 * (P(satisfeito) + 0.5 * P(neutro))`, entao conversa classificada com certeza como neutra pontua 50, vira nota 5 e cai na faixa 0-6. A faixa neutra do NPS (7-8) so seria alcancada com `P(satisfeito)` entre 0,4 e 0,8 — um empate entre classes, nao uma neutralidade confiante. As tres classes do modelo e as tres categorias do NPS foram decididas em lugares separados e nunca se encontraram.

**Decisao de projeto: manter e declarar.** A alternativa avaliada era subir o peso do neutro de 0,5 para 0,75 (o centro da faixa passiva do NPS), o que faria neutro puro pontuar 75, virar nota 8 e cair em neutro — alinhando as tres classes as tres categorias. Ficou registrada aqui e na docstring de `Fusor.pontuar` para nao ser "corrigida" por engano.

## RETREINO DO FUSOR APOS AS FEATURES DE INCONGRUENCIA (03/09/2026)

O contrato subiu de 35 para 40 features: a familia `incongruencia_*` (`incongruencia_polaridade`, `incongruencia_emoji_texto`, `incongruencia_marcador_contraste`, `incongruencia_hiperbole`, `incongruencia_aspas_ironicas`) entrou em `NOMES_FEATURES`, anexada ao FIM da lista para preservar a ordem das 35 antigas. O fusor salvo em `modelos/` foi treinado com 35 e `vetorizar` agora produz 40 — ele NAO serve mais. `Fusor.carregar` (`joblib.load` puro, sem validacao de forma) carrega esse artefato sem erro nenhum -- a API real SOBE com ele. O erro de dimensao no `StandardScaler` so aparece na primeira pontuacao de verdade (`/ingestao` ou `/conversas/importar`), como HTTP 500, nao na carga do servidor. Isso e risco de demonstracao ao vivo, nao uma trava de subida -- ver a tabela do Fusor no README para o detalhe medido. A intencao da invariante 7 (falha alta e explicita) so se cumpre na metade "explicita"; a metade "na carga" fica para o retreino resolver o descompasso, nao para uma validacao que este codigo ainda nao tem.

**O corpus sintetico tambem mudou, e isso importa mais do que a contagem de features.** `FRASES_POR_ROTULO`, em `fraus/ingest/simulador.py`, ganhou frases novas em duas levas para curar dois vazamentos de rotulo — o mesmo tipo de falha silenciosa que ja custou o primeiro fusor (ver acima), so que desta vez em features que ainda nao estavam no vetor:

- **Intensificadores.** Antes, `incongruencia_hiperbole` era nao-zero na maior parte das conversas do rotulo satisfeito e ZERO nos rotulos insatisfeito e neutro — um previsor unilateral quase perfeito de "satisfeito". A literatura inclui hiperbole na deteccao de ironia justamente porque elogio hiperbolico e a forma classica da ironia, entao o fusor teria aprendido a feature ao contrario do que ela significa. Depois da correcao, medido com `gerar_lote(FRASES_POR_ROTULO, 180, semente=7)` — o mesmo lote e semente da guarda `test_nenhuma_feature_e_previsor_unilateral` em `tests/test_simulador.py`: `insatisfeito 34/60 | neutro 0/60 | satisfeito 39/60`.
- **Negacao.** `lexico_frac_negados` era nao-zero so no rotulo insatisfeito e ZERO nos outros dois — previsor unilateral de "insatisfeito", pre-existente desde antes da familia `incongruencia_*` e nunca notado por falta de guarda na familia `lexico_*`. Depois, medido com o mesmo lote e semente acima: `insatisfeito 15/60 | neutro 17/60 | satisfeito 21/60`.

**Consequencia pratica: o notebook 02 precisa REGERAR o corpus sintetico, nao reusar um lote cacheado de uma execucao anterior.** O notebook 02 ja gera o lote do zero a cada execucao — `gerar_lote` roda

na propria celula, com semente fixa, e nao le nenhum artefato salvo de rodada passada — entao rodar o notebook de novo, sem alterar nada, ja traz o corpus corrigido. O risco e so para quem tiver guardado um `.csv`/`.pkl` de conversas de uma sessao antiga do Colab e tentar reaproveitar: um lote assim traz os dois vazamentos de volta.

**O notebook ja importa o contrato do pacote, nao repete a lista.** A celula marcada com o comentario `# 5. Os SETE sinais de cada conversa, pelo MESMO codigo que a API usa` (indice 8 do `.ipynb`, contando a partir de 0) faz `from fraus.fusor import NOMES_FEATURES, montar_features` e chama `montar_features` com os tres classificadores — as cinco features novas entram sozinhas, sem editar o notebook. Isso ja era assim antes desta rodada (o vazamento do ramo obsoleto registrado na celula 2 foi outra causa, a de importar de um ramo desatualizado do repositorio, nao de reimplementar a lista) — nao ha lista duplicada para divergir aqui.

Procedimento histórico de setembro, preservado como registro, não roteiro da campanha atual:

1. Conferir que a celula do clone do repositorio (`RAMO`) aponta para `main` com a Task 5 mergeada, e rodar o notebook do inicio — sem tentar reaproveitar nenhum lote de conversas salvo de uma execucao anterior.
2. Antes de subir ao Colab (ou depois, localmente), rodar as guardas de vazamento: `uv run pytest tests/test_simulador.py -k unilateral`. Elas cobrem as familias `lexico_*`, `emoji_*`, `estilo_*` e `incongruencia_*` contra previsor unilateral e precisam estar verdes antes do treino valer a pena.
3. Conferir a acuracia contra a do fusor de 35 features (0,93). **Acuracia alta demais e sintoma, nao vitoria** — se saltar muito acima do valor anterior, suspeite de vazamento antes de comemorar, releia a secao "O vazamento do primeiro fusor" acima e confira os PESOS, nao so o numero: se as features que lideram forem as de conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais, procure o vazamento.
4. Conferir os coeficientes das cinco features novas em `Fusor.importancias()` / `Fusor.eixo_global()`.

**CORRECAO DE 04/09/2026, e a licao vale mais que o numero:** este passo dizia para esperar peso proximo de zero em `incongruencia_marcador_contraste` e `incongruencia_aspas_ironicas`, porque as duas sao constantes em zero no corpus do SIMULADOR. A previsao estava errada, e o erro foi confundir dois corpora diferentes. O simulador fornece a ESTRUTURA temporal (latencia, turnos, escalacao) a partir de seis frases por rotulo; o TEXTO do treino e o B2W-Reviews01, resenha real de e-commerce, onde marcador de contraste e aspas ironicas ocorrem normalmente. Medir uma feature de texto no gerador de estrutura e olhar para o corpus errado.

Os pesos reais do primeiro retreino de 40 features, no eixo satisfeito-menos-insatisfeito:

| FEATURE | PESO |
|---|---|
| `incongruencia_emoji_texto` | −0,95 |
| `incongruencia_hiperbole` | −0,65 |
| `incongruencia_marcador_contraste` | −0,38 |
| `incongruencia_polaridade` | −0,37 |
| `incongruencia_aspas_ironicas` | −0,15 |

As cinco sairam NEGATIVAS, que e a direcao que a literatura preve: mais incongruencia empurra a nota para insatisfeito. A hiperbole em −0,65 e a evidencia de que o cruzamento do corpus (secao acima) funcionou — sem ele ela teria aprendido o sinal contrario.

O que continua valendo da versao anterior deste passo: as guardas de vazamento medem o SIMULADOR e sao sobre a estrutura, nao sobre o texto do B2W. `incongruencia_hiperbole` melhorou mas nao chegou aos tres: depois da correcao ela dispara em DOIS dos tres rotulos — `insatisfeito 34/60` e `satisfeito 39/60` (numeros medidos acima, na secao do retreino) —, e `neutro` continua em `0/60`. Isso e limitacao declarada, nao vazamento remanescente: fala neutra raramente intensifica termo polar com hiperbole, e a decisao foi nao forcar uma frase artificial no gerador so para a classe deixar de ficar muda. A guarda `test_nenhuma_feature_e_previsor_unilateral` (`tests/test_simulador.py`) tolera isso por construcao — ela so falha quando a feature dispara em EXATAMENTE UM rotulo, entao 2-de-3 passa verde. Isso e posicao aceita sobre o corpus sintetico atual, nao descuido: fica registrado aqui para o proximo leitor nao achar que a guarda promete "sinal nos tres rotulos" quando ela so promete "nao previsor unilateral de UM rotulo so". A guarda nao foi alterada. 5. A etapa histórica substituía os arquivos do diretório ativo e conferia o artefato com `scripts/conferir_fusor.py` e a suíte de testes. **Não repetir essa substituição na campanha atual**: candidatos, caches e resultados ficam em destino externo, com manifesto explícito, sem promoção.

## A IRONIA SAI DO VETOR (04/09/2026)

Medicao sobre o proprio corpus de treino do fusor (B2W-Reviews01, amostra equilibrada de 500 resenhas por rotulo, semente 20260904), rodando so a cabeca de ironia:

```
resenha NEGATIVA (nao): media P(ironia) = 0,1012  mediana 0,0013  frac(P>0,5) = 0,092
resenha POSITIVA (sim): media P(ironia) = 0,7146  mediana 0,9687  frac(P>0,5) = 0,740
                                    diferenca (sim-nao) = +0,6134
```

A cabeca, treinada no IDPT 2021 (tweet e noticia), aplicada a resenha de e-commerce funciona na pratica como **detector de sentimento positivo**: marca 74% das resenhas satisfeitas como ironicas contra 9% das insatisfeitas. O fusor de 40 features aprendeu **+0,77** de peso para `ironia_prob_media` no eixo satisfeito-menos-insatisfeito — mais ironia empurrando para SATISFEITO, o inverso do que o nome da feature promete. O fusor de 35 ja tinha o mesmo sinal (+0,48), entao nao e regressao desta rodada: e o mesmo vazamento medido de novo, agora com numero.

Consequencia observavel: a frase canonica de ironia deste projeto — "que atendimento maravilhoso, so esperei 3 horas" — pontuava **99,98 / nota 10 / promotor** no fusor de 40, porque as duas features de ironia empurravam para cima em vez de para baixo. (A cabeca isolada acerta essa frase: 0,998 nela, 0,002 num elogio sincero. Quem lia ao contrario era o fusor, e so por causa do corpus.)

**Decisao:** `ironia_prob_media` **e** `ironia_prob_max` **saem de** `NOMES_FEATURES`**.** Feature que mede outra coisa que nao o nome dela e pior que feature ausente — ela nao adiciona sinal, duplica `texto_prob_satisfeito *` com ruido, e inverte o caso que o projeto usa como exemplo de manual. A cabeca de ironia CONTINUA carregada e obrigatoria (invariante 7) e continua sendo lida por mensagem e exibida na dashboard, onde e honesta e ja vem com a ressalva de confiabilidade que a tela mostra — o que muda e que ela para de pontuar. O retreino com 38 features foi executado no mesmo dia; ver a secao seguinte.

**Nota sobre a janela entre a mudanca do contrato e o artefato novo.** Enquanto `modelos/fusor.joblib` era o artefato de 40 e o contrato ja estava em 38, a API **nao subia** — `Fusor.carregar` passou a comparar `n_features_in_` com `len(NOMES_FEATURES)` e levantar `FusorIncompativelError` (PR #29, 04/09/2026). Antes dessa validacao, o mesmo desencontro deixava a API subir normalmente e estourar HTTP 500 so na primeira pontuacao real — falha tardia, no pior momento possivel. A validacao troca isso por falha alta e explicita na subida, que e o que a invariante 7 pede.

## O FUSOR DE 38 FEATURES — 04/09/2026

Notebook 02 rodado de novo com a ironia fora do vetor. **Acuracia 0,9433** (f1_macro 0,9436), contra 0,9467 do fusor de 40. A queda de ~0,003 e o resultado saudavel e esperado: as duas features removidas *estavam* ajudando o classificador — so que como detectores de sentimento positivo duplicando `texto_prob_satisfeito_*`, nao como medida de ironia. Perder um pouco de acuracia trocando uma feature que mente por nenhuma feature e o negocio certo.

Conferido com `uv run python scripts/conferir_fusor.py` (ver secao "Conferencia do artefato" abaixo): contrato 38/38, `classes_ = [0, 1, 2]` (invariante 8), nenhuma feature com peso perto de zero, e as cinco de incongruencia mantiveram a direcao negativa que a literatura preve:

| FEATURE | FUSOR DE 40 | FUSOR DE 38 |
|---|---|---|
| `incongruencia_emoji_texto` | −0,95 | −1,03 |
| `incongruencia_hiperbole` | −0,65 | −0,64 |
| `incongruencia_polaridade` | −0,37 | −0,37 |
| `incongruencia_marcador_contraste` | −0,38 | −0,27 |
| `incongruencia_aspas_ironicas` | −0,15 | −0,19 |

**Tres pesos com sinal aparentemente invertido, e por que NAO sao vazamento.** O laudo sinaliza `texto_prob_satisfeito_ultima` (−0,50), `emoji_frac_positivos` (−0,18) e `emoji_frac_negativos` (+0,60) como suspeitos, porque o nome promete o sinal contrario. Conferidos contra os artefatos de 40 e de 35, os tres ja estavam assim nos dois — nao e efeito da remocao da ironia. A explicacao e **colinearidade**: `texto_prob_satisfeito_media` pesa +2,81 e `emoji_score_medio` +1,56, e features fortemente correlacionadas com elas recebem coeficiente negativo de correcao na regressao logistica. E o comportamento normal do modelo com features redundantes, nao inversao de semantica como a de `ironia_prob_media`. A diferenca entre os dois casos e verificavel: a ironia foi medida diretamente no corpus e provou medir outra coisa; estas tres continuam medindo o que o nome diz, so nao carregam informacao independente das dominantes. Fica registrado aqui porque o laudo vai sinalizar as tres em todo retreino futuro, e o proximo leitor precisa saber que ja foram investigadas.

**O que este retreino NAO resolveu.** A frase canonica do projeto continua pontuando alto:

```
"que atendimento maravilhoso, so esperei 3 horas"   score 99,95  nota 10
"otimo, mais uma vez ninguem resolveu nada"         score 99,74  nota 10
"resolveu rapido, muito obrigado! adorei"           score 97,82  nota 10
```

As cinco features de incongruencia dao **0,0 nas duas frases ironicas** e disparam na satisfeita de verdade (`polaridade` 0,5, `hiperbole` 0,5) — ou seja, hoje elas pegam entusiasmo genuino e nao pegam ironia. Nao e regressao: e o limite estrutural das cinco, todas funcao de `anotar_texto`/emoji, e "so esperei 3 horas" nao tem nenhuma palavra polar no SentiLex — e negativo por pragmatica. O diagnostico completo e a rota proposta (`incongruencia_situacao_negativa`, que levaria o vetor a 39 e exige mais um retreino) estao em `docs/superpowers/specs/2026-09-04-incongruencia-implicita-design.md`.

Corrigindo a previsao registrada na secao anterior: esperava-se que a frase deixasse de ser empurrada para cima com a saida da ironia. Nao foi o que aconteceu — o `texto_prob_satisfeito_media` da propria cabeca de texto e 0,858 nela, e domina o eixo sozinho. A remocao consertou o vetor; nao tocou nesta frase.

## O CONTRATO SOBE PARA 39: INCONGRUENCIA IMPLICITA (04/09/2026)

`incongruencia_situacao_negativa` entrou em `NOMES_FEATURES` -- elogio convivendo com situacao negativa de atendimento na mesma fala. E a unica das seis que alcanca a frase canonica do projeto, porque nao depende de um segundo termo polar no lexicon.

**O retreino do notebook 02 e OBRIGATORIO antes de a API voltar a subir.** O artefato de 38 nao carrega mais: `Fusor.carregar` valida a dimensao e levanta `FusorIncompativelError`. Isso e o comportamento desejado, nao um bug.

Medido ANTES de aceitar, no B2W-Reviews01 (2000 resenhas por rotulo, semente 20260904) -- que e o corpus que da o TEXTO do treino:

```
insatisfeito  82/2000 = 4,10%
neutro        53/2000 = 2,65%
satisfeito    35/2000 = 1,75%
```

Dispara nos tres rotulos, gradiente suave, razao 2,3:1 entre os extremos. Compare com a cabeca de ironia que saiu do vetor no mesmo dia (74% contra 9%, razao 8:1): aquilo era disjuncao pratica, isto e sinal.

**Uma armadilha de metodo que quase repeti.** Medir esta feature no corpus do SIMULADOR dava zero nos tres rotulos, e eu quase concluí que ela nasceria com peso zero. Errado, e pelo mesmo motivo registrado na correcao de 04/09 mais acima: `FRASES_POR_ROTULO` **nao treina o fusor**. O notebook 02 usa 400 frases do B2W por classe e pega do simulador so a ESTRUTURA da conversa. O simulador importa para a guarda `test_nenhuma_feature_e_previsor_unilateral`, nao para o peso aprendido. As frases dos tres rotulos foram cruzadas mesmo assim -- sem isso a feature ficava constante em zero la e a guarda passaria por vacuidade, que e o modo de falha silencioso dela.

## CONFERENCIA DO ARTEFATO

```
uv run python scripts/conferir_fusor.py             # laudo completo
uv run python scripts/conferir_fusor.py --sem-sonda  # pula as frases-sonda
```

Laudo do artefato vigente em `modelos/`, em seis itens: contrato, ordem das classes, sinal dos pesos por familia, pesos perto de zero, tres frases-sonda e as metricas do treino. Instrumento, nao botao — nao escreve nada, no modelo de `scripts/medir_faixas.py`.

Ele existe porque **acuracia nao denuncia** os dois defeitos que esta semana custaram dois retreinos: o peso invertido de `ironia_prob_media` e o vazamento de `incongruencia_hiperbole`. Os dois so apareceram porque alguem olhou o sinal dos coeficientes e a distribuicao por rotulo — exatamente o que uma pessoa cansada depois de um treino longo pula. Por isso a acuracia nunca e impressa sozinha no laudo: vem sempre com o lembrete da invariante 10, e com alerta explicito acima de 0,97.

## 2.3.6 Contrato de features

`NOMES_FEATURES`, em `fraus/fusor.py`, e a lista canonica. `vetorizar` levanta `KeyError` se faltar chave — nunca zero silencioso (invariante 9).

Hoje sao **39 features**, de SETE familias no vetor (OITO sinais existem no sistema — a ironia continua existindo e sendo lida por mensagem, so nao entra mais aqui, ver a secao acima). O contrato subiu de 16 para 35 em 21/08/2026, quando os notebooks 03 e 04 passaram a existir e a condicao que justificava a espera acabou; de 35 para 40 em 03/09/2026, quando a familia `incongruencia_*` entrou (ver Retreino do fusor apos as features de incongruencia acima); e caiu de 40 para 38 em 04/09/2026, quando `ironia_prob_media` e `ironia_prob_max` sairam (ver A ironia sai do vetor acima).

| SINAL | MODULO | FEATURES | NO VETOR? |
|---|---|---|---|
| texto | `fraus/sinais/texto.py` | 4 | sim |
| emoji | `fraus/sinais/emoji.py` | 5 | sim |
| tempo | `fraus/sinais/tempo.py` | 7 | sim |
| emocao | `fraus/sinais/emocao.py` | 8 | sim |
| lexico | `fraus/sinais/lexico.py` | 3 | sim |
| ironia | `fraus/sinais/ironia.py` | 2 | **nao** (lida por mensagem, nao pontua) |
| estilo | `fraus/sinais/estilo.py` | 6 | sim |
| incongruencia | `fraus/sinais/incongruencia.py` | 5 | sim |

**Consequencia:** `montar_features` exige DOIS classificadores (texto e emocao); `Motor` continua exigindo os TRES (a ironia entra na atribuicao por mensagem, nao no vetor), e a API nao sobe sem os tres artefatos em `modelos/`. E o comportamento correto da invariante 7.

## 2.3.7 Notebook 03 — classificador de emocao

Requisito de banca: distinguir as **sete emocoes humanas**. A cabeca treinada tem sete classes, e a setima emocao e derivada — ver abaixo.

A ordem das classes **nao e redigitada no notebook**: ele importa `NOMES_EMOCOES` de `fraus.sinais.emocao`, a mesma lista que o sinal usa para ler as probabilidades. Redigitar seria o jeito mais facil de treinar `raiva` no indice 2 e le-la no indice 3.

### ORDEM CANONICA DAS CLASSES

`fraus/sinais/emocao.py` fixa a ordem, e ela vale no notebook, no sinal e no fusor. Inverter nao gera erro: atribui a emocao errada em silencio, a mesma armadilha da invariante 8.

| INDICE | CLASSE |
|---|---|
| 0 | alegria |
| 1 | tristeza |
| 2 | raiva |
| 3 | medo |
| 4 | nojo |
| 5 | surpresa |
| 6 | neutro |

### CORPUS: `GO_EMOTIONS_PTBR`

Traducao PT-BR do GoEmotions (Demszky et al., 2020 — 58k comentarios do Reddit, 27 emocoes + neutro), com 43.410 exemplos de treino, 5.426 de validacao e 5.427 de teste. Licenca Apache 2.0. As 27 categorias sao reduzidas as seis de Ekman pelo **mapeamento oficial que o proprio GoEmotions publica** — nao por agrupamento inventado aqui.

O arquivo vem no formato CRU do GoEmotions — **uma linha por anotador**, nao por exemplo. Um rotulo conta quando **pelo menos 2 anotadores** o marcaram (a maioria dos exemplos tem 3 anotadores, entao 2 e maioria simples), e exemplos que caem em mais de uma classe de Ekman sao descartados: a tarefa e multiclasse de rotulo unico, e forcar vencedor entre duas emocoes empatadas inventaria rotulo.

O limiar mais alto rende MAIS exemplos, o que so parece contra-intuitivo ate ver o motivo — com 1 anotador quase todo exemplo recebe varios rotulos Ekman e cai fora do filtro de rotulo unico:

| LIMIAR | EXEMPLOS COM ROTULO UNICO |
|---|---|
| 1 anotador | 17.367 |
| **2 anotadores** | **46.014** |

## O DESEQUILIBRIO, E POR QUE NAO SUBAMOSTRAR

Distribuicao medida com limiar 2:

| CLASSE | EXEMPLOS |
|---|---|
| alegria | 19.284 |
| neutro | 13.603 |
| raiva | 5.155 |
| surpresa | 4.339 |
| tristeza | 2.601 |
| medo | 588 |
| **nojo** | **444** |

Sao **43:1** entre a maior e a menor. Os notebooks 01 e 02 balanceiam por subamostragem, mas aqui isso seria destrutivo: cortar tudo ao tamanho de `nojo` deixaria **3.108 exemplos** para sete classes, jogando fora 93% do corpus.

**Escolha: pesos de classe na perda**, pela formula `n / (k * n_c)` — a mesma do `class_weight='balanced'` do scikit-learn. Os 46 mil exemplos ficam. Por isso a metrica de selecao do melhor checkpoint e o **F1-macro, nunca a acuracia**: com 43:1, um modelo que chutasse `alegria` sempre marcaria 42% sem ter aprendido nada.

## O TETO DO DESPREZO — A DECLARAR NO RELATORIO

`nojo` e a classe mais rara do corpus, e o **desprezo e derivado de raiva x nojo**. A qualidade do desprezo fica limitada de cima pela qualidade da pior classe do modelo: se o F1 de `nojo` sair baixo, o desprezo herda isso. O notebook imprime esse teto explicitamente e grava o F1 por classe em `metricas_emocao.json`, para o numero aparecer no relatorio em vez de ser descoberto pela banca.

**Limitacao a declarar:** a traducao foi feita por maquina (Google Tradutor), sem revisao humana. Por isso a avaliacao reporta tambem o **XED-pt** (7.220 linhas, CC-BY, Plutchik 8 + neutro) como conjunto de teste independente, que nao passou por traducao automatica. O mapeamento Plutchik -> Ekman descarta *anticipation* e *trust*; o resto casa 1:1.

Duas ressalvas sobre o XED-pt: ele e **portugues europeu** ("pa", "artola"), enquanto o treino e PT-BR — diferenca de variedade, nao so de dominio; e o numero dele vai sair **mais baixo** que o interno, o que e esperado e e justamente o ponto, porque e ele que responde "mas nao e so traducao automatica?".

## POR QUE O DESPREZO NAO E TREINADO

A setima emocao e o **desprezo**, e nenhum corpus rotulado em portugues o anota. Havia um corpus multilingue sintetico com `contempt` entre os rotulos, e ele foi **recusado de proposito**: treinar seis classes com dado humano e so o desprezo com dado de outra procedencia ensinaria o modelo a reconhecer o *estilo do texto sintetico* em vez do desprezo. E o mesmo vazamento da latencia disjunta, com outra roupa.

O desprezo e derivado da **diade primaria raiva + nojo**, que e como Plutchik (1980) o define — referencia citavel, nao rotulo inventado. A composicao usa media **geometrica**, nao aritmetica: desprezo exige as duas emocoes juntas, e a aritmetica devolveria 0,5 para raiva pura sem nojo nenhum, o que e raiva e nao desprezo.

### 2.3.8 Notebook 04 — classificador de ironia

Classificacao binaria: 0 nao-ironico, 1 ironico.

> **Registro de 19/10/2026: candidato MultiPICo-PT.** O notebook 04 passou a treinar no MultiPICo (CC BY 4.0, pares post-resposta com irônico e não irônico vindos das mesmas fontes) com `fraus.receita_treino`. A comparação histórica abaixo foi contra a cabeça IDPT; ela não identifica o checkpoint publicado hoje: MultiPICo F1 0,511 → 0,660, diferença +0,149 com IC 95% [+0,088; +0,210] e McNemar p = 0,0002; régua de 20 frases 0,394 → 0,520 (falso positivo 7/10 → 2/10; n pequeno, IC cruza zero). O baseline só de estilo dá 0,499 no teste: o corpus não entrega o rótulo pela forma. ECE 0,34 → 0,06 com a temperatura. **O ONNX dela é fp32**: em int8 ela troca de classe em 2 das sondas de `encolher_modelos.py` (`--precisao fp32`, Δp 0,0000). A ironia continua fora do vetor do fusor, então a troca não pede fusor novo. O texto abaixo é o histórico do IDPT.

**Corpus:** IDPT 2021, a tarefa de *Irony Detection in Portuguese* do IberLEF — 15,2k tweets e 18,4k noticias anotados.

> **08/10/2026: o treino publico do IDPT entrega o rotulo pela fonte.** Os tweets nao-ironicos sao 100% `#economia` de portais de noticia; as noticias sao rotuladas pelo site. Retreinado com a divisao certa, o BERTimbau deu F1 0,994 no teste e P(ironico) = 1,0 em "ok, obrigado" -- e nao foi exportado. Analise, plano e o treino do Laya so em ironia: Ironia: corpus e Laya.

#### FONTES PUBLICAS DO TREINO E TESTE OFICIAL FECHADO

O **teste oficial**, anotado para a competicao, continua protegido e deve ser solicitado aos organizadores em https://sites.google.com/inf.ufpel.edu.br/idpt2021/. Ele nunca entra no treino. As fontes publicas usadas para formar o treino do IDPT, porem, estao abertas: os tweets em https://github.com/fabio-ricardo/deteccao-ironia e as noticias em https://github.com/schuberty/PLNCrawler. O notebook baixa essas fontes no runtime por commits imutaveis e valida cada Git blob antes de usar.

O notebook aceita **tres origens** para o corpus, escolhidas na variavel `ORIGEM` da celula 3. Todas alimentam a mesma pasta e a mesma deteccao de esquema, entao trocar de fonte nao muda o resto do notebook:

| `ORIGEM` | QUANDO USAR |
|---|---|
| `idpt_publico` (**padrao**) | baixa as fontes publicas originais, remove hashtags que vazam o rotulo, duplicatas e conflitos |
| `sintetico` | gera o corpus com `fraus.ingest.gerador_ironia` — fica como ablacao, nao como fonte principal |
| `drive` | os arquivos ja estao em `DIR_CORPUS` — foi assim que o IDPT entrou, se liberado |
| `kaggle` | baixa da conta Kaggle; exige `KAGGLE_USERNAME`/`KAGGLE_KEY` nos Secrets do Colab |
| `upload` | seletor de arquivos do navegador, para corpus que chegou por e-mail |

### A DIVISAO POR AUTOR — CONSERTO DE 02/10/2026

A regra da divisao mora em `fraus/divisao.py` e e a mesma nos notebooks 04 e 07: um grupo por autor, e **cada linha sem autor e um grupo proprio**.

Ate 02/10/2026 a regra estava escrita nos notebooks com `autores.astype(str)`, que transforma o campo vazio no autor "nan". As 18.373 noticias nao tem autor: viraram uma pessoa so e cairam inteiras no teste.

| | ANTES | DEPOIS |
|---|---|---|
| Treino | 12.271 linhas, 86% ironicas, so tweets | 28.365 linhas, 59% ironicas |
| Teste | 20.872 linhas (63% do corpus), 43% ironicas | 4.778 linhas (14%), 59% ironicas |

Os numeros de "depois" foram medidos aplicando a regra nova ao corpus real, fora do Colab; os do treino saem quando o notebook rodar.

Consequencias:

- A cabeca de ironia treinada antes desta data **nunca viu uma noticia** no treino e foi avaliada num teste que era 88% noticias. O F1-macro de 0,511 que o notebook 07 recalculou para ela nesse teste mede mudanca de dominio, nao a tarefa.
- `conferir_divisao` agora recusa, com erro, divisao cujo teste foge do tamanho pedido ou cuja proporcao de classe difere entre os dois lados.
- O notebook 04 grava em `metricas_ironia.json` a impressao (SHA-256) dos textos do teste. O notebook 07 para antes do treino se o teste dele nao tiver a mesma impressao: sem isso, frases do treino do BERTimbau cairiam no teste da comparacao.
- Nas noticias a fonte entrega o rotulo (Estadao e sempre nao-ironico; os dois sites de satira, sempre ironicos). Com as noticias dos dois lados, o numero agregado tende a subir por reconhecer o jornal. Os dois notebooks imprimem o resultado **separado por meio**, e o que se le como desempenho em ironia e o dos tweets e o da regua de dominio. Numero alto no agregado e sintoma (invariante 10), nao vitoria.

CANDIDATO LAYA

O Laya multilíngue pode substituir apenas a cabeça de ironia, sem alterar satisfação, emoção ou o fusor. Ele é opt-in porque baixa aproximadamente 647 MB de pesos. O backend `laya` exige torch; o `laya-onnx` (30/09/2026) roda a mesma cabeça pelo ONNX Runtime, **sem PyTorch**, e é o que a API serverless usa quando o ZIP da Laya está configurado (ver `docs/deploy-vercel.md`, interno):

```
uv sync --extra laya
FRAUS_IRONIA_BACKEND=laya uv run uvicorn fraus.api.main:app
```

O checkpoint fica preso à revisão declarada em `FRAUS_LAYA_REVISAO`. Antes de promovê-lo, execute a mesma régua de transferência de domínio:

```
uv run python scripts/validar_candidato_ironia.py --backend laya
```

A aba **Modelo → Ironia · Laya** chama somente essa cabeça e não persiste nem altera notas. O tipo de pergunta usado é `choice` com chaves neutras, não `noul`, por causa do viés de rótulo documentado pelos próprios autores.

**FINE-TUNING DO LAYA — NOTEBOOK 07 (EXECUTADO EM 02/10/2026, NÃO PROMOVIDO — VER `notas/2026-10-02-treino-laya.md`)**

O candidato acima responde a frio. `notebooks/07_treino_laya.ipynb` treina o mesmo checkpoint nas duas perguntas do Fraus — emoção (`PERGUNTA_EMOCAO`, em `fraus/treino_laya.py`) e ironia (`PERGUNTA_IRONIA`) — com o script oficial de GPU única do Laya, e o compara com os BERTimbau dos notebooks 03 e 04 nos mesmos exemplos de teste: McNemar exato e IC por bootstrap da diferença de F1-macro (`fraus/comparacao_modelos.py`), mais o XED-pt e a régua de domínio.

Ele não promete 100% de acerto, e número perto disso seria sintoma (invariante 10). No fim, poda o vocabulário de 256 mil tokens para o que o português usa, exporta para ONNX e só adota INT8 se no máximo 1% das decisões mudar.

A última célula grava `comparacao_modelos.json` (acurácia, F1-macro, matriz de confusão, VP/FP/FN/VN por classe, McNemar, IC e latência em CPU) e o mesmo laudo em Markdown. Copiado para `modelos/`, ele aparece na aba **Modelo → Comparação** da dashboard (`GET /modelo/comparacao`), que também lê uma fala com as cabeças carregadas no servidor, lado a lado. Cada leitura do Laya diz se o artefato do servidor é o treinado pelo notebook 07 ou o checkpoint base ("Laya sem treino"): em produção é o base, que lê ironia e, a frio, emoção.

**Para o laudo chegar à produção** ele é versionado com o código, em `fraus/dados/comparacao_modelos.json`: `modelos/` fica fora do git e, na Vercel, é trocada inteira pelo ZIP dos pesos. A rota serve o laudo de `modelos/` quando ele existe e, na falta, o versionado. O campo opcional `ressalvas` (lista de textos) registra o que se soube sobre a medição depois de feita; ele aparece no topo da aba e do relatório. Os números do laudo nunca são editados à mão — laudo novo é notebook rodado de novo.

Rode primeiro com `MODO_FUMACA = True`. O que foi pesquisado, o que foi medido localmente e o que ainda falta está na nota interna `docs/notas/2026-10-02-treino-laya.md`.

### ALTERNATIVAS AO IDPT — LEVANTAMENTO DE 14/08/2026

Registrado para ninguem repetir a busca. O que **nao serve**, e por que:

| FONTE | VEREDITO |
|---|---|
| `arbml/multilingual_irony` (HF) | so `tweet_id` + `label`, **sem texto**; reidratar exige API paga do Twitter |
| `ramondomiingos/ptbr-irony-idioms-regionalism` (HF) | < 1.000 linhas, e e *benchmark de LLM* com gabarito em prosa — nao e corpus de classificacao |
| `rafaelanchieta/PiLN` (GitHub) | sao os **modelos** treinados do IDPT, nao o corpus; embeddings dependem de servidor antigo do NILC |
| IroSvA / SemEval-2018 traduzidos | **desaconselhado**: traducao automatica reintroduz o vazamento de procedencia que ja custou o primeiro fusor |

O candidato mais citado e o corpus de **Goncalves et al., BraSNAM 2015** ("Bazinga! Caracterizando e Detectando Sarcasmo e Ironia no Twitter", UFMG), que circula no Kaggle. Lendo o artigo: sao ~2.628 tweets por classe coletados pelas hashtags `#sarcasm` **e** `#irony` e caracterizados com LIWC — ou seja, e um trabalho brasileiro sobre o Twitter **anglofono**. Corpus em ingles. Usa-lo aqui seria o vazamento de procedencia com selo academico.

Verificado tambem e descartado: `OpenNeuro ds004533` ("Ironia VEV") e **neuroimagem** — ressonancia de 40 sujeitos ouvindo frases ironicas, sem nenhuma frase rotulada para treino.

### ONDE EXISTE CORPUS PT-BR DE IRONIA (E COMO PEDIR)

Dois, os dois **sem download publico**:

1. **Vieira e Silva, A. (2025)** — *Deteccao automatica de ironia por meio de representacoes contextuais*, tese de doutorado em Linguistica, USP, orientacao de Marcos Lopes. Produziu um corpus de ironia em portugues com conversas da rede X: **1.186 exemplos** finais (de 1.200), anotados por **tres voluntarios humanos**, com o contexto da conversa. Achado relevante para o relatorio: os anotadores precisaram do **contexto de producao em ~30% dos casos** — evidencia direta de que ironia em mensagem isolada tem teto baixo, e ela vale citada mesmo sem o corpus em maos.
2. **Projeto IDPT / UFPel** — https://institucional.ufpel.edu.br/projetos/id/u3345, coordenacao de **Larissa Astrogildo de Freitas**. E o grupo por tras do IDPT 2021. Enderece o pedido a coordenacao.

**Tamanho importa aqui:** 1.186 exemplos anotados a mao nao treinam um BERT do zero, mas sao excelentes como **conjunto de teste independente** — exatamente o papel que o XED-pt cumpre no notebook 03. E a arquitetura de honestidade que o projeto ja usa: treinar no que ha em volume, medir no que ha em qualidade.

### O PADRAO: CORPUS SINTETICO

Enquanto nenhum dos dois chega, o notebook gera o proprio corpus com `fraus.ingest.gerador_ironia`. E o mesmo impasse do sinal de tempo e a mesma saida ja aceita: o fusor treina em conversas sinteticas porque nenhum corpus de review tem timestamps.

O gerador foi escrito **contra** o vazamento que matou o primeiro fusor. A armadilha obvia seria "ironico = elogio + fato ruim", que ensinaria o modelo a procurar fato ruim em vez da incongruencia. Tres cruzamentos impedem isso, cada um travado por teste em `tests/test_gerador_ironia.py`:

- os mesmos **fatos ruins** aparecem nas reclamacoes diretas (nao-ironicas);
- as mesmas **palavras elogiosas** aparecem em elogios sinceros e em reclamacoes que comecam agradecendo;
- **numeros e unidades de tempo** aparecem nas duas classes;
- ha uma familia de ironia **por atenuacao**, sem palavra elogiosa nenhuma, para que "tem elogio" nao seja condicao necessaria.

**A metrica interna sera otimista por construcao** — ela mede o quanto o modelo aprendeu estes padroes, nao o quanto le ironia humana. Isso precisa estar no relatorio.

Por isso o `metricas_ironia.json` grava a **procedencia derivada de `ORIGEM`**, nunca `IDPT 2021` fixo: artefato que mente sobre a propria fonte e pior que artefato ausente.

Como o notebook **nunca viu os arquivos**, a celula de carga tem esquema **configuravel**: ela le todo `.csv`/`.tsv` da pasta, imprime as colunas encontradas, tenta achar a de texto e a de rotulo pelos nomes mais provaveis e **para nomeando as colunas disponiveis** se nao achar. Rotulo fora do mapeamento tambem para com erro, em vez de virar 0 silenciosamente.

**Se o corpus nao for liberado a tempo**, a saida honesta e declarar a ironia como trabalho futuro no relatorio. Trocar por um corpus de sarcasmo em ingles traduzido repetiria, com outro nome, o vazamento de procedencia que ja custou o primeiro fusor.

### PORTAO DE PROMOCAO DO ARTEFATO

A acuracia/F1 do holdout sintetico **nao autoriza** substituir o modelo em producao. Depois do notebook 04, copie primeiro o checkpoint para uma pasta de candidato e rode:

```
uv run python scripts/validar_candidato_ironia.py /caminho/do/candidato \
  --saida-json metricas_regua_ironia.json
```

O comando exige no maximo 30% de falso positivo nas dez falas sinceras com marcador e 30% de falso negativo nas dez ironias sem marcador. Ele devolve codigo 1 se qualquer fatia falhar, para que o artefato nao seja promovido por uma unica media boa. As falas e o calculo canonicos moram em `fraus/avaliacao_ironia.py`; o laudo por fatias usa a mesma fonte.

Essa e uma **regua autoral de regressao**, deliberadamente desenhada contra os atalhos ja encontrados. Nao e corpus independente, nao estima acuracia em atendimento real e nao torna honesto reativar a ironia no fusor. Um corpus humano PT-BR separado do treino continua sendo o bloqueio cientifico.

A metrica de selecao do melhor checkpoint e o **F1 da classe ironica isolada**, nao a macro nem a acuracia: e essa classe que o sinal consome, e corpus de ironia costuma ser desequilibrado o bastante para um modelo que responde "nao e ironia" sempre marcar boa acuracia. O treino tambem usa pesos de classe, pelo mesmo motivo.

**Por que uma cabeca separada** e nao mais uma classe do sinal de texto: ironia nao e um sentimento, e uma relacao entre o que o texto DIZ e o que ele SIGNIFICA. "Que atendimento maravilhoso, so esperei 3 horas" e lexicamente positivo e pragmaticamente negativo ao mesmo tempo. Como quarta classe de satisfacao, o modelo seria obrigado a escolher uma das duas leituras — e a informacao de que ha conflito, que e justamente o sinal de ironia, se perderia.

**Limitacao a declarar:** o IDPT e anotado em tweets e comentarios de noticia, nao em atendimento de chatbot. A transferencia de dominio nao esta verificada — ironia em reclamacao de suporte pode ter forma diferente de ironia em comentario de politica.

### 2.3.9 Sinal lexico — SentiLex-PT02

Nao tem notebook: e recurso pronto, nao treinado.

**Fonte:** SentiLex-PT02 — Silva, Carvalho e Sarmento, "*Building a Sentiment Lexicon for Social Judgement Mining*" (PROPOR 2012), CC-BY. 82.347 formas flexionadas na origem, **79.189** depois da conversao.

**Preparo:** `scripts/preparar_sentilex.py` converte o arquivo original em `fraus/dados/sentilex_pt02.csv` e imprime o relatorio do que mexeu — 315 entradas cuja polaridade veio do complemento (`POL:N1`), 9 com sinal normalizado (marcadas `REV:POL` na fonte, com o contador vazado no campo de polaridade) e 2.032 formas homografas com polaridades conflitantes, que foram **zeradas** em vez de decididas pela ordem do arquivo.

**Por que um lexicon se o BERTimbau ja le o texto:** porque ele nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho — e a **discordancia** entre eles e o formato tipico da ironia.

**Negacao por escopo:** "nao foi otimo" inverte a polaridade de "otimo", com alcance de tres tokens, e o escopo **morre na pontuacao forte** — em "nao chegou. otimo atendimento" o *otimo* continua positivo.

**Acento:** o SentiLex e acentuado e cliente de chat nem sempre. Ha indice de reserva sem acento; das 74.443 chaves resultantes, apenas **24** colidem (`incomodo`/`incomodo`, `ingenua`/`ingenua`) e essas ficam zeradas — chute de polaridade errado e pior que termo ausente.

**Limitacao do recurso, a declarar:** o SentiLex e lexicon de **julgamento social** — anota polaridade dirigida a entidades humanas. E forte no adjetivo que julga (`pessimo`, `otimo`, `incompetente`) e **neutro em verbo de afeto do proprio falante**: `gostar`, `adorar` e `odiar` valem 0 nele. Por isso o sinal lexico complementa o BERTimbau e nao o substitui — quem le "adorei o produto" e o transformer.

### 2.3.10 Sinal de estilo

Nao tem notebook: e **deterministico**, nao treinado — `fraus/sinais/estilo.py` mede a FORMA da escrita (caixa alta, pontuacao enfatica, alongamento, palavrao com gradacao, censura), nao o conteudo.

**Por que deterministico e nao um classificador.** O BERTimbau nao aprende enfase porque o B2W-Reviews01 — resenha moderada de e-commerce — praticamente nao contem gritaria nem xingamento. Modelo nao aprende fenomeno que o corpus de treino nao tem, e mais epocas sobre o mesmo texto so reproduziriam o mesmo artefato. Regra escrita a mao cobre exatamente o que falta: forma, nao semantica.

**Fonte do lexicon de palavrao:** `fraus/dados/palavroes_ptbr.csv`, curadoria propria, ~172 termos, colunas `termo,intensidade,alvo`. A intensidade e graduada (`leve`/`medio`/`pesado`, mapeada para 0,33/0,66/1,0) porque "que droga" e "vai tomar no cu" nao sao o mesmo evento, e o alvo (`pessoa`/`desabafo`) existe porque xingar o PRODUTO e reclamacao enquanto xingar o ATENDENTE e ruptura da conversa. Nenhuma dependencia externa entrou para isso, e o SentiLex-PT02 tambem nao serve — ele anota polaridade de julgamento, nao gradacao de baixo calao nem alvo do xingamento.

**Ordem censura-antes-de-normalizacao.** `tem_censura` roda sobre a palavra CRUA, com a mascara (`*@#$%&`) ainda intacta, e so depois `normalizar` reverte homoglifo e apaga acento. Inverter a ordem apagaria a marca de censura antes de ela ser vista — "p*rra" perderia o asterisco e viraria uma palavra qualquer, nunca reconhecida como autocensura.

**Casamento por curinga.** `casar_censurado` trata cada simbolo de mascara como uma letra qualquer (regex com `.` no lugar do simbolo), nao como uma letra fixa: `p*rra` casa `porra`, `c*ralho` casa `caralho`. Mapear o simbolo para uma vogal fixa acertaria um caso e erraria o outro em silencio. Quando mais de um termo do lexicon casa o mesmo padrao com curinga, o desempate e pela MAIOR intensidade — deterministico, e nao pela ordem em que a linha aparece no CSV, o que deixaria o resultado dependente de onde alguem inseriu a linha.

**A excecao do `kkkk`.** Alongamento de caractere (`MINIMO_ALONGAMENTO = 3` repeticoes seguidas) e tratado como enfase, exceto para as letras de riso (`k`, `h`): "kkkk" e o marcador POSITIVO mais comum do chat brasileiro, e contar risada junto de "naooooo" inverteria o sentido da feature em boa parte das conversas reais.

**Limitacao a declarar:** o lexicon de palavrao e curadoria propria, nao recurso academico publicado. Diferente do SentiLex-PT02 (Silva, Carvalho e Sarmento, PROPOR 2012) e do Emoji Sentiment Ranking usados pelos outros sinais, `palavroes_ptbr.csv` nao tem paper citavel nem revisao por pares — a cobertura e a gradacao vieram de julgamento proprio, e isso precisa aparecer no relatorio como o que e.

---

## 2.3.11 Calibração — a temperatura mora no artefato (desde 19/10/2026)

O antigo `notebooks/05_calibracao.ipynb` saiu em 19/10/2026. Ele ajustava a temperatura num conjunto "cego" que vazava para o treino da satisfação, e gravava `modelos/calibracao.json`, que nada no runtime lia.

Agora cada notebook de treino (01, 03, 04) ajusta a temperatura **na validação**, via `fraus.receita_treino.ajustar_temperatura`, e grava o valor em `fraus_receita.json` dentro da pasta do modelo. O runtime (`fraus/sinais/texto.py` e `fraus/sinais/onnx.py`) aplica essa temperatura; artefato sem o arquivo continua com `T = 1,0`, então os modelos em produção não mudam de score.

**Temperature scaling não muda a decisão do modelo.** Dividir os logits pelo mesmo `T > 0` preserva o `argmax`: acurácia e F1 ficam idênticos, só a confiança declarada muda (ECE).

**A ironia é o caso a não confundir.** Calibrar melhora o ECE dela, mas ela continua errando falas sinceras de atendimento. Temperatura ajusta *o quanto* o modelo se diz confiante, não *sobre o que*.

## 2.3.12 Notebook 02 — a seção 10 é uma pendência, não uma etapa

A saturação das features de tempo está descrita lá com a medição, o conserto (`winsorizar no p99` + `log1p` antes do scaler) e — o que mais importa — **o acoplamento**: a transformação precisa existir no notebook *e* no runtime, ou

o score sai errado em silêncio. Um `fusor.joblib` treinado sem a compressão contra um runtime que comprime continua tendo `n_features_in_ = 39`, então `Fusor.carregar` **não pega** essa incompatibilidade.

### AS TRÊS SAÍDAS, MEDIDAS EM 15/09/2026 — DECIDIDO: SÓ `LOG1P`

O instrumento histórico `scripts/retreinar_fusor_local.py`, na versão de medição do commit `e719218`, reproduziu o notebook localmente (mesmo B2W, mesmas sementes e `montar_features`, extração por ONNX): acurácia 0,9500 nos dois, score diferindo no máximo 0,0001 do fusor daquela data. O script atual exige manifesto agrupado e não faz essa comparação carregando o fusor ativo; os números desta seção não são nova reprodução.

A coluna de espera é a frase canônica (texto 0,858 satisfeito, todo o resto na média do treino), com score, nota e categoria:

| VARIANTE | ACURÁCIA | F1 | CATEGORIAS ≠ VIGENTE (DE 300) | 60 S | 300 S | 600 |
|---|---|---|---|---|---|---|
| vigente | 0,9500 | 0,9501 | — | 99,9 · 10 P | 98,1 · 10 P | 44,5 |
| só `log1p` | 0,9467 | 0,9469 | 6 | 99,7 · 10 P | 96,6 · 10 P | 90,1 |
| só teto p99 | 0,9433 | 0,9434 | 1 | 99,9 · 10 P | 96,1 · 10 P | 18,4 |
| p99 + `log1p` | 0,9467 | 0,9469 | 5 | 99,7 · 10 P | 96,6 · 10 P | 90,7 |

O que a tabela muda na recomendação escrita acima:

- **p99 + `log1p` conserta o relógio mandando sozinho apagando a espera longa.** O teto do treino é ~7 min (`latencia_mediana_s` p99 = 424 s), então três horas pesam o mesmo que dez minutos: a frase canônica, que é **irônica**, sai promotor. Consertar um defeito criando o oposto não é conserto.
- **Só teto** não resolve nada: o penhasco só muda de lugar (10 min → nota 2).
- **Só `log1p`** é a única que mantém a espera **monótona e sem penhasco**: 10 min ainda promotor, 30 min detrator 6, 3 h detrator 1 — o texto modula, o relógio pesa cada vez mais devagar, e nenhum dos dois apaga o outro. Custa 0,33 ponto de acurácia no teste sintético.
- **Ablação da invariante 10:** só as quatro features de tempo acertam 0,513 (cru) e 0,527 (p99+log1p), contra 0,333 do acaso. O relógio carrega sinal do rótulo no simulador por construção, e nenhuma das variantes muda isso.

PROMOVIDO EM 15/09/2026

As três partes indivisíveis, feitas juntas:

1. **Função única.** `log1p` mora em `fraus.fusor.vetorizar`, sobre `FEATURES_EM_LOG`. As chaves continuam em segundos (atribuição, tela e deriva leem segundos); só o vetor que o scaler recebe é comprimido. Treino, pontuação, contribuição e deriva passam pela mesma função.
2. **Artefato regerado e promovido naquela data**, pela versão histórica do script: acurácia 0,9467, F1-macro 0,9469. O anterior foi preservado como backup. A campanha atual não repete essa promoção. O artefato passou a carregar `fraus_escala_do_tempo = "log1p"`, e `Fusor.carregar` **recusa** artefato sem essa marca: é o caso que `n_features_in_` não pega.
3. **Medição de categoria.** No teste do B2W, 27 notas e 6 de 300 categorias mudam. Nas 180 conversas do simulador, 23 notas e 7 categorias, com a distribuição indo de 60/63/57 para 60/60/60 (detrator/neutro/promotor).

`scripts/medir_dominio_do_tempo.py` contra o artefato novo: o relógio empata com o texto em **613 s** (antes 411 s), e com 3 h pesa **2,1×** o texto (antes 20×). Uma conversa sincera ("resolveu, muito obrigado") com 30 min de espera saía 0,30; agora sai 55,3, ainda detrator. Com 3 h, 21,95 em vez de 0,00.

Esse registro explica por que um `fusor.joblib` anterior à mudança de escala é recusado. Não é instrução para sobrescrever pesos em outro ambiente: promoção e deploy são operações separadas, fora desta campanha.

# 2.4 GENERALIZAÇÃO E EVIDÊNCIAS

## 2.4.1 Estado da investigação — 27/10/2026

> ⚠ CAMPANHA CONCLUÍDA; CANDIDATOS NOVOS, NÃO MODELOS PUBLICADOS
>
> A reprodução de 26–27/10 concluiu **nove treinos em T4**, exportou os nove bundles e avaliou o candidato do fusor e suas ablações. As curvas comparáveis sustentam **sobreajuste nesta reprodução**, com padrões detalhados abaixo. O diagnóstico dos pesos antigos e dos publicados continua **inconclusivo**: suas curvas e proveniência não foram recuperadas. Os candidatos não foram promovidos; a sessão própria foi encerrada pela CLI.

Esta página distingue quatro perguntas: **vazamento**, **sobreajuste**, **transferência de domínio** e **insuficiência de evidência**. Uma métrica alta não responde às quatro; uma queda entre corpora tampouco prova overfitting. As medições históricas continuam no Cartão do modelo e no diário de treinamento, com suas datas.

## 2.4.2 Matriz de evidências

Os identificadores abaixo nomeiam saídas de pesquisa preservadas fora do site público. Não são links para manifestos individuais, dados brutos ou pesos. Base do inventário: commit `836d440bbe246018fb8808a4ede4c6f84d6a0ef9`. A data é a da observação, não uma alegação sobre quando o modelo foi treinado.

A campanha nova usa a fonte congelada `03f91770ec92dec934d794cbad50ea3224f4cb8a`, vinculada por `manifesto-bundles.json`. O upload não continha `.git`: o campo de commit interno dos notebooks ficou nulo. A ligação externa

com o commit resulta da comparação integral dos três `codigo_manifesto.json` com os bytes de `git archive` dessa fonte, não de um commit interno inventado. Pesos, logs e manifestos individuais permanecem fora do Git e do site.

| ARTEFATO / COMPONENTE | DATASET E DIVISÃO | DATA | O QUE FOI OBSERVADO | LIMITE |
|---|---|---|---|---|
| `inventario.json` / três cabeças Torch locais | arquivos, configurações e hashes; sem inferência | 26/10/2026 | checkpoints locais identificados por bytes | não identifica produção nem recupera manifestos históricos |
| `auditoria.json` / corpora locais completos | leitores canônicos e splits 42, sem inferência | 26/10/2026 | auditoria integral concluída; duplicatas, conflitos e interseções normalizadas registrados | a reconstrução não prova a origem supervisionada dos pesos antigos |
| `historico.json` / receita antiga do fusor | pools posicionais B2W, 900 linhas de treino e 300 de teste | 26/10/2026 | zero textos normalizados comuns; **41 produtos comuns**, envolvendo 63 linhas de treino e 52 de teste | reconstrução da receita atual, não prova de quais exemplos treinaram pesos antigos |
| `satisfacao-atual.json` / cabeça local de satisfação | B2W, amostra estratificada de 5.000 dos 18.895 exemplos do teste reconstruído | 26/10/2026 | F1-macro **0,764556**, acurácia **0,820000**, ECE **0,018971** | pesos anteriores à divisão podem ter visto exemplos; independência histórica não verificável |
| `satisfacao-atual.json` / mesma cabeça | ConsumerBR, amostra estratificada de 5.000 dos 143.717 exemplos do teste reconstruído | 26/10/2026 | F1-macro **0,538760**, acurácia **0,690600**, ECE **0,090972** | transferência já consultada; independência depende de treino somente em B2W, sem manifesto histórico que a comprove |
| `ironia-atual.json` / cabeça local de ironia | teste reconstruído MultiPICo-PT, 462 exemplos | 26/10/2026 | F1-macro **0,660294**, acurácia **0,683983**, ECE **0,055439** | benchmark já consultado; independência histórica do sistema não verificável |
| | | 26/10/2026 | | conjunto pequeno e já |

| ARTEFATO / COMPONENTE | DATASET E DIVISÃO | DATA | O QUE FOI OBSERVADO | LIMITE |
|---|---|---|---|---|
| `ironia-atual.json` / mesma cabeça | régua de 20 frases de atendimento | | F1-macro **0,520000**, acurácia **0,550000**, ECE **0,242688** | consultado; não estima qualidade em produção |
| `emocao-atual.json` / cabeça local de emoção | teste GoEmotions-PT reconstruído, 7.328 exemplos | 26/10/2026 | F1-macro **0,691023**, acurácia **0,720524**, ECE **0,056790** | pesos anteriores à nova divisão podem ter visto parte dos exemplos; independência histórica não verificável |
| `emocao-atual.json` / mesma cabeça | XED-pt, 4.286 exemplos, seis classes presentes (sem neutro) | 26/10/2026 | F1-macro **sete classes 0,240398**; **seis presentes 0,280465**; acurácia **0,285814**, ECE **0,451814** | transferência já consultada; rótulos projetados de outras línguas, não validação de suporte real |
| satisfação e emoção locais / curvas antigas | treino supervisionado histórico | 26/10/2026 | curvas e manifestos individuais antigos não recuperados | não permite medir gap treino–validação dos pesos atuais |
| `resumo-cientifico.json` + nove bundles / satisfação, emoção e ironia novas | B2W, GoEmotions-PT e MultiPICo PT; splits congelados e sementes 13/42/77 | 26–27/10/2026 | nove avaliações finais, ECE bruto/calibrado e curvas por época; tabelas abaixo | reprodução nova, não recuperação do histórico nem identificação de produção |
| `fusor-candidato/metricas.json` / completo e ablações | B2W agrupado + 900/120/300 conversas sintéticas de treino/validação/teste | 26–27/10/2026 | extração real com cabeças candidatas e avaliação de 39/4/7 features concluídas | uma execução por variante, não atendimento real rotulado |
| `independencia-sistema.json` / prova complementar | validação/teste B2W contra treinos reais das três cabeças novas | 27/10/2026 | fingerprints correspondem às receitas; zero colisões textuais normalizadas nos seis cruzamentos | não detecta paráfrases; grupos entre fontes não são comparáveis |
| | | 27/10/2026 | | |

| ARTEFATO / COMPONENTE | DATASET E DIVISÃO | DATA | O QUE FOI OBSERVADO | LIMITE |
|---|---|---|---|---|
| `transferencia-sobreposicoes.json` + `resumo-cientifico.json` / candidatos selecionados | ConsumerBR amostrado (5.000) e XED integral (4.286) contra os treinos reais correspondentes | | zero colisões ConsumerBR × treino B2W; seis XED × treino GoEmotions, dois conflitos | benchmarks já consultados, não testes virgens; XED não é textualmente independente |
| produção | checkpoint não identificado | 26/10/2026 | nenhuma inferência autenticada ou proveniência de pesos demonstrada nesta investigação | saúde da API não comprova inferência nem igualdade com artefatos locais |

Os pesos locais de ironia medidos têm SHA-256 `512841782d167327b21e1c2c98e602c28dbdf30d6fdafa9733b9f078f4d09388`. O conjunto MultiPICo medido tem fingerprint textual `4eee9c62e8c9a11fb5e39d1609238b7175420ca171e8352a3516e4785693595e`; a régua de 20 frases, `90f3ab25892bef90b09167120808ae9357919384667fdd74ec4132d4ef27e668`. Os pesos locais de emoção têm SHA-256 `e9735f12ef459ba93102d868be7455d9a0d453099530597f3855269b5ab83cd9`; os fingerprints textuais de GoEmotions-PT e XED-pt são, respectivamente, `8b48cefed8df6595e35c6aee7b24fc33fae1237bede03b4436e51c7e51e182ec` e `a1d2cd6db52ca19183d9a4cbaa7d1f59aae74e1f1add01d7251ef9e5ecf4c49a`. Os pesos locais de satisfação têm SHA-256 `9f9448aed6c655ce0fab2d3e32db485d0b8de986c6d907eb82336f69c62edad2`; os fingerprints textuais das amostras B2W e ConsumerBR são, respectivamente, `481da017399ae008fa610886c740b5e7b64dfa1f076b6a85e5a0473f7b38040a` e `99a53fd45904ac297d5cf62d4936945b413e8fc3b4f7720246d016e15309d237`. Essas impressões identificam conjuntos observados agora, não comprovam que ficaram fora de todo ajuste supervisionado ou de decisões históricas.

No XED, o campo `f1_macro` do JSON representa as sete classes do contrato, enquanto `f1_macro_classes_presentes` representa as seis observadas. A leitura de console pode destacar o segundo; não é intercambiável com o primeiro. As métricas das três cabeças nas linhas de 26/10 acima são **baselines dos artefatos locais anteriores**, não resultados das nove rodadas novas. As quedas B2W → ConsumerBR e GoEmotions → XED mostram limites de transferência sob essas condições, sem identificar isoladamente a causa (domínio, variedade, projeção dos rótulos ou sobreajuste).

## O QUE A RECONSTRUÇÃO DO FUSOR MOSTRA

A divisão por posição não isolava produtos, apesar de não repetir textos normalizados entre os dois pools. Logo, **disjunção textual não é independência por produto**. Não se mede aqui o efeito dessa dependência sobre a acurácia.

Estrelas (alvo da cabeça de satisfação) e recomendação com precedência do neutro de três estrelas (alvo do fusor) discordam em **16 linhas do pool de treino e 5 do pool de teste**. São alvos diferentes: a discordância não deve ser apagada nem tratada automaticamente como erro de anotação. Os pools não são as conversas efetivamente sorteadas pelo simulador e não demonstram independência do treino supervisionado das cabeças.

### 2.4.3 Auditoria integral dos corpora

Contagens aceitas pelos leitores, antes de qualquer amostragem. “Conflito” conta **linhas** removidas porque o mesmo texto normalizado recebeu rótulos diferentes dentro daquele corpus; não escolhe uma das respostas.

| CORPUS | ACEITOS | DUPLICATAS EXCEDENTES EXCLUÍDAS | LINHAS CONFLITANTES EXCLUÍDAS | APÓS LIMPEZA |
|---|---|---|---|---|
| B2W | 129.098 | 2.373 | 50 | 126.675 |
| ConsumerBR | 1.063.112 | 32.414 | 72.587 | 958.111 |
| GoEmotions-PT | 49.469 | 321 | 140 | 49.008 |
| MultiPICo PT | 1.891 | 8 | 3 | 1.880 |
| XED-pt | 4.286 | 9 | 2 | 4.275 |

O benchmark XED acima preserva seus **4.286 exemplos integrais**; a coluna “após limpeza” é uma contagem de auditoria, não uma mudança silenciosa desse benchmark. A divisão reconstruída de treino GoEmotions compartilha **seis textos normalizados** com XED, dois deles com rótulos conflitantes. Logo, nem separação por nome de corpus comprova disjunção textual.

B2W e ConsumerBR aceitos compartilham nove textos normalizados, quatro com conflitos de rótulo. A demonstração de isolamento do auxiliar ConsumerBR removeu duas linhas conflitantes e quatro duplicatas do treino principal; **não foi aplicada a um treino** desta campanha.

O pool posicional antigo de teste do fusor compartilha **215 textos** com o treino B2W da divisão 42 reconstruída. Misturar esse pool antigo com uma cabeça treinada nessa divisão nova contaminaria a avaliação do sistema. O manifesto prospectivo compartilhado entre cabeça e fusor evita essa mistura; não comprova retroativamente a independência dos modelos antigos.

### 2.4.4 Protocolo de auditoria

`scripts/auditar_generalizacao.py` reutiliza os leitores canônicos de corpora e gera JSON agregado versão 1, com corpora, artefatos, divisões, sobreposições e limitações, além de manifesto privado adjacente. Ele não treina, não infere e não desserializa pickle/joblib desconhecido.

A auditoria registra contagens aceitas pelos leitores, classes, grupos, duplicatas, conflitos e hashes. Texto é normalizado antes da comparação; o fingerprint forte também incorpora rótulos e grupos. Índices do manifesto são posições **aceitas pelo leitor**, não números de linhas brutas.

- Congelar validação e teste antes de anexar dados auxiliares. Textos ou grupos auxiliares que colidam com holdouts saem do treino auxiliar; os holdouts não são movidos para acomodá-los. Conflitos de rótulo são registrados, não resolvidos escolhendo silenciosamente uma fonte.
- Colisão entre validação e teste bloqueia o novo treino.
- Dividir por produto, autor ou post quando o leitor oferece essa identidade. Sem grupo, usar identidade textual e declarar que autores/produtos desconhecidos podem atravessar os lados.
- ConsumerBR não demonstra independência por empresa: o benchmark não usa esse agrupamento, dada a concentração dos exemplos em grandes empresas.
- Hash calculado hoje não substitui manifesto histórico. Sem cadeia de proveniência, escrever **independência não verificável**, não “sem vazamento”.

## 2.4.5 Resultados da reprodução — 26–27/10/2026

Fonte: `resumo-cientifico.json`, derivado das receitas, históricos e avaliações finais preservados nos bundles. Os valores abaixo são arredondados, não substituem os registros completos. Precisão/recall/F1/suporte por classe, matrizes de confusão, temperaturas e hashes estão nesses artefatos externos; não publicamos exemplos individuais.

### TRÊS SEMENTES POR CABEÇA

O desvio é **amostral** (`n − 1`, três sementes), não intervalo de confiança. Os tamanhos são os efetivamente usados, após limpeza e divisão agrupada. A semente escolhida é a mediana **da validação**, congelada antes do teste.

| CABEÇA / CORPUS | TREINO / VALIDAÇÃO / TESTE | F1-MACRO VALIDAÇÃO, MÉDIA ± DESVIO | F1-MACRO TESTE, MÉDIA ± DESVIO | SEMENTE ESCOLHIDA NA VALIDAÇÃO |
|---|---|---|---|---|
| satisfação / B2W | 95.722 / 12.058 / 18.895 | 0,757917 ± 0,002995 | **0,755156 ± 0,005943** | **42** |
| emoção / GoEmotions-PT | 36.806 / 4.874 / 7.328 | 0,578719 ± 0,001800 | **0,592371 ± 0,005218** | **13** |
| ironia / MultiPICo PT | 1.126 / 292 / 462 | 0,688503 ± 0,015942 | **0,647941 ± 0,023741** | **42** |

Detalhamento dos nove treinos. “Época” é a do checkpoint escolhido por F1-macro de validação, não necessariamente a de menor perda.

| CABEÇA / SEMENTE | ÉPOCA | F1 VALIDAÇÃO | F1 TESTE | ACURÁCIA TESTE |
|---|---|---|---|---|
| satisfação / 13 | 1 | 0,760614 | 0,754337 | 0,830590 |
| satisfação / **42** | 4 | 0,758444 | 0,761466 | 0,847208 |
| satisfação / 77 | 1 | 0,754693 | 0,749664 | 0,817253 |
| emoção / **13** | 3 | 0,578321 | 0,597280 | 0,657751 |
| emoção / 42 | 5 | 0,577151 | 0,586892 | 0,659934 |
| emoção / 77 | 4 | 0,580685 | 0,592943 | 0,666075 |
| ironia / 13 | 4 | 0,675866 | 0,665176 | 0,681818 |
| ironia / **42** | 2 | 0,683228 | 0,620862 | 0,670996 |
| ironia / 77 | 4 | 0,706413 | 0,657787 | 0,686147 |

ECE top-1 no mesmo teste: **bruto, sem temperatura**, e **com temperatura ajustada somente na validação**.

| CABEÇA / SEMENTE | ECE BRUTO | ECE COM TEMPERATURA |
|---|---|---|
| satisfação / 13 | 0,023847 | 0,024383 |
| satisfação / **42** | 0,075406 | 0,020136 |
| satisfação / 77 | 0,030691 | 0,022800 |
| emoção / **13** | 0,106976 | 0,037097 |
| emoção / 42 | 0,158665 | 0,036317 |
| emoção / 77 | 0,129541 | 0,026192 |
| ironia / 13 | 0,056899 | 0,033080 |
| ironia / **42** | 0,047387 | 0,066038 |
| ironia / 77 | 0,135616 | 0,057866 |

A mediana retrospectiva do **teste** seria 13/77/77, respectivamente; ela **não** substituiu 42/13/42. Temperatura não garante melhora de ECE em todo teste: piorou na satisfação 13 e na ironia 42. Não escolher outra temperatura ou semente por esse resultado final.

## CURVAS COMPARÁVEIS: EVIDÊNCIA DE SOBREAJUSTE

São medições por época em modo **eval**, sem dropout, no diagnóstico fixo estratificado de treino (até 2.000 exemplos, semente 42) e na validação, com a mesma função de perda ponderada. Não são médias de

minibatches do treino. As séries completas observadas estão abaixo; o treino diagnóstico não é uma estimativa em dados independentes.

| CABEÇA / SEMENTE | ÉPOCA | PERDA TREINO EVAL | PERDA VALIDAÇÃO | F1 TREINO EVAL | F1 VALIDAÇÃO |
|---|---|---|---|---|---|
| satisfação / 13 | 1 | 0,4540 | 0,5316 | 0,8015 | 0,7606 |
| satisfação / 13 | 2 | 0,3661 | 0,5393 | 0,8063 | 0,7474 |
| satisfação / 13 | 3 | 0,2742 | 0,5789 | 0,8581 | 0,7545 |
| satisfação / 42 | 1 | 0,4754 | 0,5455 | 0,7538 | 0,7195 |
| satisfação / 42 | 2 | 0,3681 | 0,5231 | 0,8314 | 0,7567 |
| satisfação / 42 | 3 | 0,2823 | 0,5619 | 0,8349 | 0,7424 |
| satisfação / 42 | 4 | 0,2231 | 0,7633 | 0,9054 | 0,7584 |
| satisfação / 42 | 5 | 0,1816 | 0,8071 | 0,9165 | 0,7560 |
| satisfação / 77 | 1 | 0,4444 | 0,5172 | 0,7930 | 0,7547 |
| satisfação / 77 | 2 | 0,3507 | 0,5359 | 0,8343 | 0,7513 |
| satisfação / 77 | 3 | 0,2565 | 0,6102 | 0,8620 | 0,7488 |
| emoção / 13 | 1 | 0,8665 | 1,1082 | 0,5909 | 0,5518 |
| emoção / 13 | 2 | 0,6800 | 1,1377 | 0,6426 | 0,5593 |
| emoção / 13 | 3 | 0,4778 | 1,3006 | 0,7625 | 0,5783 |
| emoção / 13 | 4 | 0,3429 | 1,3887 | 0,7960 | 0,5729 |
| emoção / 13 | 5 | 0,3100 | 1,4869 | 0,8175 | 0,5769 |
| emoção / 42 | 1 | 0,8804 | 1,0785 | 0,5890 | 0,5567 |
| emoção / 42 | 2 | 0,6518 | 1,0771 | 0,6619 | 0,5691 |
| emoção / 42 | 3 | 0,4384 | 1,2388 | 0,7589 | 0,5729 |
| emoção / 42 | 4 | 0,3380 | 1,3445 | 0,8057 | 0,5709 |
| emoção / 42 | 5 | 0,2917 | 1,4710 | 0,8454 | 0,5772 |

| CABEÇA / SEMENTE | ÉPOCA | PERDA TREINO EVAL | PERDA VALIDAÇÃO | F1 TREINO EVAL | F1 VALIDAÇÃO |
|---|---|---|---|---|---|
| emoção / 77 | 1 | 0,8492 | 1,0976 | 0,5838 | 0,5311 |
| emoção / 77 | 2 | 0,6131 | 1,0670 | 0,6961 | 0,5805 |
| emoção / 77 | 3 | 0,4550 | 1,1954 | 0,7433 | 0,5741 |
| emoção / 77 | 4 | 0,3308 | 1,3682 | 0,8109 | 0,5807 |
| emoção / 77 | 5 | 0,2968 | 1,4625 | 0,8246 | 0,5760 |
| ironia / 13 | 1 | 0,6619 | 0,6637 | 0,5546 | 0,4816 |
| ironia / 13 | 2 | 0,5648 | 0,6067 | 0,7101 | 0,6546 |
| ironia / 13 | 3 | 0,4430 | 0,5903 | 0,7829 | 0,6638 |
| ironia / 13 | 4 | 0,3703 | 0,6040 | 0,8391 | 0,6759 |
| ironia / 13 | 5 | 0,3441 | 0,6132 | 0,8608 | 0,6746 |
| ironia / 42 | 1 | 0,6415 | 0,6541 | 0,6574 | 0,5798 |
| ironia / 42 | 2 | 0,5308 | 0,6095 | 0,7471 | 0,6832 |
| ironia / 42 | 3 | 0,3764 | 0,6069 | 0,8175 | 0,6430 |
| ironia / 42 | 4 | 0,2986 | 0,6320 | 0,8714 | 0,6520 |
| ironia / 77 | 1 | 0,6073 | 0,6280 | 0,6501 | 0,6199 |
| ironia / 77 | 2 | 0,4684 | 0,5897 | 0,7915 | 0,6826 |
| ironia / 77 | 3 | 0,3241 | 0,5940 | 0,8615 | 0,6925 |
| ironia / 77 | 4 | 0,2517 | 0,6364 | 0,9152 | 0,7064 |
| ironia / 77 | 5 | 0,2337 | 0,6524 | 0,9220 | 0,7029 |

- **Satisfação:** perda de treino cai enquanto a de validação sobe nas épocas 1–3 de 13/77 e 2–5 de 42. O F1 de validação não acompanha o ganho do treino.
- **Emoção:** o padrão de perdas persiste em 1–5 de 13 e 2–5 de 42/77. O F1 de validação ainda melhora em algumas épocas, mas muito menos que o de treino; seleção por F1 e seleção por perda não são a mesma decisão.
- **Ironia:** perda de validação sobe em 3–5 de 13 e 2–5 de 77 enquanto o treino melhora. Na 42, o F1 perde o pico da época 2; a perda só aumenta de 3 para 4, não desde a época 2. Não atribuímos uma subida monotônica inexistente.

Esses padrões sustentam **sobreajuste nas fases indicadas desta reprodução**, não uma causa única, um limiar universal de gap ou um diagnóstico dos pesos publicados. O checkpoint continuou selecionado somente pela validação.

### TRANSFERÊNCIA E RÉGUA OPERACIONAL DOS CANDIDATOS ESCOLHIDOS

| CANDIDATO / BENCHMARK | N | F1-MACRO | ACURÁCIA | ECE COM TEMPERATURA |
|---|---|---|---|---|
| satisfação 42 / ConsumerBR amostrado | 5.000 | 0,573398 | 0,756600 | 0,063823 |
| emoção 13 / XED-pt integral | 4.286 | **sete classes 0,233188; seis presentes 0,272053** | 0,278115 | 0,371403 |
| ironia 42 / régua de atendimento já consultada | 20 | 0,520000 | não registrada | não registrado |

O notebook 04 também registrou a régua operacional em `ironia-generalizacao-20261026/metricas.json`, campo `regua_20`: dois falsos positivos e sete falsos negativos. Não é um corpus novo de atendimento; acurácia e ECE dessa régua não constam nesse relatório, por isso ficam ausentes. ConsumerBR amostrado não colide textualmente com os 95.722 exemplos de treino B2W; isso não o torna teste virgem nem prova independência por empresa/autor. XED tem seis colisões textuais com os 36.806 exemplos de treino GoEmotions, duas com conflito de rótulo; o benchmark integral foi preservado, sem limpar retrospectivamente o holdout para melhorar a comparação. Grupos entre fontes não são comparáveis. Não há ECE bruto de transferência no resumo: não inferir uma melhora de calibração sem essa medição.

As quedas fora do corpus-fonte descrevem transferência sob mudança de domínio, distribuição e alvo; não isolam causalmente sobreajuste. XED ainda usa rótulos projetados entre línguas e não contém neutro. Nenhum dos dois é um corpus novo de atendimentos reais rotulados.

### CANDIDATO DO FUSOR E ABLAÇÕES

São **900/120/300 conversas sintéticas** de treino/validação/teste, equilibradas por classe em cada lado, com textos dos grupos correspondentes do manifesto B2W reservado antes do treino de satisfação. Usaram os candidatos escolhidos pela validação; não os baselines locais antigos. Features excluídas foram removidas da matriz, cada scaler foi ajustado somente no treino, e as quatro esperas mantêm `log1p`. O cache vincula manifesto, checkpoints e contrato.

| VARIANTE EXPERIMENTAL | FEATURES | F1 VALIDAÇÃO | F1 TESTE | ACURÁCIA TESTE | ECE TESTE |
|---|---|---|---|---|---|
| completo | 39 | 0,932486 | **0,931557** | 0,933333 | 0,030091 |
| somente texto | 4 | 0,836454 | **0,883318** | 0,886667 | 0,044542 |
| somente tempo | 7 | 0,509600 | **0,558372** | 0,573333 | 0,048255 |

Há uma execução por variante: **não há média/desvio de três sementes do fusor**. O ECE é das probabilidades do candidato; não houve nova temperatura do fusor nem avaliação conformal nesta campanha. Essas ablações não substituem o runtime nem medem causalidade dos sinais.

O relatório original do instrumento conservou a ressalva de independência não verificável da emoção, pois só examina fingerprints agregados de outro corpus. Ele **não foi sobrescrito**. A prova complementar `independencia-sistema.json` confirma os fingerprints dos treinos reais das três cabeças e encontra zero colisões textuais normalizadas com validação e teste B2W. Em satisfação, há também isolamento por produto no manifesto. Isso estabelece independência **observável por esses identificadores nesta reprodução**; não cobre paráfrases, traduções, proveniência ausente ou grupos incomparáveis entre fontes. Não altera a insuficiência histórica dos pesos antigos, nem converte o teste sintético em generalização comprovada em suporte.

### 2.4.6 Protocolo da campanha fechada

A receita vigente fixa sementes **13, 42 e 77**, no máximo cinco épocas e os mesmos hiperparâmetros nas três rodadas de cada cabeça. Não há busca oportunista de encoders, regularizadores ou novos corpora. É uma **nova reprodução datada**, não recuperação do histórico perdido.

Satisfação usa B2W completo após split, sem ConsumerBR auxiliar; emoção usa GoEmotions-PT; ironia usa MultiPICo em português **sem IDPT auxiliar**. A divisão usa semente 42, com 75/10/15 para B2W e GoEmotions e 60/15/25 para MultiPICo, respeitando os grupos disponíveis.

Checkpoint e temperatura usam validação. A semente mediana é escolhida por F1-macro de validação; configuração e seleção são gravadas antes da avaliação final das três sementes no teste congelado. O fingerprint divergente bloqueia a avaliação. Não selecionar retrospectivamente outra semente pelo teste.

Por época, preservar perda e F1 de validação e de um subconjunto diagnóstico fixo de treino (até 2.000 exemplos, seed 42, modo eval e mesma perda ponderada). Relatar melhor época, argumentos efetivos, F1-macro, acurácia, precisão, recall, F1 e suporte por classe, matriz de confusão e ECE antes/depois da temperatura. As três sementes fornecem média e **desvio amostral**. Não existe um limiar universal de gap que diagnostique sobreajuste.

#### VERIFICAÇÃO DE IMPLEMENTAÇÃO NÃO É RESULTADO CIENTÍFICO

Em 26/10/2026, a verificação integrada da receita registrou 77 testes focados aprovados e um smoke real com BERT **aleatório e minúsculo**, treinado por duas épocas em dados construídos. O smoke salvou/recarregou o modelo, executou a avaliação final separada e recusou manifesto divergente. As 18 células Python dos três notebooks foram compiladas; esse smoke local não executou o caminho de GPU. Isso verifica contratos do código, **não** mede satisfação, emoção, ironia ou generalização do Fraus. A campanha T4 posterior é evidência separada, acima.

#### TESTE DO SISTEMA E ABLAÇÕES DO FUSOR

Reservar os grupos do teste B2W **antes** de treinar a cabeça de satisfação. O manifesto explícito compartilhado entre notebook 01 e treino do fusor registra origem, índices aceitos, hashes de texto/grupo e rótulos de satisfação por lado. `--manifesto` é obrigatório no script local; caches são vinculados a esse manifesto, bytes dos checkpoints, features/`log1p`, simulador e léxicos. Cache antigo sem essa identidade não é comparável por nome de arquivo.

O fusor continua usando recomendação e neutro de três estrelas; o rótulo de satisfação do manifesto identifica a divisão da cabeça, não troca o alvo do fusor. Treino, validação e teste do fusor só usam seus respectivos grupos. Comparar completo, somente texto e somente tempo removendo colunas e ajustando cada scaler **só no treino**. Nenhuma ablação é artefato de runtime.

O experimento concluído preserva o relatório conservador original e o complementa com a comparação textual dos treinos reais, conforme a seção de resultados. Conversas sintéticas continuam não sendo atendimentos reais rotulados, ainda que os identificadores observados sejam disjuntos.

### 2.4.7 Como interpretar as conclusões

1. **Vazamento/dependência observada:** os 41 produtos atravessam os pools da receita posicional reconstruída. Os atalhos históricos de latência e de marcadores de ironia estão documentados no diário; não são diagnóstico automático dos pesos publicados hoje.
2. **Sobreajuste:** as curvas novas mostram treino melhorando e perda de validação piorando persistentemente nos intervalos explicitados acima. A conclusão vale para **esta reprodução de 26–27/10**, sem limiar universal de gap e sem diagnóstico causal. Para pesos antigos/publicados, faltam curvas e proveniência: o diagnóstico continua **inconclusivo**, não “sem sobreajuste”.
3. **Transferência:** ConsumerBR, XED e as réguas de ironia já foram consultados. Medem desempenho em outro domínio sob essas condições, não teste virgem. Para XED, distinguir **F1-macro nas sete classes** de F1-macro somente nas classes presentes; não comparar os dois como se fossem a mesma métrica. MultiPICo anota ironia vendo o post; o modelo lê só a resposta, uma limitação contextual adicional.
4. **Insuficiência:** faltam curvas e proveniência completa dos treinos **históricos**, checkpoint publicado identificado e corpus novo de atendimento rotulado. Nem o baseline isolado de uma cabeça nem o teste sintético novo autorizam promessa de acurácia operacional em suporte real.

Para os limites de uso, veja Limitações e Conformidade. O método e a hipótese estão em A tese.

# 2.5 CARTÃO DO MODELO

> Formato de *model card* (Mitchell et al., 2019): o que cada peça é, com o que foi treinada, o que mediu, **onde erra** e para que **não** pode ser usada. **Métricas históricas de 15/09/2026**, não uma reavaliação do checkpoint publicado hoje. As fatias foram medidas com `scripts/avaliar_por_fatias.py` (`--n 600`, semente `20260915`). A investigação de 26–27/10/2026, os baselines locais e a campanha concluída de três sementes por cabeça estão em Generalização e evidências; não sobrescrevem estas medições nem identificam os pesos publicados. As curvas sustentam sobreajuste **dos candidatos novos**, não um diagnóstico dos modelos antigos.

## 2.5.1 Uso pretendido e uso proibido

**Pretendido:** estimar, a partir do texto e do ritmo de um atendimento de chatbot, se o cliente saiu insatisfeito, neutro ou satisfeito, e agregar isso num **NPS estimado** para quem opera o atendimento enxergar tendência e caso problemático. Toda exibição carrega a etiqueta de estimativa: o NPS é **inferido**, não perguntado.

**Proibido:**

- **Pontuar atendente.** "Score por atendente" é expansão de finalidade de alto impacto e permanece proibida por política do produto. O Fraus infere emoção de texto, não de dados biométricos; por isso a documentação não afirma mais enquadramento automático na definição do EU AI Act. Ver Conformidade.
- **Decidir sobre o cliente individualmente** (crédito, prioridade, cobrança). A nota de uma conversa é evidência para um humano olhar, não veredito.
- **Apresentar a nota como NPS declarado.**

### 2.5.2 As peças — métricas históricas de 15/09/2026

| PEÇA | O QUE É | TREINO | MÉTRICA DECLARADA | ONDE A MÉTRICA ENGANA |
|---|---|---|---|---|
| satisfação | BERTimbau, 3 classes por mensagem | B2W-Reviews01 (resenha de produto, não atendimento) | acurácia 0,783 · F1-macro 0,782 | domínio: resenha ≠ conversa de suporte |
| emoção | BERTimbau, 7 classes (+ desprezo derivado de raiva+nojo) | `go_emotions_ptbr` (tradução automática) | F1-macro 0,584; **0,281 no XED-pt** (sem tradução) | a queda no XED-pt é a medida honesta |
| ironia | BERTimbau binário | corpus **gerado** (`fraus.ingest.gerador_ironia`) | acurácia 1,0 | otimista por construção — ver abaixo; **não entra na nota** |
| fusor | `LogisticRegression` sobre 39 features (espera em `log1p`) | conversas sintéticas: texto do B2W + estrutura e tempo do simulador | acurácia 0,947 · F1-macro 0,947 (holdout do notebook 02) | o tempo é sintético; ver Treinamento |

Os três BERTimbau rodam em CPU, por torch ou por ONNX Runtime (`FRAUS_BACKEND`), com **0 de 180 categorias trocadas** entre os dois executores. Ver Encolher os modelos.

### 2.5.3 Onde o fusor erra — avaliação histórica por fatia

600 conversas do simulador, fora das sementes usadas em outras medições. **Isto mede onde o modelo erra dentro do domínio sintético, não desempenho em atendimento real**: o projeto não dispõe de corpus PT-BR de atendimento rotulado com timestamps para essa validação.

Acurácia geral: **0,940** (598 conversas com sinal; 2 eram só cortesia).

| EIXO | FATIA | N | ACURÁCIA | F1-MACRO |
|---|---|---|---|---|
| falas do cliente | 1 | 30 | **0,733** | **0,735** |
| falas do cliente | 2–3 | 424 | 0,936 | 0,936 |
| falas do cliente | 4+ | 144 | 0,993 | 0,992 |
| emoji | com | 524 | 0,952 | 0,953 |
| emoji | sem | 74 | **0,851** | **0,836** |
| escalou para humano | não | 454 | 0,947 | 0,948 |
| escalou para humano | sim | 144 | 0,917 | 0,887 |
| latência mediana | < 30 s | 236 | 0,936 | 0,918 |
| latência mediana | 30–180 s | 306 | 0,941 | 0,934 |
| latência mediana | > 180 s | 56 | 0,946 | 0,823 |

O que a tabela diz, e o que ela **não** pode dizer:

1. **Conversa de uma fala só é a fatia mais fraca** (0,733). É esperado — há menos texto para agregar —, e é o caso comum de "cliente respondeu uma vez e sumiu". A nota dessas conversas merece menos confiança do que a das longas.
2. **Sem emoji, o acerto cai 10 pontos.** Suspeita declarada: o simulador sorteia o emoji **por rótulo** (`EMOJIS_POR_ROTULO`), então parte do acerto "com emoji" pode ser o modelo lendo uma pista que o gerador plantou — a mesma família de defeito da invariante 10. Isso não permite prever a acurácia em atendimento real; a linha “sem emoji” ainda pertence ao simulador.
3. **A fatia de latência é parcialmente circular** pelo mesmo motivo: o simulador sorteia a latência por rótulo. O F1 menor acima de 180 s é a fatia a acompanhar: mesmo com a espera em `log1p` desde 15/09/2026, o relógio ainda empata com o texto por volta de 10 min (`scripts/medir_dominio_do_tempo.py`).

### 2.5.4 Conversa só de cortesia — sem sinal desde 15/09/2026

"ok, obrigado" fecha atendimento bom **e** ruim, então não tem rótulo verdadeiro. Medido antes da regra, com o bot respondendo em 10 s, o Fraus dava:

| FALA | SCORE (ANTES) | CATEGORIA (ANTES) | HOJE |
|---|---|---|---|
| ok, obrigado | 76,2 | neutro | sem sinal |
| ta bom | 75,7 | neutro | sem sinal |
| ok | 79,0 | neutro | sem sinal |
| certo, entendi | 75,3 | neutro | sem sinal |
| valeu | 93,2 | **promotor** | sem sinal |
| obrigada | 92,9 | **promotor** | sem sinal |

Uma palavra de agradecimento, sozinha, bastava para **promotor**: o texto de gratidão era lido como satisfação, e a emoção lia "ok, obrigado" como alegria 0,98 — o mapeamento do GoEmotions para as 6 de Ekman juntou `gratitude` com `joy`. **Decisão:** conversa em que *toda* fala do cliente é fórmula de cortesia é **sem sinal** (`score: None`, invariante 2), e não sinal fraco. A lista é fechada e conservadora (`fraus/cortesia.py`): só casa a fala inteira, e "valeu, resolveu na hora" continua pontuada. O conserto de verdade é retreinar a emoção separando gratidão de alegria.

### 2.5.5 A cabeça de ironia — taxa de erro histórica publicada

Contra-exemplos escritos para quebrar o atalho que o gerador ensinou: falas **sinceras com marcador de discurso** ("nossa", "realmente", "né", "que ...") e **ironias sem marcador**, em que o elogio é desmentido pela situação.

| CONJUNTO | ERRO |
|---|---|
| sincera com marcador marcada como irônica | **70%** (7 de 10) |
| irônica sem marcador que passou como sincera | **40%** (4 de 10) |

Os erros vêm saturados: "realmente, agora funcionou" → 0,999 de ironia; "perfeito, cancelaram sem avisar" → 0,001. A cabeça aprendeu **registro e marcador**, não pragmática. É por isso que ela **não entra na nota** desde 04/09/2026 e que a interface a mostra com ressalva, nunca como veredito. A lista completa, fala a fala, sai do script.

### 2.5.6 Calibração e conformal do fusor

O fusor dispõe de conjunto conformal (Mondrian por classe, sobre o score gravado) e instrumentos de calibração. O protocolo histórico dividia um holdout B2W de 300 conversas em calibração (150) e avaliação (150), estratificado, com semente fixa. **Nenhum número da tabela abaixo foi medido**: o fusor compatível com `log1p` não estava disponível na máquina de desenvolvimento em 14/10/2026. O cache posicional antigo não comprova isolamento por produto ou independência das cabeças e não deve ser reutilizado no novo experimento.

A campanha de 26–27/10/2026 produziu um **candidato novo**, com avaliação sintética e ECE top-1, descritos em Generalização. Essa medição não preenche retroativamente a tabela histórica abaixo: MCE, calibração por classe/fatia e cobertura/abstenção conformal não foram medidos nessa campanha. O relatório original conservador de independência foi preservado e complementado por cruzamentos textuais dos treinos reais; isso não transforma conversas sintéticas em atendimento real rotulado.

### COMANDOS ATUAIS PARA PREPARAR UM CANDIDATO

Defina `B2W` como o CSV exato, `EXPERIMENTO` como diretório de pesquisa **externo ao repositório e aos checkpoints** e `CABECAS` como a raiz dos checkpoints candidatos. Reserve o manifesto **antes** do treino de satisfação e passe-o ao notebook 01 por `FRAUS_MANIFESTO_FUSOR`; só execute a extração depois de congelar os candidatos. Não há opção de promoção nesta sequência.

```
# 1. Reserva a divisão agrupada; não faz inferência.
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" --gerar-manifesto
# 2. Extrai features e avalia candidatos/ablações, sem ler ou substituir fusor ativo.
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" \
  --modelos "$CABECAS" --backend torch \
  --cache "$EXPERIMENTO/features-fusor.json" --saida "$EXPERIMENTO/candidato"
# 3. Instrumentos sobre o candidato explícito, sem ler fusor ativo ou gravar conformal ativo.
# É outra medição; não sobrescreve a tabela histórica abaixo.
uv run python scripts/calibrar_conformal.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" --cache "$EXPERIMENTO/features-fusor.json"
uv run --with matplotlib python scripts/medir_calibracao.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" \
  --cache "$EXPERIMENTO/features-fusor.json" --png "$EXPERIMENTO/calibracao.png"
uv run python scripts/medir_abstencao.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" --cache "$EXPERIMENTO/features-fusor.json"
```

| MEDIDA | VALOR |
|---|---|
| ECE (top-1) | não verificado — fusor log1p ausente na máquina de desenvolvimento em 14/10/2026 |
| MCE | não verificado — idem |
| ECE por classe | não verificado — idem |
| ECE por fatia (falas, latência, emoji, escalou; "n/d" abaixo de 5 exemplos) | não verificado — idem |
| Cobertura empírica no alvo de 90% (total e por classe) | não verificado — idem |
| Acurácia das conversas que não abstêm (só o critério do conjunto; a rota de detalhe também abstém por deriva) | não verificado — idem |
| Taxa de abstenção no alvo de 90% | não verificado — idem |

Garantia: cobertura ≥ alvo **por classe**, sob permutabilidade com o conjunto de calibração; **não vale** sob mudança de distribuição (atendimento real ≠ holdout sintético); o sinal de tempo do fusor é sintético. Com 150

exemplos de avaliação a cobertura tem granularidade de cerca de 2 a 3 pontos percentuais. Os scripts exigem no mínimo 5 exemplos por classe em cada metade (escolha de projeto, sem procedência na literatura), e a mediana do bootstrap é impressa com uma nota de viés.

### 2.5.7 Limitações que valem para tudo acima

- **Tempo sintético.** Latência, escalação e abandono do treino do fusor vêm de distribuições da literatura, não de atendimento observado.
- **Texto fora de domínio.** Satisfação aprendeu com resenha de produto; emoção, com Reddit traduzido por máquina.
- **Acurácia alta pede investigação**, não celebração automática (invariante 10). Os 0,947 e 0,940 acima são medidas sintéticas históricas, não uma estimativa ou um teto demonstrado de desempenho em atendimentos reais.

Ver também Limitações conhecidas.

# 2.6 LIMITAÇÕES CONHECIDAS

Esta página existe porque um sistema que estima satisfação **precisa** declarar onde erra. Nada aqui é segredo, e nada aqui está escondido na tela: cada item tem contrapartida visível na interface.

## 2.6.1 Generalização e proveniência — atualização de 27/10/2026

O relatório público de generalização centraliza protocolo, baselines locais, resultados dos nove treinos e ablações do fusor. As medições datadas nas demais seções abaixo são **históricas**: não identificam automaticamente os pesos publicados hoje. Uma resposta de saúde da API não demonstra inferência real nem proveniência do checkpoint.

Há dependência por produto na antiga divisão posicional do fusor reconstruída em 26/10. Faltam curvas e manifestos históricos suficientes para diagnosticar sobreajuste dos pesos antigos/publicados: esse diagnóstico permanece **inconclusivo**, não “sem sobreajuste”.

A reprodução de 26–27/10 concluiu as três sementes por cabeça em T4 e o novo experimento sintético do fusor. Suas curvas comparáveis mostram treino melhorando enquanto a perda de validação piora persistentemente em fases explicitadas no relatório: **evidência de sobreajuste desses candidatos novos**, sem identificar uma causa única ou invalidar a seleção isolada por validação. Nenhum candidato foi promovido; esta medição não identifica o checkpoint em produção.

ConsumerBR e XED já foram consultados e não são testes virgens. A amostra ConsumerBR não tem colisões textuais normalizadas com o treino B2W novo; XED tem seis com o treino GoEmotions novo, duas com conflito de rótulo. No XED, F1-macro nas sete classes não é o mesmo indicador que F1-macro nas seis presentes. Queda fora de domínio, isoladamente, não diagnostica sobreajuste. A régua pequena de ironia tampouco valida atendimento real.

O manifesto do fusor novo isola produtos B2W. A prova complementar confirma zero colisões textuais normalizadas da validação/teste B2W com os treinos reais das três cabeças novas, sem sobrescrever o relatório conservador original. Essa independência observável não cobre paráfrases, traduções ou proveniência ausente; grupos entre fontes não são comparáveis. Nada disso comprova retroativamente a independência das cabeças antigas, nem valida acurácia em suporte real: as conversas e os tempos do experimento do fusor são sintéticos.

## 2.6.2 Limitações operacionais do deploy gratuito

A hospedagem da API deixou de ser pendência em 17/09/2026, mas “publicada” não significa “sem limites”. A função ONNX tem cerca de 1,95 GB (pacote publicado em 30/09/2026; eram 1,46 GB em 17/09) e usa Large Functions na Vercel. Depois de esfriar, o primeiro `/saude` observado levou 10,8 s; requisições seguintes foram rápidas, mas não há garantia de latência.

**Cold start e "motor aquecendo".** A API é serverless: cada instância tem o próprio motor, e ele só ficava pronto depois da primeira predição. Desde 30/09/2026 o motor aquece em segundo plano no boot da instância e a cada sonda de saúde, e um workflow sonda a API a cada 5 minutos. Isso **reduz** o cold start, mas não o elimina — a Vercel não garante que a sonda e o usuário caiam na mesma instância. Eliminá-lo exige um processo sempre

ligado com pelo menos 1,5–2 GB de RAM; o plano gratuito do Render (512 MB, dorme em 15 min) não serve, porque a API mede cerca de 1,06 GB de RSS com os três modelos.

Vercel Hobby, Supabase Free e Oracle Object Storage gratuito oferecem cotas, não SLA. Ao ultrapassá-las, a aplicação pode ser limitada ou pausada. O estado fica no Supabase; o Oracle participa do build, não da inferência. Uma restauração do projeto pode alterar a URI do pooler, e o transaction pooler precisa usar a porta indicada no painel (6543 no deploy atual).

Essas limitações são operacionais. Elas não alteram as limitações científicas abaixo e não autorizam apresentar a estimativa como NPS declarado.

---

### 2.6.3 O NPS é inferido, não perguntado

Toda exibição carrega a etiqueta de estimativa e o sublinhado pontilhado de proveniência. Apresentar como NPS declarado seria falso.

**O intervalo de confiança cobre a incerteza AMOSTRAL, não a do modelo.** Ele responde "quantas conversas sustentam este número"; ele **não** responde "o quanto o modelo pode estar errado". A temperatura gravada no artefato (`fraus_receita.json`, desde 19/10/2026) calibra a confiança de cada cabeça, mas não entra neste intervalo. Um IC apresentado como se cobrisse o erro do modelo é pior que não ter IC nenhum, e por isso a ressalva está impressa junto do número, não só aqui.

O intervalo usa **Wald ajustado triangular AW(3,T)**: pseudocontagens de 0,75 para cada extremo e 1,5 para neutros, com variância dividida por `n + 2`, conforme Turk, Cinderich e McNeill (2026). Isso evita intervalo de largura zero quando todas as conversas pertencem à mesma categoria. O NPS pontual e a contagem continuam sendo os observados; o ajuste só participa dos limites do intervalo.

Abaixo de **30 atendimentos com sinal** o cartão principal e a leitura do intervalo ocultam o ponto estimado e mostram o `n`. Esse corte é uma regra de exibição, não uma garantia de precisão a partir de 30. A série diária e o campo `nps` da API preservam o agregado observado; `nps_intervalo.nps` é o campo que autoriza a exibição do ponto no cartão. Sem intervalo (API antiga ou falha do endpoint), a dashboard preserva o agregado disponível.

### CALIBRAÇÃO E CONJUNTO CONFORMAL DO FUSOR (E2)

Nada recalibra em runtime e o score gravado não muda. O que existe agora é um *conjunto conformal* (Mondrian, por classe, sobre o score já gravado) e um marcador de abstenção ("leitura sem confiança suficiente"), mais os instrumentos que medem a calibração (`fraus/calibracao.py`, `scripts/medir_calibracao.py`, `scripts/medir_abstencao.py`). A garantia é só esta: cobertura ≥ alvo **por classe**, sob permutabilidade com o conjunto de calibração. Ela **não vale** sob mudança de distribuição — e atendimento real é mudança de distribuição em relação ao holdout sintético do B2W —, e o sinal de tempo do fusor é sintético. A abstenção só marca: a conversa **continua no NPS e no CSAT**. Sem artefato calibrado para a régua vigente as rotas respondem 503, nunca um conjunto vazio fingindo confiança. Os números de ECE, cobertura e acurácia por abstenção ainda **não foram medidos**: não verificado — fusor log1p ausente na máquina de desenvolvimento em 14/10/2026. Por conversa, basta uma feature rara longe da média de treino (|z| > 4; por exemplo, features binárias com prevalência de treino abaixo de ~6%, ou contagens de turnos acima da faixa do simulador) para disparar o critério "deriva"; em tráfego real, portanto, ele deve disparar com frequência — honesto por construção, não medido.

### SINALIZADORES POR REGRA (E5)

Cinco regras de superfície, derivadas na leitura e sem gravar nada: `loop_do_bot`, `cliente_repete`, `pedido_de_humano_ignorado`, `escalada_tardia` e `abandono`. Três limitações valem para todas. (1) São heurísticas: resposta legitimamente repetida ("posso ajudar em algo mais?") dispara `loop_do_bot` sem ser defeito. (2) O léxico de pedido de humano é fechado (`atendente`, `humano`, `humana`, `pessoa`, `operador`, `falar com alguém`) e casa por palavra inteira: pedido em outras palavras passa em branco, e o plural ("atendentes", "pessoas", "humanos") **não** casa (falso negativo declarado); já "a pessoa que me atendeu" conta como pedido (falso positivo declarado). (3) Os limiares (0,85, 0,7, "mais de 2" respostas, N = 3) são **sem procedência** e não foram medidos em dado real; o `detalhe` de cada achado diz isso. No `abandono`, o falso positivo comum é a despedida do bot depois que o cliente já resolveu e saiu. Nenhuma regra lê o texto nem o relógio da fala do atendente: só a presença e a posição da primeira fala humana contam, e o autor da última mensagem (usado pelo `abandono`); o texto e o relógio da fala humana nunca são lidos para julgar, porque o Fraus não pontua atendentes. Os casos construídos (15) **verificam a implementação, não medem qualidade**; precisão e recall em dado real: **não verificado — fonte da amostra anotada pendente**.

### IMPACTO POR TEMA E DETERIORAÇÃO (E6)

O impacto de um tema é a diferença entre o NPS inferido das conversas do tema e o do resto do recorte, com IC por bootstrap: é **associação, não causa**. Exige no mínimo 30 conversas com sinal nos dois lados; abaixo disso o impacto é nulo, nunca 0. O tema vem de TF-IDF com DBSCAN e é instável com pouco dado. A deterioração acha, numa conversa **encerrada**, o início da deterioração (`indice_virada`): a mensagem a partir da qual a estimativa por prefixo fica detratora e não volta. É **retrospectiva e em lote, nunca alerta ao vivo**. O lead time em dado sintético não prevê o de produção, e o simulador não gera trajetória (cada conversa sorteia as falas do cliente de um só rótulo), então a medição feita nele é artefato do gerador. Não há margem em torno do limiar: uma estimativa que oscila perto da fronteira de detrator marca o início com uma variação pequena (na demonstração, de cerca de 67 para 61 pontos). O início da deterioração não aumenta o recall sobre o score final; ela só acrescenta lead time. Medição com o fusor real: **não verificado — fusor log1p ausente em 14/10/2026** (`scripts/medir_deterioracao.py` sai com código 2 e diz isso).

---

## 2.6.4 O sinal de tempo é treinado em dados sintéticos

Nenhum corpus público de review em português tem timestamps de diálogo. O simulador (`fraus/ingest/simulador.py`) gera conversas determinísticas com latências calibradas por literatura de live chat.

**E isso teve uma consequência medida, não hipotética.** As features de latência não têm teto, e o corpus nunca passou de minutos (medianas de 5/30/200 s). Numa conversa com **3 horas** de espera, medida em 08/09/2026 contra a API real, com o fusor da época (em segundos crus):

```
latencia_primeira_resposta_s   -85,405
duracao_total_s                -35,759
latencia_mediana_s             -20,569
latencia_p90_s                 -12,326
texto_prob_satisfeito_media     +4,574   ← o texto inteiro
```

O tempo pesa **20× o texto**, e a conversa sai em `4,4e-24`. O peso *por unidade* continua pequeno (0,12–0,16); o que explode é o **z-score** que entra no `StandardScaler`.

**Conserto em 15/09/2026:** as quatro features de espera entram no vetor em `log1p` (`FEATURES_EM_LOG`, em `fraus/fusor.py`), o fusor foi retreinado nessa escala e `Fusor.carregar` recusa artefato treinado em segundos crus. A winsorização no p99 foi medida e recusada: fazia 3 h pesarem o mesmo que 10 min. Com `log1p` o relógio ainda pesa mais quanto maior a espera, mas cada vez mais devagar — 3 h pesam 2,1× o texto, não mais 20× (medições em Treinamento). O que continua valendo é a origem sintética do sinal: espera muito acima do treino segue fora da distribuição, e `GET /saude/deriva` torna o caso **visível** (com o fusor em segundos crus, essa conversa dava `|z| = 124,9` em `latencia_mediana_s`, contra um limiar de 4).

---

### 2.6.5 A cabeça de ironia é pouco confiável

As medições antigas desta página se referem a um checkpoint treinado em corpus gerado, que reportava acurácia `1.0` e falhava em frases de atendimento com alta confiança. O atalho de marcador de discurso foi observado nesse contexto; ele **não diagnostica por si só a cabeça local ou publicada hoje**.

Os baselines de 26/10/2026 e a campanha diagnóstica proposta com MultiPICo estão em Generalização; campanha proposta não é retreino executado. A ironia continua fora do vetor desde 04/09/2026 por outro motivo medido historicamente (ver Arquitetura), e é exibida por mensagem, com ressalva. Não há promessa de que detecte ironia corretamente em atendimentos reais.

---

### 2.6.6 Desprezo não é uma classe treinada

Nenhum corpus em português a anota. Ela é derivada da díade raiva + nojo (Plutchik, 1980) pela **média geométrica** — que exige as *duas* emoções juntas, enquanto a média aritmética daria meio ponto para raiva pura sem nojo nenhum, o que é raiva, não desprezo.

---

### 2.6.7 Nenhuma feature agregada reverte uma probabilidade saturada

A frase canônica — "*que atendimento maravilhoso, só esperei 3 horas*" — sai promotora sempre que o relógio não a contradiz.

`incongruencia_situacao_negativa` foi criada para alcançá-la e tem peso **−0,193**; ela não vence os **+2,78** de `texto_prob_satisfeito_media` quando o BERTimbau lê a frase como elogio sincero.

Por isso a **contestação** (08/09/2026) **marca em vez de corrigir**: quando `score > 95` e `latencia_mediana_s > 180`, a tela avisa — e o atendimento **continua contando no NPS**, com teste provando isso. Tirá-lo do agregado esvaziaria o indicador em silêncio se o limiar estivesse mal calibrado.

O limiar de score em 95 é o único número do projeto **sem procedência**, e está declarado assim no código e na spec. O de tempo vem de IJHCI 2025.

---

### 2.6.8 A explicação é do modelo, não do cliente

O "por que esta nota" desmonta o que o **fusor** fez, e isso tem limites que a tela não deve esconder:

- **O efeito por família não soma a nota.** A soma exata mora no eixo do logit (intercepto + contribuições = log da razão entre classes); o score é uma projeção não linear desses logits. Os pontos por família são uma leitura aproximada, e a tela os trata como tal.
- **Contribuição é associação, não causa.** Um peso alto diz o que o modelo aprendeu a associar ao rótulo, não o que faria o cliente mudar de opinião.
- **"Faixa de treino" quer dizer** `|z| ≤ 4` sobre média e desvio do scaler (o mesmo limiar de `fraus/deriva.py`). O artefato não guarda mínimo nem máximo, então o contrafactual de espera recusa o que sai dessa faixa.
- **O sinal de tempo é sintético**, e por isso o contrafactual de espera herda a limitação da seção acima: é uma hipótese sobre o modelo, não uma previsão.
- **Os drivers são uma amostra com semente** (padrão 50 conversas, teto 200), cada uma custando uma passagem de modelo. O intervalo vem de bootstrap sobre a amostra e mede incerteza amostral, não erro do modelo. O teto limita só o número de conversas, não de mensagens: uma amostra de conversas longas numa instância fria pode se aproximar do tempo limite do proxy.

---

### 2.6.9 O que não dá para consertar sem corpus de atendimento real

O projeto não dispõe de corpus novo de **diálogo de atendimento** rotulado e com proveniência suficiente para validar estas duas frentes:

1. **retreinar a cabeça de ironia** em domínio certo;
2. **a interação tempo × texto** como feature. Ela nasceria com o peso invertido: o texto vem do B2W, a latência sai de distribuição por rótulo, e satisfeito-e-lento está rotulado *satisfeito* por construção. Isso é leitura do corpus, não experimento.

# 2.7 CONFORMIDADE REGULATÓRIA — O FRAUS E O EU AI ACT

> Escrito em 10/09/2026 e corrigido em 28/09/2026. Este documento existe porque o Fraus **classifica emoção a partir de texto**. Ele descreve o escopo, o que o sistema infere, o que ele nunca decide sozinho e — a parte que virou regra de código — **o que ele está proibido de fazer**.

## 2.7.1 1. Por que este documento existe

`fraus/sinais/emocao.py` classifica **sete classes** — as seis emoções básicas de Ekman mais neutro — e deriva **desprezo** da díade raiva + nojo. O Regulamento (UE) 2024/1689 define "sistema de reconhecimento de emoções" como inferência baseada em **dados biométricos**. O Fraus recebe texto de chat, não imagem, voz ou outro dado biométrico; portanto não deve ser apresentado como enquadrado automaticamente nessa definição. Ainda há obrigações de proteção de dados, transparência e limitação de finalidade, que dependem do contexto concreto de implantação.

Esta documentação é orientação técnica, não parecer jurídico. O enquadramento de uma implantação comercial deve ser revisto por profissional qualificado.

## 2.7.2 2. Escopo do sistema

| | |
|---|---|
| **O que analisa** | transcrições de atendimento por chatbot, já encerradas |
| **Sobre quem infere** | o **cliente** — e apenas o cliente |
| **O que infere** | satisfação (score 0–100 → nota 0–10 → categoria NPS), emoção por mensagem, ironia por mensagem, indicadores agregados |
| **Quando roda** | em lote, sobre conversa terminada. Nunca em tempo real durante o atendimento |
| **O que decide sozinho** | **nada** |
| **Base do modelo** | três BERTimbau fine-tunados + regressão logística, inferência local em CPU |

## 2.7.3 3. As três obrigações que já valem, e a que vem

### 3.1 TRANSPARÊNCIA COMO COMPROMISSO DO PRODUTO

Mesmo sem afirmar o enquadramento no Art. 50(3), inferência sobre emoção não deve ser escondida de quem é afetado nem de quem interpreta o resultado.

**Como o Fraus cumpre:** toda tela onde a leitura de emoção aparece (`components/CabecasDeLeitura.tsx`, compartilhado pelo simulador e pela análise) carrega a etiqueta de exposição, em linguagem de gente, junto da nota metodológica que já qualificava o NPS como inferido. A obrigação de *informar o cliente final* é de quem opera o chatbot — o Fraus fornece o texto e diz que ela existe; ele não fala com o cliente.

### 3.2 LIMITE DE FINALIDADE: NÃO PONTUAR TRABALHADORES

O Art. 5(1)(f) proíbe determinados sistemas biométricos de inferência de emoção no trabalho. O Fraus textual não deve reivindicar que essa proibição específica se aplica automaticamente a ele. Ainda assim, pontuar atendentes seria uma expansão de finalidade de alto impacto e permanece proibida por política do produto.

> REGRA: **O FRAUS NÃO PONTUA ATENDENTES.**
>
> O atendente humano aparece no transcript — a escalação é parte do dado — e o classificador de texto **recusa pontuar mensagem que não seja do cliente** (`sinais/texto.py`; `qtd_humano` e `latencia_mediana_humano_s` são contagem e relógio, nunca julgamento). "Score de performance por atendente" é a sugestão mais recorrente em analytics de atendimento, e no Fraus ela está **fechada por desenho**, não esquecida no backlog.
>
> Vale com o mesmo peso das invariantes do `CLAUDE.md`: implementar isso não é feature nova, é violação.

### 3.3 CONTROLES RECOMENDADOS PARA UMA IMPLANTAÇÃO COMERCIAL

Não se afirma aqui que o Fraus textual seja automaticamente “alto risco” pelo Anexo III. Os controles abaixo continuam recomendados porque tratam dados de atendimento e inferências sobre pessoas.

Onde o Fraus já está, hoje, em relação a cada uma:

| OBRIGAÇÃO | ESTADO NO FRAUS |
|---|---|
| **Supervisão humana** | o sistema não decide nada. O analista curou o léxico (`sinais/curadoria.py`), move as faixas de NPS (`PUT /configuracoes`) e lê os transcritos. Nenhuma saída aciona ação automática sobre ninguém |
| **Logging** | `entregas_webhook` registra toda entrega e toda recusa, com veredito e motivo; `auditoria_eventos` guarda eventos encadeados por SHA-256. O encadeamento detecta alteração no meio da cadeia, mas não a remoção dos eventos mais recentes nem a reescrita integral (não há âncora fora da tabela) |
| **Transparência** | etiqueta de estimativa em todo número inferido; limitações declaradas em tela e em `docs/treinamento.md` |
| **Monitoramento pós-mercado** | `GET /saude/deriva` reporta quais features estão fora da faixa de distribuição em que o modelo foi treinado |
| **Avaliação de conformidade** | não feita. É trabalho de quem for colocar isto em produção comercial, e está fora do escopo do TCC |
| **Avaliação de impacto em direitos fundamentais** | não feita, pelo mesmo motivo |

**Isto não é uma declaração de conformidade.** É um mapeamento honesto do que já está de pé e do que não está.

### 2.7.4 4. Limitações que importam para quem lê um número do Fraus

- **O NPS é inferido do texto, não perguntado.** O intervalo de confiança que a interface mostra cobre a **incerteza amostral** — não a incerteza do modelo.
- **A cabeça de ironia é pouco confiável** e por isso **não pontua** desde 04/09/2026: no corpus de treino ela funciona como detector de sentimento positivo. Continua sendo exibida, com a ressalva colada.
- **O sinal de tempo é treinado em dados sintéticos**, calibrados por literatura, porque nenhum corpus público em português tem timestamps de diálogo. Desde 15/09/2026 a espera entra em `log1p` e pesa cada vez mais devagar (3 h ≈ 2,1× o texto), mas espera fora da faixa do treino continua fora da distribuição — é limitação medida e declarada, e `GET /saude/deriva` a torna visível.
- **Desprezo não é classe treinada**; é derivado da díade raiva + nojo.
- **Ausência de dado nunca vira zero.** Conversa sem fala do cliente sai "sem sinal", em toda superfície.

### 2.7.5 5. Fontes

- Regulamento (UE) 2024/1689 — texto oficial
- Art. 50 — obrigações de transparência
- Diretrizes de transparência da Comissão Europeia

- Obrigações de transparência em vigor desde 02/08/2026 — Cooley
- Emotion AI de cliente vira alto risco — CX Today

## 2.8 IRONIA: CORPUS E LAYA

> **Estado em 08/10/2026.** O BERTimbau de ironia foi retreinado no notebook 04 com a divisão por autor corrigida (treino 28.365 linhas, 58,8% irônicas; teste 4.778, 59,2%). Deu F1-macro de **0,994** no teste interno (0,991 só nos tweets) e P(irônico) = **1,000** em frases neutras de atendimento, como "qual o prazo de entrega do meu pedido" e "ok, obrigado". O modelo **não foi exportado** e o Laya não foi treinado. Esta página explica por quê e o que fazer antes do próximo treino. Medições e passos da sessão: nota interna `docs/notas/2026-10-08-ironia-corpus.md`.
>
> **Produção não muda:** a ironia em produção é lida pelo Laya sem treino (`laya-onnx`) e não entra no vetor do Fusor.

### 2.8.1 Resumo: consertar o corpus antes de treinar o Laya

Treinar o Laya (mmBERT-base, `laya==0.3.21`) só em ironia é fácil de fazer e não resolve o problema que motivou a ideia. O vício do IDPT 2021 está nos dados, não no modelo nem na mistura de tarefas: a classe não irônica dos tweets é **100% de tweets com #economia**, publicados por ~658 contas, quase todas portais de notícia, só em maio e junho de 2017 (arquivo bruto). As notícias foram rotuladas **pelo site de origem**, "sem nenhuma rotulação manual" (Marten & de Freitas, ENIAC 2020). Qualquer encoder afinado nesse rótulo aprende registro e procedência, seja o Laya sozinho, o Laya multitarefa ou o BERTimbau. O formato de pergunta do Laya não protege contra isso, porque a instrução e os critérios são tokens idênticos em todos os exemplos e o gradiente vem só do texto. Chegar a "100% de exatidão" nesse teste não seria a prova esperada. Pela invariante 10 do projeto, seria o sintoma: ironia é uma tarefa em que anotadores humanos concordam pouco, e um acerto perfeito mostra que o teste mede a fonte. O ganho real virá de três coisas: trocar o negativo por fala coloquial sincera, medir numa régua de domínio com 200 a 400 itens em pares mínimos e rodar três treinos que separem as causas. Pela literatura, a diferença de encoder entre o Laya e o BERTimbau deve ficar em poucos pontos. Contra o multitarefa de 02/10, o Laya só-ironia quase certamente vai "ganhar" no teste interno, mas esse ganho virá da divisão corrigida, e não do fato de treinar só ironia.

### 2.8.2 O IDPT entrega o rótulo, e nenhum encoder escapa disso

O achado de 08/10 confirma o que os próprios arquivos mostram. No `nao-ironico.csv`, o commit `b735dd9` que o Fraus fixa, as hashtags dominantes são **2.519** `#economia` **e 185** `#econom`, e as datas vão só de 2017-05 a 2017-06. As contas mais frequentes são `noticiaminutobr`, `TSFRadio` (rádio portuguesa) e `portalR7`, e há tweets em espanhol e em português europeu no meio. Do outro lado, o `ironia.csv` tem ~12.700 tweets auto-marcados com `#ironia`, de ~9.300 autores, publicados entre 2014 e 2017. Só 78 tweets irônicos falam de economia, contra ~2.564 não irônicos (dados verificados). O `scripts/preparar_corpus_ironia.py` remove a hashtag `#ironia`, mas o vazamento continua no tópico, na fonte e na época. Nas notícias, `estadao.json` é sempre 0, e `sensacionalista.json` e `the_piaui_herald.json` são sempre 1. Ou seja, a tabela fonte × rótulo tem **células vazias**: nenhum tweet coloquial sincero e nenhuma notícia séria irônica.

O histórico público do IDPT já mostrava isso antes do Fraus. O treino oficial tem 15.212 tweets e 18.494 notícias, enquanto os testes são "300 tweets and 300 news gathered and annotated by the organizers" (García Subies, CEUR Vol-2943). No teste de tweets, anotado por humanos, **o melhor time chegou a 0,52 de acurácia balanceada** (Luz et al., BraSNAM 2023), e o BERTimbau afinado ficou em 0,49–0,50, abaixo do acaso (García Subies). Enquanto isso, o SiDi-NLP relatou **F1 1,000 na validação de tweets**, tanto com BERTimbau quanto com mBERT (CEUR Vol-2943). O F1 de 0,99 que o BERTimbau retreinado deu no teste interno do Fraus, junto com P(irônico)=1,0 em frases neutras de atendimento, repete esse padrão: um número alto dentro da distribuição que desaba quando o rótulo vem de um humano. O padrão é conhecido na literatura. Quando o rótulo de sarcasmo passa a ser dado pelo próprio autor, o melhor modelo cai de F1 0,874 para 0,364 (Oprea & Magdy, ACL 2020). Filtrar os vieses do SNLI derruba um modelo de 92% para 62% (Le Bras et al., 2020).

A esperança de que o Laya "evitaria dados viciados" esbarra em como ele aprende. O fine-tuning do script oficial é completo, com encoder e cabeça juntos, sem LoRA. A sequência sempre começa com a mesma instrução e os mesmos critérios A/B de `PERGUNTA_IRONIA` (`laya/common.py`, `build_sequence`). O que separa A de B é só o `state`, isto é, o texto. Se registro, língua ou gênero textual predizem o rótulo, o mmBERT aprende isso igualzinho ao BERTimbau. O próprio guia do Laya avisa: "The loop is only as good as the targets… treat their quality as the ceiling" (finetune.md). O único ponto a favor do Laya tinha sido o prior zero-shot. Sem treino, ele fez F1 0,554 no teste interno e 0,524 na régua, com 0% de falso positivo, melhor que o BERTimbau. O run de 02/10 mostrou que o fine-tuning completo apaga esse prior (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`).

O "100% de exatidão" é inalcançável por um motivo independente do corpus: ironia é ambígua até para humanos. No Tweets8JanSPI, anotado à mão, o Fleiss κ da ironia foi **0,38**, contra 0,59 do sentimento (BraSNAM 2026). O MultiPICo usa em média cinco anotadores por item justamente porque o julgamento varia (ACL 2024). Um modelo não pode concordar com um rótulo mais do que os humanos concordam entre si, a menos que esteja lendo outra coisa. A invariante 10 diz isso com outras palavras: "acurácia alta demais é sintoma, não vitória".

### 2.8.3 Treinar só ironia exige mexer em sete células e num rótulo de produção

O pacote `laya==0.3.21` só faz inferência e não tem API de treino. O fine-tuning roda pelo script de pesquisa `finetune_single_device.py`, no commit `d8a2e597` (script). O notebook 07 já o executa em subprocesso e confere o SHA-256. O script aceita tarefa única direto: basta o JSONL ter apenas linhas de `caso_ironia(texto, p)`, com `gold.ironia.probabilities = {"A": p, "B": 1-p}`, em que A = irônico. A ordem das chaves de `criteria` define a ordem do alvo. Por isso `PERGUNTA_IRONIA` tem de ser importada de `fraus/sinais/ironia_laya.py` e nunca redigitada, que é o mesmo cuidado que a invariante 8 pede para a ordem das classes.

As mudanças no notebook 07 (`notebooks/07_treino_laya.ipynb`) são estas, pela numeração "#" dos comentários das células:

| CÉLULA | HOJE | PARA SÓ IRONIA |
|---|---|---|
| #2 (imports) | importa emoção e ironia | pode ficar; os nomes de emoção deixam de ser usados |
| #4 (corpus de emoção) | GoEmotions-pt + XED-pt | **pular inteira**; define `dados_emocao`, `emocao_teste`, `frases_xed`, usados adiante |
| #5 (divisão de ironia) | `dividir_por_autor`, `conferir_divisao`, guarda de impressão contra `metricas_ironia.json` | **não mexer**: é o que garante que o BERTimbau do notebook 04 foi treinado na mesma divisão |
| #6 (JSONL) | laço de emoção com `repeticoes_por_classe` + ironia suavizada 0,05 | só ironia; `proibidos` = teste de ironia ∪ régua; manter o `assert` de vazamento |
| #7 (treino) | `MODO_FUMACA`, `EPOCAS=4` | fumaça primeiro (600 casos), depois completo |
| #8 (comparação) | confere `DIR_BERT_EMOCAO`, prevê emoção e XED | tirar a emoção; manter `prever_*`, `comparar`, `laya.load` |
| #9 (ironia) | teste interno, recortes `tweets`/ `noticias`, régua | sem mudança |
| #10 (poda + ONNX) | vocabulário com emoção, `cobertura_xed` | `perguntas = {**PERGUNTA_IRONIA}`; a cobertura precisa de outro texto independente |
| #11 (manifesto) | `perguntas: ["emocao","ironia"]` | `["ironia"]`; **ver a armadilha abaixo** |
| #12 (laudo) | conjuntos de emoção e ironia | só `ironia_interno`, `ironia_tweets`, `ironia_noticias`, `ironia_regua` |

A **armadilha do manifesto** é a parte que não aparece no notebook. Em `fraus/sinais/emocao_laya.py`, `laya_treinado_pelo_fraus(diretorio)` devolve `"emocao" in perguntas_do_artefato(diretorio)`, e um manifesto ausente equivale a `("ironia",)`. Assim, um artefato treinado só em ironia é **indistinguível do checkpoint base**, e a dashboard o rotularia como "Laya sem treino", o que é falso. Há um segundo efeito, mais silencioso: `obter_classificador_emocao_laya_onnx` reaproveita o mesmo agente e responderia emoção com uma cabeça afinada só em ironia. A correção precisa vir antes do treino e com TDD. O manifesto ganha um marcador explícito de procedência, por exemplo `"treinado_pelo_fraus": true` junto com a revisão e `rodada_de_fumaca: false`. `laya_treinado_pelo_fraus` passa a ler esse campo, e a leitura de emoção pelo Laya passa a recusar, com falha alta, um artefato que não declara `emocao`. Os dois testes falham primeiro com o manifesto só-ironia e passam depois.

Os hiperparâmetros estão quase todos fixos no código do script. A linha de comando aceita só `--data`, `--model-dir`, `--output-dir`, `--device`, `--epochs` e `--seed`. Mudar qualquer outra coisa obriga a editar o arquivo, o que quebra o SHA-256 conferido e precisa ser declarado como desvio do caminho oficial.

| PARÂMETRO | VALOR NO SCRIPT | RECOMENDAÇÃO PARA SÓ IRONIA |
|---|---|---|
| Micro-lote / acumulação | 8 / nenhuma (lote efetivo 8) | manter; a receita 2×T4 do autor usa lote efetivo 64, mas isso exige editar o script |
| LR encoder / cabeça | 2,5e-5 / 1e-4, AdamW wd 0,01 | manter |
| Agenda | cosseno até 1e-6, sem warmup; σ do ruído de 0,4 a 0,1 | manter |
| Precisão | fp16 autocast + gradient checkpointing, clip 1,0 | manter |
| `max_len` | 1024 (forçado) | controlar pelo `MAX_CARACTERES` do notebook: **280–400** em vez de 1200, para aproximar o tamanho de uma mensagem de chat e baratear o treino |
| Épocas | padrão 4 | **2 a 3**: o corpus tem um atalho forte, e mais épocas tendem a decorá-lo |
| Semente | padrão 0 | 42 e mais duas sementes se o tempo couber, porque o fine-tuning é instável entre sementes (Dodge et al. 2020; Mosbach et al. 2021) |
| Peso de classe | inexistente | dispensável: a divisão corrigida tem 59% de irônicas dos dois lados (≈1,4:1) |
| Suavização do alvo | 0,05 | considerar 0,1: rótulo por procedência não sustenta certeza de 0,975 |
| Calibração | temperatura por LBFGS em `min(400, n//10)` itens separados antes do treino | manter; ela não muda o argmax, mas muda qualquer limiar de 0,5 |

Sobre o tempo na T4: o run de 02/10 processou 142.204 itens (71.102 × 2 épocas) em 89 minutos, cerca de 26,6 itens/s com a carga incluída (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). Com os 28.365 itens de treino da divisão corrigida, a conta direta dá **~18 minutos por época, ~35 minutos para 2 épocas e ~70–75 minutos para 4**. Essa é uma extrapolação de um único run. As notícias são bem mais longas que as frases do GoEmotions que dominavam aquele treino, então o tempo real pode ser 1,5 a 2× maior. A estimativa que vale é a que a rodada de fumaça imprime. O script não salva checkpoint por época. Chamá-lo várias vezes com `--epochs 1` também não equivale a um treino de N épocas, porque reinicia o cosseno, a σ do ruído e o otimizador e separa de novo a fatia de calibração. Como 2 a 3 épocas cabem numa sessão T4, o caminho mais simples é o atual: copiar para o Drive no fim.

### 2.8.4 Consertar o negativo vale mais que trocar o encoder

Os métodos de debiasing que mexem no modelo, como product-of-experts, autoviés, group DRO e DANN, só redistribuem peso entre exemplos que **contradizem** o atalho (Clark et al. 2019; Sagawa et al. 2020). Com as células "coloquial × não irônico" e "notícia séria × irônico" vazias, não existe exemplo conflitante para repesar. Se a fonte é quase igual ao rótulo, apagar a fonte por treino adversarial apaga o rótulo junto. A correção tem de ser nos dados. O primeiro passo custa minutos: imprimir a tabela de contingência fonte × rótulo e, além dela, treinar um classificador "adivinhe a fonte". Se o rótulo sai da fonte, o corpus reprova antes de qualquer GPU, do mesmo jeito que a invariante 10 manda checar a sobreposição de distribuições por rótulo.

Não existe hoje um corpus PT-BR de ironia que seja grande, anotado por humanos e com negativos coloquiais naturais. O melhor candidato é o **MultiPICo-PT**: 1.994 pares post-resposta de Reddit e Twitter, metade brasileira e metade europeia, 49 anotadores nativos, ~38% irônicos, licença CC BY 4.0 e acesso livre no Hugging Face (MultiPICo, ACL 2024; HF). As duas classes saem da mesma amostragem e o rótulo vem do julgamento humano, não da fonte. O formato post-resposta combina com o Fraus, em que a fala do bot vem antes da do cliente. O Tweets8JanSPI (2.935 tweets, 224 irônicos, κ 0,38) tem o mesmo desenho, mas é distribuído desidratado, só com IDs, e trata de um único tópico (BraSNAM 2026). O Kaggle "viniciuscleves" é um clone do mesmo corpus do IDPT (Kaggle). O de manchetes The Onion × HuffPost reproduz o rótulo por procedência (Misra & Arora).

A receita recomendada tem quatro camadas, da mais à menos importante. A primeira é **descartar as notícias**, porque nenhum dos dois lados é fala de cliente, e **substituir o** `nao-ironico.csv` por negativos coloquiais estratificados por polaridade: muito elogio sincero e muita reclamação literal, por exemplo B2W com 5 e com 1 estrela (licença CC BY-NC-SA 4.0, compatível com uso acadêmico (README B2W)) e frases do GoEmotions-PT, que é Apache 2.0. Isso ataca o modo de falha já medido: a cabeça atual marca 74% das resenhas positivas da B2W como irônicas, contra 9% das negativas (`docs/treinamento.md`, 04/09/2026). A segunda é **somar o MultiPICo-PT**, como fatia de treino e de validação. A terceira são os **pares mínimos escritos à mão** no domínio de atendimento, no molde da subtarefa pareada do iSarcasmEval (Abu Farha et al. 2022) e dos negativos por micro-edição do Puntuguese (HF). Um exemplo de par: "ótimo serviço, resolveram rápido" × "ótimo serviço, só esperei 3 horas". Esses pares rendem mais como teste do que como treino, porque dados contrafactuais protegem as features perturbadas, mas podem piorar as que não foram perturbadas (Joshi & He 2022). A quarta é **minerar positivos de domínio** com heurística (elogio + situação negativa, "só que não", "parabéns" numa nota 1) e **confirmar cada um à mão**. Se a heurística virar o rótulo, o projeto volta ao rótulo por marcador. Os tweets irônicos do IDPT podem continuar como positivos, desde que tenham negativos do mesmo meio. Normalizar o estilo (minúsculas, sem URL, @, hashtag nem ponto final) é barato e remove pistas tipográficas, mas não remove o léxico de registro ("PIB", "Meirelles"). Por isso o efeito tem de ser medido com o baseline de estilo, antes e depois.

### 2.8.5 Três treinos e uma régua de 300 pares separam as causas

O colapso de 02/10 tem uma explicação demonstrada que não depende da multitarefa. A divisão saiu com treino 86% irônico e teste 43% irônico, porque as 18.373 notícias sem autor caíam todas no teste. O Laya treinado chegou a recall de **0,988 no irônico e 0,087 no não irônico**, com F1-macro de 0,388. A mesma divisão fez o BERTimbau ficar em 0,511, e a rodada de fumaça já mostrava 100% de falso positivo (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). Ainda assim, a hipótese de interferência entre tarefas tem base. No pre-finetuning multitarefa, o desempenho cai quando há poucas tarefas (Muppet), e a mistura de 02/10 era 83%

emoção para 17% ironia. Mas emoção e sentimento como tarefas auxiliares **ajudam** o sarcasmo em outros trabalhos (Chauhan et al. 2020). Só um desenho controlado decide.

| TREINO | O QUE VARIA | COMPARAÇÃO QUE RESPONDE |
|---|---|---|
| (i) Laya só ironia | tarefa única, divisão corrigida | (i) × (ii): a multitarefa atrapalha? |
| (ii) Laya emoção + ironia **balanceado** | amostragem por temperatura ou uma época de ironia por época de emoção | (i) × (ii): se Δ ≈ 0, a hipótese da multitarefa cai |
| (iii) BERTimbau só ironia (notebook 04) | encoder | (i) × (iii): o encoder importa, com dado e tarefa iguais? |

Os três treinos usam a mesma divisão, conferida pela impressão de `metricas_ironia.json`, e a mesma semente. Cada um deve rodar duas vezes: uma no IDPT corrigido e outra no corpus consertado da seção anterior. A troca de corpus é o fator que a literatura indica como dominante, e esse fator 2 × 3 é o que a banca vai querer ver. A divisão por autor não basta para medir o atalho. Hoje cada notícia sem autor vira um grupo próprio e se espalha pelos dois lados. Por isso é preciso uma função irmã de `dividir_por_autor` que agrupe por **fonte** (leave-one-source-out) e reaproveite `conferir_divisao`. Essa função vai falhar alto se a fonte for quase igual ao rótulo, e essa falha já é o diagnóstico.

O teste interno não decide nada, já que o BERTimbau marca 0,99 nele. Quem decide é uma **régua de domínio independente**, escrita por quem não viu as predições, congelada com `impressao_dos_textos` antes da primeira avaliação e marcada `independente=True` no laudo. A régua tem **200 a 400 itens em pares mínimos** e quatro estratos: elogio sincero, elogio irônico, reclamação literal (que separa ironia de sentimento negativo) e ironia sem marcador. Inclui ainda testes de invariância no estilo do CheckList (Ribeiro et al. 2020): o mesmo texto com e sem ponto final, em caixa alta ou baixa e com um emoji neutro tem de receber a mesma predição. A métrica principal é a **acurácia por par**, em que o modelo acerta os dois membros. O acaso aqui é 25%, e um modelo de estilo não consegue acertar os dois lados de um par com o mesmo registro. O tamanho sai de uma conta binomial própria: com acurácia em torno de 0,8, n=100 dá IC de ±7,8 pontos, n=200 dá ±5,5 e n=400 dá ±3,9. A régua atual tem 20 itens e IC de −0,28 a +0,17, larga demais para decidir qualquer coisa. Pares mínimos com troca de rótulo derrubam modelos em até 25% em relação ao teste original (Gardner et al. 2020), e é essa queda que o Fraus precisa conseguir medir.

Três baselines entram como modelos extras em `preditos_por_modelo`, no espírito do baseline "só a hipótese", que acerta ~67% no SNLI (Gururangan et al. 2018). O primeiro é **só estilo**: regressão logística sobre ~15 features sem léxico (comprimento, proporção de maiúsculas, ponto final, `!?…`, dígitos e R$, URL, @, hashtag, emoji, alongamento, "kkk"). O segundo é **só fonte**. O terceiro é **TF-IDF + LR**, que mostra a parte lexical do registro. Se o baseline de estilo também chega perto de 0,99 no teste interno, o teste mede procedência. O modelo só conta como detector de ironia se vencer o baseline de estilo **na régua**, com diferença demonstrada. `fraus/comparacao_modelos.py` já tem `mcnemar_exato`, `bootstrap_da_diferenca` (2.000 reamostras, F1-macro), `avaliar_conjunto` e `montar_laudo`, que já distinguem procedência igual de independente. As quatro peças que faltavam já existem no mesmo módulo: IC de um único modelo, **bootstrap por par** (reamostrar pares, e não itens, para preservar a dependência dentro do par), acurácia por par e **ECE** com diagrama de confiabilidade (Guo et al. 2017). O ECE importa porque o Laya sai superconfiante e a temperatura dele é ajustada numa fatia do próprio corpus viciado. Para o McNemar, o que conta são os pares discordantes: com ~20 discordantes, só uma divisão de 15:5 ou mais

dá p<0,05. Como guarda de regressão, entra a medição que já existe em `docs/treinamento.md`: P(irônico) médio por estrela da B2W. Se a diferença entre resenhas positivas e negativas continuar grande, o atalho continua lá.

### 2.8.6 O ganho esperado é de poucos pontos, e vem do dado

Contra o multitarefa de 02/10, o Laya só-ironia vai melhorar muito no teste interno: de 0,388 para algo bem acima, talvez perto do 0,99 do BERTimbau. Esse salto vem da divisão corrigida e da proporção de 59/59, que eliminam o colapso "tudo é irônico". Ele não deve ser atribuído ao treino em tarefa única nem apresentado como detecção de ironia. Na comparação justa, o treino (i) contra o (ii) balanceado, o tamanho de efeito publicado para multitarefa é de **poucos pontos, em qualquer direção** (Standley et al. 2020; Chauhan et al. 2020). O que se espera é empate dentro do IC.

Contra o BERTimbau, com tarefa e dado iguais, a literatura não dá motivo para esperar diferença grande. Em PT-BR, o BERTimbau vence o mBERT por margens de 2 a 4 pontos, por exemplo 89,2 contra 86,8 de F1 no ASSIN2 RTE (neuralmind-ai). O mmBERT-base supera o XLM-R base no XTREME (72,8 contra 70,4), mas português é só ~2,4% da mistura de pré-treino, e o artigo não traz nenhum benchmark de classificação em português (Marone et al. 2025). Há um precedente em que o mmBERT-base venceu o BERTimbau em NER clínico em português, com micro-F1 de 0,76, num preprint ainda sem revisão (arXiv 2603.26510). Não existe resultado de mmBERT, XLM-R ou Laya no IDPT, nem de afinação do Laya em ironia em qualquer língua. Na CPU, o Laya é mais lento: 459 ms contra 273 ms por mensagem na ironia, com ressalva, porque o Laya rodou em ONNX FP32 e o BERTimbau em PyTorch (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). A velocidade do ModernBERT só aparece em GPU e em sequências longas.

A única vantagem estrutural do Laya para ironia é o vocabulário multilíngue, que permite misturar o MultiPICo das outras oito línguas. A literatura, porém, é desfavorável à transferência de ironia entre línguas: o viés de tópico dos corpora atrapalha (Ortega-Bueno et al. 2023), e detectores de sarcasmo não generalizam entre datasets (Jang & Frassinelli 2024). O ganho que de fato muda o produto aparece na régua de domínio, quando se troca o corpus. Um modelo treinado no IDPT deve ficar perto do acaso por par, como o 0,52 oficial nos tweets. Um modelo treinado com negativos coloquiais e pares mínimos tem espaço para ganhar dezenas de pontos ali, na escala das quedas de 25 a 30 pontos que a literatura de contrast sets e filtragem adversarial atribui ao viés. Essa magnitude é inferência por analogia e não foi medida em português. A régua de 300 pares existe justamente para medi-la.

## 2.8.7 Régua de pares mínimos

Estado: **rascunho pronto, anotação em andamento.** Spec em `docs/superpowers/specs/2026-10-08-regua-pares-ironia-design.md` (interno).

- 150 pares (elogio, reclamação, neutra), registro equilibrado conferido por `conferir_registro_equilibrado` e, par a par, mesma vírgula e mesmo número de "?" nos dois lados. O baseline só de estilo acerta 0,18 dos pares fora da amostra no rascunho, abaixo do acaso (0,25). O rascunho é autoral; o rótulo que vale é o da anotação às cegas.
- A anotação mora no Fraus, por link e sem conta: `scripts/criar_link_anotacao.py --quantos N` chama `POST /anotacao/anotadores` (só dev) e imprime `<id>  https://fraus-one.vercel.app/anotar/<token>`. O banco guarda só o hash do token e um id opaco (`a_` + 8 hex), nunca nome. A página recebe de `GET /anotacao/<token>` só `id` e `texto`, na ordem daquele anotador, e grava por `POST /anotacao/<token>/respostas`; o instante é do servidor. A primeira rodada (João, na página antiga do claude.ai) está versionada em `fraus/dados/anotacao_regua/anotador-claude-ai.json`, anotador `claude-ai-1`.
- Para um grupo de mensagens, `scripts/criar_link_anotacao.py --grupo 5` cria **um** link (`https://fraus-one.vercel.app/anotar/grupo/<token>`) com 5 vagas. Cada pessoa que o abre ganha o próprio anotador (a vaga é ocupada de forma serializada no banco, então 5 não viram 6) e a página guarda o link pessoal só naquele navegador. Em outro aparelho, abrir o link do grupo de novo gasta outra vaga e começa do zero; quem quiser trocar de aparelho guarda o endereço da página pessoal. Repassar o link do grupo também gasta vaga. Grupo cheio responde "Este link já foi usado por todas as pessoas previstas". Revogar o grupo (`POST /anotacao/grupos/<g_id>/revogar`) fecha a entrada; com `?anotadores=1` revoga também quem entrou, e as respostas saem do export.
- O token viaja na URL e fica nos logs de requisição da Vercel e no histórico do navegador, como o link de convite. Link vazado se revoga com `scripts/criar_link_anotacao.py --revogar <id>`; as respostas desse anotador saem do export.
- **Critério de promoção de qualquer cabeça de ironia:** vencer o baseline só de estilo (`fraus/baseline_estilo.py`) em acurácia por par na régua, com IC 95% da diferença excluindo zero. O teste interno do IDPT não entra. No laudo isso é o campo `candidato_vence_por_par` (limite inferior do IC da diferença acima de zero). `por_par_demonstrada` só diz que o IC exclui zero e também acende quando o candidato é significativamente pior; para promover, vale `candidato_vence_por_par`.
- Depois da anotação: `uv run python scripts/exportar_anotacao.py respostas-fraus.json` baixa `GET /anotacao/respostas` (só dev) e `uv run python scripts/consolidar_regua_ironia.py fraus/dados/anotacao_regua/anotador-claude-ai.json respostas-fraus.json` concatena os arquivos e consolida. Menos de 100 pares confirmados para o script com erro dizendo quantos faltam.
- Cada registro é `{"anotador": "...", "frase_id": "...", "resposta": "ironico|nao_ironico|contexto", "instante": "2026-10-08T12:00:00.000000+00:00"}`; o script usa a última resposta de cada anotador por frase.
- A página embaralha a ordem por anotador (semente derivada do id) e nunca põe os dois lados de um par em sequência. A régua congelada guarda duas impressões no meta: uma dos textos e outra de `par_id|rótulo|texto`, de modo que trocar um rótulo ou mover um par também falha alto na carga.

## 2.8.8 Conclusão

A pergunta "Laya ou BERTimbau, só ironia ou multitarefa" foi formulada sobre um corpus que não sustenta nenhuma das quatro respostas, porque nele o rótulo é a fonte. A decisão que de fato move o resultado vem antes do encoder. São três passos: trocar o `nao-ironico.csv` de #economia e as notícias por fala coloquial sincera, construir a régua de pares mínimos e comparar cada modelo com um baseline de estilo que não sabe nada de

ironia. O Laya só-ironia continua valendo como experimento, como a célula (i) de um desenho controlado e não como conserto, e só depois de corrigir `laya_treinado_pelo_fraus` para que o produto não rotule um modelo treinado como "sem treino".

Para a banca, a meta de 100% se transforma num argumento a favor do projeto. Mostrar que o BERTimbau marca 0,99 no teste interno, que um classificador de pontuação e caixa alta chega perto disso, e que ambos caem para perto do acaso numa régua de pares mínimos é uma demonstração empírica da invariante 10 em ironia. Essa demonstração convence mais do que qualquer número alto. Promover o modelo depende de ele vencer o baseline de estilo na régua independente, com IC que exclua zero. O teste interno não serve para essa decisão.

# 2.9 DIFERENCIAIS

## 2.9.1 Diferenciais — relatório de entrega (14/10/2026)

Este relatório fecha a missão "diferenciais do Fraus, ancorados em concorrentes reais e medidos". Para cada entrega, diz:

- o que entrou;
- o que foi medido;
- o que **não foi verificado**, e por quê;
- como reproduzir.

Nenhum número aqui foi estimado à mão. Onde não houve medição, está escrito "não verificado" e o script já está pronto para rodar.

### RESUMO

| ENTREGA | ESTADO | MEDIÇÃO REAL |
|---|---|---|
| E1 — por que esta nota, contrafactual e drivers | entregue | aditividade exata provada no fusor sintético; **no fusor real: não verificado** |
| E2 — calibração, abstenção e conjunto conformal | entregue | **não verificado** (artefato conformal não gerado) |
| E3 — declarado × inferido | **não iniciado**: exige schema novo (`nota_declarada`), ainda não aprovado | — |
| E4 — benchmark contra LLM | **descartado** em 14/10/2026 | — |
| E5 — sinalizadores por regra | entregue | casos construídos e taxa de disparo no simulador; **precisão e recall em dado real: não verificado** |
| E6 — impacto por tema e início da deterioração | entregue | **não verificado** no fusor real; o lead time do simulador é artefato do gerador |

**Por que tanta coisa "não verificado".** A máquina de desenvolvimento não tem o `fusor.joblib` treinado em `log1p`. O artefato que estava disponível foi treinado em segundos crus, e `Fusor.carregar` o recusa de propósito (invariante 10). Sem esse fusor:

- nenhuma métrica que dependa da nota real pôde ser medida;
- cada instrumento falha alto, sem motor dublê (invariante 7).

Assim que o fusor estiver em `modelos/`, os comandos da seção Reproduzir produzem os números.

## O QUE VALE PARA TODAS AS ENTREGAS

- **Sem LLM em runtime.** Toda explicação é feita dos coeficientes ou por template, e nenhuma chamada de rede entra no caminho de predição.
- **Nada novo é gravado.** O `score`, a nota e a categoria gravados não mudam. Nenhuma coluna, tabela ou chave de payload foi criada. Tudo é derivado na leitura.
- **Nenhuma régua nova.** Nada retreina o modelo, e ficaram intocados:
- `modelos/`;
- `NOMES_FEATURES` (39);
- `PESO_NEUTRO_NO_SCORE` (0,75);
- `FEATURES_EM_LOG`;
- a cabeça de ironia.
- **Ausência não vira zero.** Conversa sem sinal, n abaixo do mínimo ou campo ausente numa API antiga aparecem nomeados, nunca como 0.
- **Conformidade.**
- Nada pontua atendente.
- Tudo roda em lote, sobre conversa encerrada; nada alerta ao vivo.
- Nada decide sozinho: as marcas (abstenção, sinalizadores, deterioração) informam e não mudam a nota. O atendimento continua no NPS.
- **O sinal de tempo do fusor é sintético**, e toda métrica nova declara isso.

## E1 — "POR QUE ESTA NOTA", CONTRAFACTUAL DE ESPERA E DRIVERS

**O que entrou.**

- **Decomposição exata do fusor** (`Fusor.decompor`). O intercepto mais as contribuições é igual ao log da razão contra a classe de referência, e o score é refeito a partir dos logits.
- **Efeito por família em pontos de score.** É um recálculo exato, mas **não é aditivo**, e a tela diz isso.
- **Frases por template** em `fraus/explicacao.py`, sem LLM. A explicação vem dentro de `GET /conversas/{id}/atribuicao`.
- **`POST /conversas/{id}/contrafactual`**, que testa a primeira resposta ou um teto de espera. Ele recusa com 422 quando a conversa sai da faixa de treino (`|z| > 4`).
- **`GET /indicadores/drivers`**, sobre uma amostra por semente (50, com teto de 200). Mostra a média por família em detratores e em promotores, com IC por bootstrap, e o rótulo "associação no modelo, não causa".
- **Duas telas:** um bloco na transcrição e um painel sob demanda na Visão geral.

**Medido.** No fusor sintético dos testes, a aditividade fecha com erro menor que 1e-9.

**Não verificado.** A aditividade e o tempo no fusor real. O script `scripts/medir_aditividade.py` está pronto.

## E2 — CALIBRAÇÃO, ABSTENÇÃO E CONJUNTO CONFORMAL

**O que entrou.**

- **Conjunto conformal Mondrian (por classe) sobre o score gravado** (`fraus/conformal.py`). Para cada nota, diz quais classes de satisfação são compatíveis com ela, com cobertura-alvo de 90% por classe. O artefato `fraus/dados/conformal_fusor.json` é chaveado pela régua.
- **Falha explícita sem artefato.** Se o artefato estiver ausente, inválido ou for de outra régua, as rotas respondem 503 ou 409 com o motivo. Nunca devolvem um conjunto vazio fingindo confiança.
- **Abstenção** ("leitura sem confiança suficiente"):
- dispara quando o conjunto não tem exatamente uma classe ou, no detalhe, quando há deriva;
- informa e não corrige.
- **Rotas:**
- `GET /conversas/{id}/confianca`;
- `GET /indicadores/confianca`, que dá a fração de abstenção com IC de Wilson.
- **Instrumentos** que não gravam nada:
- `scripts/medir_calibracao.py` (ECE, MCE e por classe, com bootstrap);
- `scripts/medir_abstencao.py`;
- `scripts/calibrar_conformal.py`.
- **Selo de confiança no atendimento.** Ele fala em "classe de satisfação", não em categoria NPS.

**Não verificado.** A cobertura empírica, o ECE e a curva de abstenção. O artefato conformal não foi gerado, então as rotas de confiança respondem 503 até alguém rodar `scripts/calibrar_conformal.py --gravar`.

## E5 — SINALIZADORES POR REGRA

**O que entrou.**

- **Cinco regras derivadas na leitura** (`fraus/sinalizadores.py`):
- `loop_do_bot`;
- `cliente_repete`;
- `pedido_de_humano_ignorado`;
- `escalada_tardia`;
- `abandono`.
- **Nenhuma regra lê o texto ou o relógio da fala do atendente.** Um teste troca os dois e prova que o resultado não muda. A fala humana também nunca entra na evidência.
- **Conversa sem fala do cliente** dá `None` ("não avaliado"), nunca `[]`.
- **Filtro** `?sinalizador=` em `GET /conversas`:
- pagina depois de filtrar;
- respeita o escopo por canal;
- responde 400 a regra desconhecida.
- **Lista de achados e marca** ⚑ na transcrição.
- **Instrumento:** `scripts/medir_sinalizadores.py`.

**Medido.**

- Os 15 casos construídos se comportam como declarado. Isso **verifica a implementação, não mede qualidade**.
- No simulador (60 conversas, sem rótulo de regra, só a taxa de disparo): `abandono` 15, `cliente_repete` 2, as outras 0.
- `loop_do_bot` deixou de ser cúbico: 1.000 respostas do bot levavam 27,98 s e agora levam 0,083 s.

**Não verificado.** Precisão e recall em dado real. Falta a amostra anotada à mão, e a fonte dela é decisão do João. O script aceita `--anotada arquivo.jsonl`. O falso positivo de despedida do bot e o léxico fechado estão declarados em Limitações.

## E6 — IMPACTO POR TEMA E INÍCIO DA DETERIORAÇÃO

**O que entrou.**

- **Impacto por tema** (`fraus/impacto.py`), em cada tema de `GET /operacao/radar`:
- é o NPS do tema menos o NPS do resto do recorte;
- o IC 95% sai de bootstrap multinomial (1.000 reamostras, semente fixa) e tem o rótulo "associação, não causa";
- abaixo de 30 conversas com sinal em qualquer um dos lados, o resultado é nulo, com o motivo e as duas contagens.
- **Início da deterioração** (`fraus/deterioracao.py`), no campo `deterioracao` de `GET /operacao/replay/{id}`:
- é a mensagem a partir da qual a estimativa por prefixo fica detratora e não volta;
- traz o lead time em mensagens e em segundos;
- é retrospectivo e nunca alerta.
- **Teste de não vazamento.** Alterar a mensagem `j` não muda nenhum prefixo `k < j`, e o teste não passa no vazio.
- **Instrumento:** `scripts/medir_deterioracao.py`.

**Medido.**

- No teto do radar, o custo do bootstrap independe de n: com 15.000 conversas por lado, caiu de 1,58 s para 0,069 s.
- Conferência estrutural do simulador: ele sorteia todas as falas do cliente de um único rótulo, então **não gera trajetória**. O lead time medido nele é artefato do gerador, e o script diz isso antes de imprimir qualquer número.
- Por construção, o início da deterioração **não aumenta o recall** sobre o score final. O que ele acrescenta é o lead time.

**Não verificado.** Recall, falso alarme e lead time com o fusor real: o script sai com código 2. A conferência visual mostrou que a regra não tem margem em torno do limiar (na demonstração, a deterioração começou com uma queda de cerca de 67 para 61 pontos). Isso está declarado.

## CONTRATO DA API

O OpenAPI foi de **76** caminhos (referência da Fase 0) para **80**. Os quatro novos:

| CAMINHO | ENTREGA |
|---|---|
| `/conversas/{conversa_id}/contrafactual` | E1 |
| `/indicadores/drivers` | E1 |
| `/conversas/{conversa_id}/confianca` | E2 |
| `/indicadores/confianca` | E2 |

E5 e E6 acrescentaram só campos e parâmetros a rotas que já existiam:

- `sinalizadores` e `?sinalizador=`;
- `impacto` no radar;
- `deterioracao` no replay.

Nenhum caminho sumiu.

## VERIFICAÇÃO FINAL

| COMANDO | RESULTADO |
|---|---|
| `uv run pytest -q` | 1585 passed, 5 skipped (baseline: 1395 passed, 7 skipped) |
| `cd dashboard && npm test` | 389 passed (baseline: 351) |
| `npx tsc --noEmit` | limpo |
| `uv run mkdocs build --strict` | limpo |
| `modelos/`, `modelos-onnx/`, `pyproject.toml`, `uv.lock`, `dashboard/` `package.json` | sem diff desde `cdaa9e2` |
| `NOMES_FEATURES`, `PESO_NEUTRO_NO_SCORE`, `FEATURES_EM_LOG` | valores intactos (o E1 só moveu o cálculo do score para `_score_das_probabilidades`) |

## REPRODUZIR

Os instrumentos de medição do fusor pressupõem o artefato `log1p` compatível. Para a extração atual, `B2W` identifica o CSV, `EXPERIMENTO` um diretório externo ao repositório/checkpoints e `CABECAS` a raiz dos candidatos. Reserve o manifesto antes do notebook 01 (`FRAUS_MANIFESTO_FUSOR`), não depois do treino da cabeça. Cache posicional antigo não é reutilizável por nome; ver Generalização.

```
uv run python scripts/medir_aditividade.py
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" --gerar-manifesto
uv run python scripts/retreinar_fusor_local.py \
  --b2w "$B2W" --manifesto "$EXPERIMENTO/manifesto.json" \
  --modelos "$CABECAS" --backend torch \
  --cache "$EXPERIMENTO/features.json" --saida "$EXPERIMENTO/candidato"
uv run python scripts/calibrar_conformal.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" --cache "$EXPERIMENTO/features.json"
uv run --with matplotlib python scripts/medir_calibracao.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" \
  --cache "$EXPERIMENTO/features.json" --png "$EXPERIMENTO/calibracao.png"
uv run python scripts/medir_abstencao.py \
  --fusor "$EXPERIMENTO/candidato/fusor.joblib" --cache "$EXPERIMENTO/features.json"
uv run python scripts/medir_sinalizadores.py --simulador 600 [--anotada amostra.jsonl]
uv run python scripts/medir_deterioracao.py --n 300
```

## PENDÊNCIAS QUE DEPENDEM DO JOÃO

1. **Fusor `log1p` nesta máquina.** Destrava todas as medições marcadas "não verificado".
2. **Schema `nota_declarada` (E3).** Precisa de aprovação, por causa da paridade SQLite × Postgres.
3. **Amostra anotada dos sinalizadores (E5).** Falta definir a fonte, quem anota e com qual material.
4. **Concorrentes.** As afirmações estão em concorrentes, com URL e data. Elas não substituem uma leitura própria antes da banca.

## 2.9.2 Diferenciais — o que os concorrentes fazem (fontes de 14/10/2026)

Esta página sustenta toda afirmação sobre concorrentes que entrar em documento ou slide. Cada linha tem URL e data de acesso, e a confiança é uma de três:

- **confirmado:** citação na página oficial;
- **parcial:** veio de trecho de busca ou de resumo da página, sem a frase literal;
- **não confirmado.**

O que não está aqui **não pode ser afirmado** na banca.

**Como foi coletado.** Busca web e depois a leitura da página, em 14/10/2026. A leitura passa por um resumo automático, então uma citação marcada "parcial" precisa ser relida na fonte antes de virar slide. Algumas páginas recusaram o acesso (403); elas estão marcadas.

### LEITURA HONESTA DO RESULTADO

**O produto mais próximo do Fraus é o Intercom CX Score.** Ele também:

- infere a satisfação sem pesquisa;
- explica a nota por motivos (*reasons*);
- deixa sem nota a conversa sem informação suficiente;
- pontua só depois de a conversa encerrar;
- declara que não serve para avaliar o desempenho individual de quem atende.

**Isso tira do Fraus o direito de chamar de exclusivos os pontos acima.** O que sobra de diferente, com base no que foi verificado:

1. **Sem LLM.** O Intercom declara que combina tecnologia própria com "*third-party large language models*". O Fraus usa modelos pequenos e locais e roda sem rede no caminho de predição.
2. **Explicação que é o modelo.** No Fraus, as contribuições somam exatamente o logit do fusor (E1), e a frase sai por template sobre essa soma. Nas fontes lidas, os *reasons* do Intercom são categorias atribuídas à conversa; não encontramos descrição de decomposição numérica.
3. **Incerteza com garantia estatística declarada.** O Fraus tem um conjunto conformal por classe, com cobertura-alvo e abstenção quando o conjunto não tem exatamente uma classe (E2). A abstenção do Intercom, nas fontes lidas, é por elegibilidade: falta de informação na conversa. **Não encontramos, em nenhum concorrente, incerteza por previsão.**
4. **Método aberto e reproduzível.** O código, as sementes, as limitações e o que não foi verificado estão publicados.

**Diferença de política.** Zendesk QA, Observe.AI e Medallia **pontuam agentes** (QA por rubrica, coaching). O Fraus não pontua atendente, por desenho. O Intercom também declara não servir para isso.

### INTERCOM — FIN CX SCORE

Fonte principal: https://www.intercom.com/help/en/articles/10495092-understand-customer-experience-at-scale-with-the-cx-score (acesso em 14/10/2026).

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| Infere sem pesquisa | "scoring every meaningful interaction—without relying on surveys" | confirmado |
| Usa LLM de terceiros | "combination of Intercom's proprietary technology and third-party large language models" | confirmado |
| Explica por *reasons* (qualidade da resposta, esforço do cliente, emoção forte, feedback de produto e de política) | rótulo "CX Score reasons"; a lista veio do resumo da página | confirmado (*reasons*); parcial (a lista) |
| Escala de 1 a 5 | trecho de busca | parcial |
| Deixa sem nota a conversa sem informação suficiente | "enough conclusive information in the conversation to determine a rating" | confirmado (por elegibilidade) |
| Intervalo de confiança ou probabilidade por previsão | — | não confirmado |
| Não serve para avaliar atendente individual | "not intended to help assess individual teammate performance" | confirmado |
| Só depois do encerramento | o resumo cita "The conversation has been closed" como requisito | parcial |

O blog https://www.intercom.com/blog/the-new-cx-score-explained/ apareceu na busca e **não foi lido**.

## ZENDESK — ZENDESK QA (EX-KLAUS) / AUTOQA

Fontes, todas acessadas em 14/10/2026:

- https://www.zendesk.com.br/blog/ai/workflow-automation/auto-qa/
- https://support.zendesk.com/hc/en-us/articles/9046364914842-Improving-AutoQA-accuracy

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| Avalia 100% das conversas contra rubricas, incluindo o agente | "calls, chats and e-mails"; pontuação de agente por rubrica (resumo) | confirmado |
| Detecta sentimento, riscos, bots em laço e sinais de churn | "changes in customer sentiment, compliance gaps, looping bots, unresolved issues, churn signals" | confirmado |
| **Pontua o desempenho do agente** | "as a confidence indicator when evaluating agent performance" | confirmado |
| Mede acurácia contra revisão humana, não a incerteza por previsão | "reflects agreement between AI and human ratings" (resumo) | parcial |
| Usa IA generativa | resumo | parcial |
| Tempo real | resumo | parcial |
| Prevê CSAT de forma nativa | só aparece num app de marketplace que não foi lido (403) | não confirmado |

Nota: o Zendesk também detecta "*looping bots*", assim como o `loop_do_bot` do E5. A diferença está em pontuar o agente, não na regra.

## OBSERVE.AI

Fonte: https://www.observe.ai/platform/interaction-intelligence (acesso em 14/10/2026). A página de CX deu 404.

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| Pontua todas as interações contra rubrica | "Score 100% of interactions - automatically" | confirmado |
| Explica por trecho da transcrição | "exact transcript moments behind every score" | confirmado (por evidência; não por *drivers*) |
| Pontua agentes (QA, coaching) | trechos de busca do blog | parcial |
| Sentimento como nota atribuída por IA | trecho de busca do blog | parcial |
| CSAT previsto | só em guia de terceiros | não confirmado |
| Confiança ou abstenção | — | não confirmado |

### MEDALLIA — CONVERSATIONAL INTELLIGENCE

Fonte: https://www.medallia.com/suite/contact-center-experience/conversational-intelligence/ (acesso em 14/10/2026).

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| Emoção e sentimento por IA | "AI-powered emotion analysis highlights sentiment, pains, and emerging CX trends" | confirmado |
| Pontuação automática de toda interação, ligada ao agente | "Automated Scoring For Every Interaction" | confirmado |
| Coaching logo após a interação | resumo | parcial |
| NPS ou CSAT previsto sem pesquisa | a página trata a conversa como complemento da pesquisa | não confirmado |
| Explicação, confiança ou abstenção por previsão | — | não confirmado |

### ZENVIA (INCORPOROU A MOVIDESK)

Fontes (acesso em 14/10/2026):

- https://zenvia.com/en/customer-cloud/serve-better/ (lida);
- o artigo da base de conhecimento sobre análise de sentimento devolveu só o título.

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| Análise de sentimento com IA em tickets e chats | "identify sentiments, and prioritize tickets" | confirmado |
| "Índice de Satisfação" inferido da conversa, sem pesquisa | "without depending on formal surveys" (trecho de busca) | parcial |
| IA generativa para categorizar e medir sentimento | trecho de busca | parcial |
| Aquisição da Movidesk em 2022 | trecho de busca | parcial |
| Sentimento por agente; tempo real por conversa | o resumo foi especulativo e foi descartado | não confirmado |
| Explicação, confiança ou abstenção | — | não confirmado |

## BLIP (TAKE BLIP)

As páginas de ajuda deram 403. O conteúdo vem de trechos de busca, exceto a extensão Feeling Monitor.

| AFIRMAÇÃO | EVIDÊNCIA | CONFIANÇA |
|---|---|---|
| "Conversation Quality": nota de 0 a 100 a partir de fluência, integridade, resolução e relevância | https://help.blip.ai/hc/en-us/articles/36747923309719-Conversation-quality: "predominant sentiment expressed by the user throughout the conversation" | parcial |
| Monitor de sentimento em tempo real | https://store.blip.ai/en/extensions/x5epboo: "Track your customers' sentiment in real time". **É extensão paga de terceiro**, não nativa | confirmado |
| Desk Score ("qualidade com IA"): o que avalia | https://help.blip.ai/hc/en-us/articles/37508015695127 (403) | não confirmado |
| Se a nota usa LLM; explicação, confiança ou abstenção | — | não confirmado |

## O QUE NÃO PODE SER AFIRMADO

- Que o Fraus é o único a inferir satisfação sem pesquisa. **O Intercom faz isso.**
- Que o Fraus é o único a explicar a nota, ou a não avaliar atendente. **O Intercom também.**
- Que o Zendesk QA, a Observe.AI ou a Medallia preveem CSAT ou NPS de forma nativa: não confirmado em fonte oficial.
- Qualquer coisa sobre a aquisição da Medallia: não pesquisado.
- Qualquer coisa sobre o Desk Score da Blip, ou se a Blip usa LLM: não confirmado.
- Qualquer comparação numérica com LLM: o E4 foi descartado e não há número.

## 2.9.3 Diferenciais — roteiro da demonstração

Roteiro para apresentar os diferenciais na banca, em cerca de 10 minutos. Cada passo diz:

- o que mostrar;
- o que falar;
- o que **não** afirmar.

A comparação com produtos do mercado está em concorrentes, e cada afirmação ali tem fonte e data.

### ANTES DE COMEÇAR

- **Com os modelos reais** (o fusor `log1p` em `modelos/`): suba a API real e a dashboard. É o único jeito de mostrar a explicação, o contrafactual, os drivers e o replay com números do modelo.
- **Sem os modelos:** `uv run python scripts/api_demo.py` sobe um snapshot sintético. Nesse caso, diga em voz alta que os números são sintéticos.
- **O selo de confiança (E2)** só aparece depois de `scripts/calibrar_conformal.py --gravar`. Sem o artefato, a tela mostra o motivo (503). **Mostrar isso também é honesto**: o sistema se recusa a fingir confiança.
- Escolha antes três atendimentos:
- um detrator com espera longa (para o contrafactual);
- um com abstenção ou sinalizador;
- um com início de deterioração no Replay.

### 1. A TESE (1 MIN)

**Falar:**

> O Fraus estima a satisfação a partir da conversa, sem perguntar ao cliente e sem LLM em produção. São modelos pequenos, locais e determinísticos. Hoje vou mostrar quatro coisas que ele faz de forma diferente:
>
> - explica a nota a partir dos próprios coeficientes;
> - diz quando não tem confiança;
> - marca riscos sem julgar o atendente;
> - mostra o que puxa o NPS, sem confundir isso com causa.

**Não afirmar:** que o NPS foi perguntado ao cliente. Ele é **inferido**, e toda tela tem a etiqueta "estimativa".

## 2. UM ATENDIMENTO, DE PONTA A PONTA (4 MIN)

Abra o atendimento 1 em Atendimentos.

1. **A nota, a categoria e a etiqueta "estimativa".** A nota sai do servidor; a tela só exibe.
2. **"Por que esta nota" (E1).** A frase é montada por template sobre a decomposição exata do fusor.
3. **Falar:** "A soma das contribuições reproduz exatamente o logit do modelo. Não é uma aproximação, e não é um texto gerado sobre o modelo: é o modelo."
4. **Não afirmar:** que o efeito por família soma a nota. Ele é um recálculo exato, mas não é aditivo, e a tela diz isso.
5. **Contrafactual de espera (E1).** Simule "primeira resposta em 60 s".
6. **Falar:** "Se a primeira resposta tivesse vindo em um minuto, a estimativa seria esta. Nada é gravado."
7. Mostre a recusa (422) com uma hipótese fora da faixa de treino: o sistema não extrapola.
8. **Selo de confiança (E2).** Mostre a classe de satisfação compatível com a nota, com calibração de 90%. Depois mostre um caso de abstenção.
9. **Falar:** "Quando a nota é compatível com mais de uma classe, ele diz 'leitura sem confiança suficiente'. A conversa continua no NPS: a abstenção marca e informa, não corrige."
10. **Não afirmar:** que a garantia vale em produção. Ela vale sob permutabilidade com o holdout, e o tempo desse holdout é sintético.
11. **Sinalizadores (E5).** Mostre a lista acima da transcrição e a marca ⚑ nas mensagens.
12. **Falar:** "São regras explicáveis sobre a fala do cliente, a do bot, contagens e posição. Nenhuma lê o que o atendente humano escreveu: há um teste que prova isso."

## 3. O AGREGADO (2 MIN)

1. **Drivers (E1), na Visão geral, sob demanda.** Mostre o que mais pesa em detratores e em promotores, com IC.
2. **Falar:** "É uma amostra sorteada por semente, e o tamanho dela é declarado. É associação no modelo, não causa."
3. **Impacto por tema (E6), na aba Radar da Operação.** Mostre o NPS inferido do tema contra o resto do recorte, com IC.
4. **Falar:** "É associação, não causa, e o IC só cobre a incerteza amostral. Com menos de 30 conversas com sinal em qualquer lado, ele não dá número nenhum e diz quanto falta."

## 4. RETROSPECTIVA DE UMA CONVERSA (1 MIN)

Abra o atendimento 3 na aba Replay da Operação.

- **Falar:** "Cada ponto do gráfico só vê as mensagens que já tinham acontecido; há um teste de não vazamento para isso. A linha tracejada marca o início da deterioração, ou seja, de onde a estimativa ficou detratora e não voltou. É retrospectivo: o Fraus não alerta durante o atendimento."
- **Não afirmar:**
- que o lead time medido prevê o de produção, porque o simulador não gera trajetória;
- que essa marca detecta mais detratores que o score final, porque, por construção, não detecta.

## 5. O QUE O FRAUS NÃO FAZ (SLIDE, 1 MIN)

| NÃO FAZ | POR QUÊ |
|---|---|
| Não pontua atendente humano | É política de conformidade, não feature. Nenhuma regra lê a fala do atendente. |
| Não roda durante o atendimento | É lote, sobre conversa encerrada. Não há alerta ao vivo. |
| Não decide nada sozinho | Abstenção, sinalizadores e deterioração marcam e informam. A nota não muda. |
| Não usa LLM em produção | São modelos locais e determinísticos. A explicação vem dos coeficientes. |
| Não afirma causa | Drivers e impacto por tema são associação. |
| Não troca ausência por zero | Sem sinal, ou com amostra pequena, a tela nomeia o que falta. |
| Não chama de NPS declarado | É NPS **inferido**, sempre com a etiqueta de estimativa. |

## 6. PERGUNTAS PROVÁVEIS DA BANCA

**"Por que não usar um LLM?"** Custo, latência, determinismo e execução local. A explicação do Fraus é exata porque o modelo final é linear. A comparação numérica com LLM foi **descartada** do escopo (E4); não há número para citar.

**"Esses números foram medidos no modelo real?"** Responda pelo relatório. As medições no fusor real estão marcadas como "não verificado", e os scripts estão prontos. Não arredonde isso para "sim".

**"O sinal de tempo é real?"** Não. O fusor treinou o tempo em dados **sintéticos**, calibrados pela literatura, porque nenhum corpus público em português tem timestamps de diálogo. Isso está declarado em todas as métricas.

**"Como vocês sabem que os sinalizadores acertam?"** Os casos construídos verificam a implementação. Precisão e recall em dado real dependem de uma amostra anotada que ainda não existe.

**"Qual a diferença para os produtos do mercado?"** Comece pelo mais próximo: o Intercom CX Score também infere sem pesquisa, explica por motivos, deixa sem nota a conversa sem informação e não avalia atendente individual. Diga isso antes que a banca diga. O que é diferente, com fonte em concorrentes:

- o Fraus não usa LLM (o Intercom declara usar LLM de terceiros);
- a explicação do Fraus é a decomposição exata do modelo;
- a incerteza do Fraus tem garantia estatística declarada, por classe. Nos concorrentes lidos, não encontramos incerteza por previsão;
- Zendesk QA, Observe.AI e Medallia pontuam agentes, e o Fraus não.

# 3. Usuários

## 3.1 GUIA DE USO

O Fraus ajuda a examinar atendimentos de chatbot e **estima satisfação** a partir das conversas. Não pergunta a nota ao cliente, não revela seu estado real e **não avalia o desempenho de atendentes**. Para entender a evidência e seus limites, consulte o cartão do modelo e as limitações.

### 3.1.1 Acesso e permissões

Acesse a dashboard da sua instalação. Quando o login estiver habilitado, entre com sua conta; o cadastro pode exigir um código ou convite fornecido pela organização. Um convite de equipe precisa ser aceito com uma sessão de usuário válida. Não envie senhas ou chaves junto com arquivos de atendimento.

Existem duas permissões diferentes:

- **Papel global:** `usuario` (analista) ou `dev` (desenvolvedor). Ambos têm as tarefas de acompanhamento e análise. **Modelo**, **Integrações** e **Configurações** são destinos de `dev`; gerar a chave do Tars e remover um atendimento também exigem esse papel.
- **Papel na equipe:** proprietário, gestor ou membro. Essa hierarquia governa ações sobre a equipe; **ser proprietário não transforma a conta em `dev`**. Controles de gestão dependem do papel informado pelo servidor, não apenas de conseguir abrir Operação.

Os canais autorizados também delimitam quais conversas cada pessoa pode ver. Uma conta criada por convite começa sem acesso às conversas; aceitar o convite atribui os canais cadastrados na equipe, respeitando políticas administrativas explícitas. Equipe sem canais não libera todo o histórico. Se faltar acesso, peça a quem administra a instalação que confira a equipe e os canais — não é motivo para reenviar os mesmos dados.

Na lateral ficam **Visão geral**, **Atendimentos**, **Analisar**, **Grafo** e, para `dev`, **Modelo**. Abra o **menu da conta**, no rodapé, para **Operação**, Integrações e Configurações conforme sua permissão, tema claro/escuro e saída. O link da documentação aparece quando configurado pela instalação.

### 3.1.2 Analisar um arquivo

1. Abra **Analisar** e arraste o arquivo à área de envio ou use **Escolher arquivo**. O seletor aceita CSV/TSV, planilhas XLSX/XLSM, JSON/JSONL, TXT, DOCX e PDF; a identificação do conteúdo depende do formato real do arquivo. Conversas exportadas do WhatsApp entram como arquivo de texto.
2. Aguarde a leitura. Há limite de **200.000 bytes por arquivo**; se a tela informar excesso, divida o conteúdo em partes menores preservando as conversas e seus horários.
3. Se a associação das colunas for confiável para o servidor, a análise segue automaticamente. **Com horários válidos, a tela solicita também a gravação**. Não existe uma segunda confirmação manual antes dessa tentativa de salvar.

4. Se houver dúvida, aparece **Conferir colunas**. Nada foi gravado nessa etapa. Confira **Fala** e **Quem falou**; em **Mais colunas — data, conversa e canal**, ajuste os campos disponíveis. Sem identificador de conversa, o arquivo é tratado como uma conversa só. Confira também a ordem dia/mês ou mês/dia quando perguntada, a amostra e os avisos de linhas rejeitadas.
5. Use **Analisar** para confirmar a conferência. Depois do resultado, a opção **Ajustar colunas**, quando disponível, permite revisar a associação; **Trocar arquivo** abre outro envio.

> ⚠ ARQUIVO SEM HORÁRIO: LEITURA AVULSA
>
> Sem horário das mensagens não há latência, **nota da conversa nem gravação nos indicadores**. A leitura por mensagem continua disponível. Não invente datas para obter uma nota; prefira exportar novamente com os horários reais.

A confiabilidade da leitura de colunas não é confiança estatística na classificação. Confira os avisos mesmo quando o fluxo seguir automaticamente. O tratamento de dados pessoais reduz exposição, mas não dispensa autorização para enviar os arquivos nem revisão das regras da organização.

### 3.1.3 Confirmar o que foi salvo

A confirmação **“atendimento(s) salvo(s) nos indicadores”** informa a quantidade salva e oferece **Ver indicadores**, **Ver atendimentos** e **Ver grafo** já no período do lote. Reenviar a mesma conversa é deduplicado pelo contrato de registro; isso não substitui conferir o resultado e o período.

**“Gravação não confirmada”** significa que a resposta não confirmou a persistência. Não apresente esses atendimentos como já incorporados aos indicadores. **“Análise concluída sem salvar nos indicadores”**, acompanhada do aviso de ausência de horários, é o resultado avulso esperado, não um zero de satisfação.

O resumo e a conversa vêm primeiro. A inspeção de sinais, probabilidades e contribuições pode ser expandida: probabilidades por mensagem e contribuições do fusor são objetos diferentes. O destaque de uma fala não demonstra a causa da nota nem a influência causal de palavras isoladas.

### 3.1.4 Encontrar o período e os atendimentos

Use os campos **De** e **até**, **Últimos 7 dias**, **Últimos 30 dias** ou **Todo o período**. Os atalhos terminam no **último dia com dados disponível na consulta, não em hoje**; as datas aparecem na tela. Isso permite trabalhar com um arquivo histórico sem obter um recorte vazio por engano. O período acompanha a navegação entre as telas pelos parâmetros da URL.

Se houver dados fora do recorte, **Limpar período** volta ao histórico disponível. Se realmente não houver atendimentos acessíveis, a orientação oferece analisar o primeiro arquivo e, para quem pode configurar, conectar uma fonte. **Falha da API não é banco vazio**: a tela deve mostrar a falha, não concluir que não existe histórico.

Na lista **Atendimentos**, abra um caso para ler a conversa e seus sinais. Use o **Grafo** como outra forma de explorar os registros, não como validação independente da classificação. A remoção de atendimento é ação administrativa de `dev`, exige confirmação e fica auditada.

### 3.1.5 Interpretar os números sem inventar certeza

- **Score, nota e categoria** vêm do servidor. As faixas de fábrica são 0–6 detrator, 7–8 neutro e 9–10 promotor; a instalação pode configurar outras faixas válidas. Não recalcule categorias por uma regra própria na planilha.
- **Sem sinal não é zero.** Ausência de fala informativa do cliente pode deixar score e nota ausentes, inclusive em encerramentos de mera cortesia. Esses casos não pertencem a uma categoria nem devem ser somados como detratores.
- **NPS inferido é estimativa**, não resposta a uma pesquisa. O cartão oculta o ponto com menos de **30 atendimentos com sinal**. O intervalo **AW(3,T)** mede incerteza amostral, **não o erro do classificador** nem uma garantia de acerto.
- Emoção é uma classificação textual auxiliar; não é leitura biométrica nem prova do estado real da pessoa. A ironia é exibida por mensagem, mas não entra no vetor que calcula a nota.
- Compare NPS e latência no conjunto, e leia casos individuais antes de agir. Associação entre tema, espera e satisfação não demonstra causalidade. O sinal de tempo foi treinado em conversas sintéticas, não em atendimentos reais rotulados com satisfação.

### 3.1.6 Quando algo não responde

Se aparecer um aviso de indisponibilidade, aguarde e use a tentativa de reconsulta oferecida. Um serviço pode responder à sondagem de saúde e ainda não conseguir carregar seus modelos para analisar; isso não confirma uma inferência bem-sucedida. Não troque ausência de resultado por nota zero.

No erro de arquivo, leia a causa e use **Tentar de novo** para reenviar o último arquivo ou **Escolher outro arquivo**. Quando oferecido, **Ajustar colunas** recupera a conferência. Se o erro persistir, encaminhe a mensagem de erro e o formato do arquivo ao responsável, sem compartilhar conversas ou credenciais fora dos canais autorizados.

### 3.1.7 Receber conversas do Tars

Quem tem papel `dev` pode abrir menu da conta → **Integrações** → **TARS** → **Conectar o TARS**. A chave é mostrada uma única vez: use-a na configuração do Tars, não em um arquivo de atendimento. Siga os passos e os limites de privacidade em Integração com o Tars. Se esse destino não aparece, solicite a configuração a uma pessoa autorizada; o papel de equipe sozinho não concede essa permissão.

### 3.1.8 Escopo deste guia

Este guia descreve os controles e contratos do código local de 26/10/2026. A dashboard e a API publicam separadamente: uma mudança documentada ou presente na interface local **não prova publicação em produção**. A inspeção local com dados sintéticos e motor dublê não comprova inferência nem gravação reais; nesta sessão, uma tentativa de upload TXT nessa demonstração retornou erro, não uma prova de análise bem-sucedida. Resultados científicos e pendências da campanha ficam em Generalização e evidências.

## 3.2 INTEGRAÇÃO COM O TARS

Os projetos continuam independentes. O Tars é o **produtor** das conversas e o Fraus é o **consumidor** analítico; nenhum acessa o banco ou importa código do outro. O único contrato entre os dois é `POST /ingestao`, a mesma rota que qualquer sistema externo usa.

Em produção desde 19/10/2026 (Fraus #97 e #98; Tars #28, #29 e #30).

### 3.2.1 Conectar, em dois passos

1. **No Fraus**, menu da conta → **Integrações** → cartão **TARS** → **Conectar o TARS**. A tela cria a fonte de canal `tars`, gera a chave `frs_...` (mostrada **uma única vez**) e mostra o endereço `https://fraus-api.vercel.app/ingestao`.
2. **No Tars**, **Integrações → Fraus** do assistente: cole o endereço e a chave e clique em **Conectar e enviar**. O Tars guarda a chave cifrada (como o token do Zendesk) e já manda as conversas da última semana — é o teste de verdade, porque só o Fraus sabe se a chave vale.

Gerar chave e segredo de fonte exige conta `dev` no Fraus (`exigir_mestra_ou_dev`, em `fraus/api/seguranca.py`). Antes de 19/10 a rota exigia a chave mestra, que o proxy da dashboard nunca manda — o botão dava 403 para o próprio desenvolvedor. A chave de **acesso** continua só com a mestra.

### 3.2.2 O que é um atendimento

Um chat do Telegram dura semanas; um atendimento, não. **30 minutos sem fala encerram o atendimento**, e a próxima mensagem abre outro, com a data dela (`dividirEmSessoes`, em `lib/integracoes/fraus-pedido.mjs` do Tars).

O id que chega ao Fraus é `tars:<instância>:<canal>:<chat>:<início>`, e o Fraus ainda o prefixa com a fonte (`fonte:<id>:...`). O mesmo id **substitui** a versão anterior (upsert), então reenviar nunca duplica.

O Tars poda o histórico em 40 mensagens por chat. Recalcular o id pelo primeiro horário que sobrou criaria um atendimento novo a cada poda; por isso o id do atendimento em curso fica guardado em `fraus_envios.sessao_id` e é herdado enquanto a conversa não pausa 30 minutos.

### 3.2.3 Quando o envio acontece

Depois de cada resposta gravada (`registrarResposta`/`registrarTurno`), o Tars agenda o envio com `after()` — **nunca atrasa a resposta ao cliente**. O envio espera **5 segundos** e só segue se nenhuma fala mais nova chegou; se chegou, o envio da fala mais nova é que manda. Uma rajada de mensagens vira um envio só, e a última versão do atendimento sempre chega.

> Até 19/10, 17h30, o Tars usava um freio de um envio por chat por minuto. Duas mensagens no mesmo minuto deixavam a última fala para o cron do dia seguinte: o atendimento terminava em "péssimo" e o Fraus o mostrava como neutro.

A **repescagem** manda o que falhou: o botão **Enviar conversas pendentes** e o cron diário do Tars, que só reenviam atendimentos com fala depois do último envio. Na 10ª falha seguida o envio desliga e o operador recebe um alerta; salvar a conexão de novo religa.

### 3.2.4 O pedido

```
{
  "id": "tars:inst_x:telegram:8624457065:1792441687",
  "mensagens": [
    {"autor": "cliente", "texto": "...", "enviada_em": "2026-10-19T20:28:07.000Z"},
    {"autor": "bot", "texto": "...", "enviada_em": "2026-10-19T20:28:09.000Z"}
  ],
  "escalou_para_humano": false
}
```

- `user` vira `cliente`; `assistant` vira `bot`; resposta escrita pela equipe no painel vira `humano` e marca `escalou_para_humano`.
- Horário em ISO com fuso — o Fraus recusa data sem fuso (invariante 6).
- O corpo cabe no teto de 32 KB do `chamarApiSegura`: se não couber, só as respostas longas do bot são encurtadas; a fala do cliente nunca é cortada.
- Score, nota e categoria **nunca** vão no pedido: o Fraus deriva no servidor (invariante 3). O canal vem da fonte cadastrada, não do corpo.

### 3.2.5 Privacidade — o que vale e o que falta

- O Fraus aplica a censura de PII ao texto antes de persistir.
- **O id leva o `chat_id` do Telegram em claro.** É um identificador de conta, não conteúdo, mas atravessa a fronteira. Trocar por um hash estável (com sal da instância) é a próxima mudança de privacidade; como o id novo não colide com o antigo, a troca pede remover antes os atendimentos do Tars no Fraus.
- Os botões "Resolveu / Não resolveu" do Tars ainda **não** são enviados como `feedback_declarado`.

### 3.2.6 Limpeza

Atendimentos que nunca deveriam ter entrado (o datado de 2243, duplicatas de teste, o chat inteiro no formato anterior a 19/10) saem pelo botão **Remover atendimento**, no detalhe do atendimento — só conta `dev`, com confirmação, e registrado na auditoria (`DELETE /conversas/{id}`).

# 4. Desenvolvedores

## 4.1 DESENVOLVIMENTO LOCAL

Este percurso prepara **uma instalação de desenvolvimento**, não um deploy. Leia as invariantes antes de alterar regras e use a arquitetura para localizar a responsabilidade de cada peça. O Fraus estima satisfação sem LLM em runtime: a predição usa modelos locais, em CPU, sem chamada de rede no caminho de inferência.

### 4.1.1 Ambiente Python e extras

Use Python **3.11** e `uv`, na raiz do repositório. Para desenvolvimento com Torch, testes e documentação:

```
uv sync --extra dev --extra torch --extra docs
```

Os extras são explícitos: um `uv sync` sem eles pode remover ferramentas que você já estava usando. Preserve os extras necessários em cada sincronização. O grupo `dev` instala pytest/httpx; `torch` instala o executor Torch; `docs` instala MkDocs Material, mkdocstrings e a ferramenta de PDF.

Para trabalhar no treinamento instrumentado, acrescente `treino`:

```
uv sync --extra dev --extra torch --extra treino --extra docs
```

Esse extra inclui **accelerate**, exigido pelo `Trainer`, além das dependências de treino/exportação. Não é dependência necessária à imagem de inferência. As receitas, os notebooks e o protocolo de seleção/avaliação estão em Treinamento e Generalização e evidências. Ter os notebooks prontos não significa que uma campanha foi executada.

A alternativa de inferência sem Torch é ONNX:

```
uv sync --extra dev --extra onnx --extra docs
```

Declare `FRAUS_BACKEND=onnx` ao iniciar a API. Os grafos exportados são outros artefatos, em `modelos-onnx/`; o backend não é escolhido pela presença de arquivos e não recua silenciosamente para Torch. Conversão é uma tarefa separada, com extra `conversao`: consulte Encolher os modelos antes de exportar ou quantizar.

### 4.1.2 Artefatos obrigatórios

Instalar dependências **não baixa os modelos treinados do Fraus**. O motor padrão exige os três classificadores — satisfação, emoção e ironia — e um fusor compatível. No backend Torch, os caminhos de fábrica são:

```
modelos/
├── bertimbau-satisfacao/
├── bertimbau-emocao/
├── bertimbau-ironia/
└── fusor.joblib
```

Cada cabeça precisa de seus pesos, configuração e tokenizador. Com ONNX, as três cabeças são lidas dos diretórios homônimos em `modelos-onnx/`; o fusor continua em `modelos/fusor.joblib`. Use artefatos de origem autorizada e rastreável; não carregue um pickle/joblib desconhecido para inspecioná-lo.

O fusor exige **39 features de sete famílias**, com as transformações vigentes (incluindo `log1p` nas quatro features de espera). Um fusor antigo treinado em segundos crus é recusado, mesmo que tenha 39 entradas. **Não contorne a recusa** para fazer a UI funcionar. A ironia continua obrigatória na leitura por mensagem, embora não pontue no vetor do fusor.

A aplicação cria banco e rotas antes de carregar o motor, que é preguiçoso. Portanto, **a API subir ou responder à saúde não prova que consegue inferir**: a primeira operação que exige modelos carrega os artefatos e falha explicitamente se faltarem ou forem incompatíveis. Não há score parcial, modelo dublê automático na API real nem valores zerados para preencher falta de dados. Consulte Cartão do modelo para a procedência e os limites das métricas; uma métrica histórica não identifica o checkpoint que uma implantação serve hoje.

### 4.1.3 Executar API e dashboard

Na **raiz**, inicie a API real em uma porta própria:

```
uv run python -m uvicorn fraus.api.main:app --host 127.0.0.1 --port 8001 --reload --reload-dir fraus
```

Ou, com os grafos ONNX preparados:

```
FRAUS_BACKEND=onnx uv run python -m uvicorn fraus.api.main:app --host 127.0.0.1 --port 8001 --reload --reload-dir fraus
```

Sem uma conexão PostgreSQL explicitamente configurada, o banco local é SQLite. Use banco e dados descartáveis para experimentos; não a conexão de produção. Configuração de credenciais pertence ao ambiente autorizado da instalação, nunca ao Markdown, ao Git ou a variáveis públicas do navegador. Não desative a autenticação nem publique um túnel para contornar um erro local.

Em **outro terminal**, entre em `dashboard/`:

```
cd dashboard
npm ci
FRAUS_API_URL=http://127.0.0.1:8001 npm run dev
```

A interface abre normalmente em `http://127.0.0.1:3000`. Não há `package.json` na raiz. `FRAUS_API_URL` é configuração **do servidor Next**, que encaminha as requisições e mantém a credencial fora do bundle do navegador. O padrão é porta 8000; o exemplo acima muda explicitamente para a porta 8001 usada na API real. Use o fluxo de acesso da instalação, sem copiar tokens para código ou para esta documentação.

#### DEMONSTRAÇÃO NÃO É VALIDAÇÃO DO MOTOR

Para demonstração isolada existe, na raiz:

```
uv run python scripts/api_demo.py
```

O script usa banco temporário e conversas sintéticas na porta 8000; pode usar motor real quando houver artefatos ou **dublê explicitamente identificado**. Nunca é o comando de produção. Gravação analítica exige motor real; uma interação bem-sucedida com dublê só exercita a interface. Mantenha essa instância separada da API real e ajuste o destino da dashboard conscientemente.

### 4.1.4 Verificar uma alteração

Os comandos abaixo são instruções para quem verifica a mudança; **não são resultados de execução desta página**. Na raiz:

```
uv run pytest -q
```

A rodada padrão exclui testes marcados como lentos. Ao alterar o gerador de corpus, execute também a seleção correspondente:

```
uv run pytest -q -m lento
```

Dentro de `dashboard/`:

```
npm test
npm run lint
npm run build
```

Teste os contratos relevantes, não só a compilação: ausência de sinal permanece nula; score, nota e categoria vêm do servidor; leitura sem horário não grava; controle administrativo não ganha permissão pela aparência da

tela. Para mudanças visuais, confira teclado, celular, temas e movimento reduzido seguindo o design da interface. Um teste com fixture precisa ser identificado como tal e não substitui a verificação de inferência real.

### 4.1.5 Trabalhar na documentação existente

O projeto usa **um site MkDocs Material**, com três percursos de leitura. Não há uma segunda cópia dos textos para cada público. Mermaid, CSS e a referência extraída de docstrings continuam na mesma infraestrutura.

Na raiz, preserve os extras e gere a cópia do design antes do build:

```
uv run python scripts/reunir_docs.py
uv run mkdocs serve -a localhost:8010
uv run mkdocs build --strict
uv run mkdocs build --strict --config-file mkdocs-publico.yml
```

`docs/design.md` é **gerado** a partir de `dashboard/DESIGN.md`: altere a fonte, nunca a cópia. A comparação histórica permanece em Antes e depois da interface.

O repositório é privado e o site é público. `mkdocs.yml` constrói a documentação interna; `mkdocs-publico.yml` exclui notas, auditorias, handoff e roteiros internos. `not_in_nav` **não impede publicação**. Preserve `exclude_docs` e `show_source: false` no recorte público: docstrings podem ser apresentadas, mas os corpos das funções não devem ser embutidos no HTML público. Manifestos privados, relatórios brutos e corpora de experimentos ficam fora do site.

O PDF herda o recorte público e mantém essa mesma fronteira. Precisa das bibliotecas nativas usadas pelo WeasyPrint (GTK/Pango/Cairo) e de Chromium para desenhar Mermaid, por isso seu build é separado, no CI Linux. `FRAUS_CHROME_PDF` seleciona o executável: o padrão local é `chromium`; o workflow usa `google-chrome`.

```
uv run mkdocs build --config-file mkdocs-pdf.yml -d site-pdf
```

`pdf_event_hook.py` transforma os diagramas em imagens SVG antes do WeasyPrint; diagrama ausente ou com erro interrompe o PDF. A biblioteca Mermaid 11.17.2 tem SHA-256 fixado e é baixada no primeiro build para `~/.cache/fraus/`. Isso ocorre na documentação, não na inferência do Fraus. O site continua usando a integração Mermaid do Material.

Publicar dashboard, API ou documentação é outra operação. Estes comandos locais não promovem pesos nem provam qual checkpoint está em produção.

### 4.1.6 Contratos para consultar antes de editar

- Invariantes: classes, timestamps, ausência de sinal, derivação no servidor e contrato de features.
- Arquitetura: mensagem, conversa, sinais, fusor e limites da explicação; modelo canônico e sinais detalham os componentes.
- Fusor, indicadores e evidência e deriva: referência das regras no código, sem criar outra implementação no cliente.

- Guia de uso e Integração com o Tars: contratos visíveis a quem opera arquivos, períodos e fontes.
- Treinamento, generalização e limitações: separar receita, medição, hipótese e evidência indisponível. Nenhum experimento candidato deve substituir o artefato ativo sem uma decisão de promoção separada.

## 4.2 AS DEZ INVARIANTES

Não são preferências de estilo. **Cada uma nasceu de um defeito real**, e violá-la é reintroduzir o defeito — por isso cada regra aqui vem com a cicatriz que a produziu. Documentação que explica cicatriz é a que não dá para copiar de tutorial.

O formato é sempre o mesmo: **a regra**, *por que ela existe*, **o que quebra se alguém a violar**, e onde ela mora no código.

---

### 4.2.1 1. Sem LLM em runtime

**A regra.** Inferência local em CPU. Nenhuma chamada de rede no caminho de predição.

**Por que existe.** É requisito do trabalho, não otimização de custo. A tese é que dá para medir satisfação com modelos pequenos, fine-tunados e interpretáveis — chamar uma API de LLM não deixaria o sistema mais rápido, e invalidaria o que ele se propõe a demonstrar.

**O que quebra.** O trabalho inteiro. Nenhum teste pega isso: o código funcionaria melhor e a defesa cairia.

**Onde mora.** Três BERTimbau em `fraus/sinais/` e um `LogisticRegression` em `fraus/fusor.py`.

---

### 4.2.2 2. Ausência de dado não é insatisfação

**A regra.** Conversa sem fala do cliente tem `score: None` e aparece como "sem sinal" — nunca `0`, em lugar nenhum: célula, gráfico, ordenação, export, agregado.

**Por que existe.** `None` e `0` significam coisas diferentes e não podem se confundir. Zero numa coluna de tempo de resposta se lê como "respondeu na hora", e a conversa que nunca teve atendente humano apareceria como **a mais ágil da operação**. Zero num NPS se lê como um número que alguém mediu.

**O que quebra.** Silenciosamente, tudo: o pior valor da escala aparece onde a verdade era "não medi". Em 10/09/2026 essa regra ainda estava furada dentro do próprio arquivo que mais a defende — `containment_rate` devolvia `0.0` no conjunto vazio, e o front tinha uma guarda para não acreditar nele.

**Onde mora.** Em toda parte. Procure `?? 0` e `|| 0` antes de commitar — e **também** `return 0` **no Python**, que é o mesmo defeito disfarçado de inicialização.

---

### 4.2.3 3. Score, nota e categoria são derivados no servidor

**A regra.** Nunca aceitos do corpo da requisição, nunca recalculados no cliente.

**Por que existe.** O `round` do Python é bancário (`round(6.5) == 6`) e o `Math.round` do JavaScript arredonda meio para cima (`Math.round(6.5) == 7`).

**O que quebra.** Já quebrou: a tabela exibia **nota 7 ao lado da etiqueta "Detrator"**, nas fronteiras 6/7 e 8/9. Duas fontes da verdade para o mesmo número, discordando na tela do usuário.

**Onde mora.** `fraus/indicadores.py::nota_0_10`. O front consome; não recalcula.

---

### 4.2.4 4. Faixas de NPS: 0–6 detrator, 7–8 neutro, 9–10 promotor

**A regra.** Padrão de fábrica em `FAIXAS_NPS`, configurável por `PUT /configuracoes` — e só se as faixas cobrirem 0..10 de forma contígua. A faixa vigente é passada **por parâmetro**, nunca lida de estado global.

**Por que existe.** Estado global mutável muda sob os pés de quem já calculou. E faixa com buraco ou sobreposição não é preferência de gosto: é nota sem categoria, ou com duas.

**O que quebra.** Já quebrou: `categoriaDaNota` no front tinha `<= 6` e `<= 8` digitados no corpo, e isso rodava no caminho feliz. Um operador que mexesse em Configurações via a **mesma tela dar dois vereditos** para o mesmo atendimento — a etiqueta da linha com as faixas novas, a cor da barra com as antigas.

**Onde mora.** `fraus/indicadores.py::FAIXAS_NPS`, `categoria_nps`, `validar_faixas_nps`.

---

### 4.2.5 5. Latência nunca é persistida

**A regra.** Sempre derivada dos timestamps, na leitura.

**Por que existe.** Latência gravada é latência que envelhece: qualquer correção na definição de "espera" deixaria o banco cheio de números calculados pela regra antiga, indistinguíveis dos novos.

**O que quebra.** Duas definições de latência convivendo — uma para a lista, outra para o detalhe — e as duas telas discordando sobre o mesmo atendimento.

**Onde mora.** `fraus/sinais/tempo.py::latencias_da_conversa`.

---

### 4.2.6 6. Timestamps timezone-aware

**A regra.** `datetime` naive é erro de validação, não conversão silenciosa.

**Por que existe.** Data sem fuso é data que muda de significado conforme quem lê.

**O que quebra.** A armadilha já paga: **a data da Totalk é `MM/DD/YYYY`**, não `DD/MM`. Lida como brasileira, espalha as mensagens por cinco meses e a latência sai absurda **em silêncio**.

**Onde mora.** `fraus/modelos.py`.

---

### 4.2.7 7. Modelo ausente é falha alta e explícita

**A regra.** Sem fallback, sem motor dublê silencioso. Nenhuma predição sai sem os três modelos treinados e o fusor. A API sobe sem carregá-los (o `ProvedorDeMotor`, em `fraus/api/motor_preguicoso.py`, carrega na primeira rota que exige o motor), mas a falha de carga é memorizada e sobe de novo em toda chamada, e `GET /saude/prontidao` responde 503 enquanto o motor não está `pronto`. Não existe queda para dublê.

**Por que existe.** Servir predição sem modelo carregado é **pior que estar fora do ar**: a interface não distingue número real de número sintético.

**O que quebra.** Já quebrou, e custou um dia: o `scripts/api_demo.py` estava no ar por causa do diretório de onde foi lançado, e a dashboard escreveu "API no ar" durante um dia inteiro enquanto **todo número da tela era sintético** — os pesos por feature saíam numa progressão 0,2 / 0,25 / 0,3 e passavam por coeficiente de regressão logística.

**Onde mora.** `Fusor.carregar` (valida `n_features_in_`), `ProvedorDeMotor` (`fraus/api/motor_preguicoso.py`), `GET /saude` (`"motor": "real" | "duble"` e `"estado_motor"`), `GET /saude/prontidao`, e `GET /saude/deriva`, que recusa diagnosticar com dublê pela mesma lógica.

---

### 4.2.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito

**A regra.** No notebook, no sinal de texto, no fusor e nos indicadores.

**Por que existe.** É a única convenção que amarra quatro lugares que não se importam por código.

**O que quebra. Nada estoura.** O sistema pontua ao contrário, em silêncio, com a mesma confiança de sempre. É a invariante mais barata de violar e a mais cara de descobrir.

**Onde mora.** `fraus/sinais/texto.py` (`INSATISFEITO`, `NEUTRO`, `SATISFEITO`).

---

### 4.2.9 9. As 39 chaves de feature batem exatamente com `NOMES_FEATURES`

**A regra.** As sete famílias do vetor produzem exatamente 39 chaves. `vetorizar` levanta `KeyError` na falta — nunca zero silencioso.

**Por que existe.** Zero numa feature ausente não é neutro: o modelo aprendeu o que zero significa naquela dimensão, e a conversa entraria como se tivesse sido medida.

**O que quebra.** Já quebrou de outro jeito: o `CLAUDE.md` chegou a declarar "38" na invariante e "(35)" no mapa — dois números de duas revisões, nenhum o vigente. A documentação que governa toda sessão de agente envelhecendo em silêncio. Hoje há teste amarrando o número do `CLAUDE.md` a `len(NOMES_FEATURES)`.

**Onde mora.** `fraus/fusor.py::NOMES_FEATURES` — e o comentário dele conta a história inteira: 40 − 2 (a ironia saiu) + 1 (`incongruencia_situacao_negativa` entrou) = 39.

---

### 4.2.10 10. O corpus de treino não pode entregar o rótulo

**A regra.** Distribuição por rótulo se sobrepõe; feature constante no treino nasce com peso zero. **Acurácia alta demais é sintoma, não vitória.**

**Por que existe.** Faixa de latência disjunta por classe fez o primeiro fusor marcar **99,3% lendo só o relógio**, com o BERTimbau apagado.

**O que quebra.** Um número excelente que não mede nada. E ela não para no treino: em 08/09/2026 a mesma invariante apareceu **em runtime** — numa conversa com 3 h de espera, a latência contribuiu −85,4 contra +4,6 do texto inteiro, e o score saiu em `4,4e-24`. O corpus nunca passou de minutos, então a guarda de vazamento, que olha o corpus, não tinha o que dizer.

**Onde mora.** `docs/treinamento.md`, e agora também `GET /saude/deriva`, que transforma a regra de disciplina de notebook em alarme de runtime.

name: Fraus — Contexto, não certeza description: Grafite, monograma FR em camadas e tipografia quadrada. Uma chegada expressiva e uma ferramenta de leitura calma, com temas escuro e claro. colors: background: "#171819" foreground: "#F3F4F5" card: "#222426" muted-foreground: "#B3B9C0" border: "#454B52" primary: "#D6BA72" dito: "#EDB466" medido: "#8DC8F5" typography: display: fontFamily: "Chakra Petch, system-ui, sans-serif" fontWeight: 500 lineHeight: 1.06 letterSpacing: "-0.025em" prosa: fontFamily: "Chakra Petch, system-ui, sans-serif" fontWeight: 400 lineHeight: 1.75 dado: fontFamily: "Chakra Petch, system-ui, sans-serif" fontFeature: "tabular-nums" chanfro: botao-lp: "12px" card-lp: "18px" botao: "9px" botao-pequeno: "7px" controle: "7px" card: "12px" etiqueta: "6px" spacing: sidebar: "224px" sidebar-recolhida: "64px" gutter-lp-desktop: "64px" gutter-lp-mobile: "24px"

## 4.3 DESIGN DA INTERFACE — CONTEXTO, NÃO CERTEZA

Direção aprovada no protótipo v3 e implementada em 16/10/2026. Este documento substitui as regras visuais do **Instrumento / Chuva de neon**; não é uma camada adicional sobre elas. A comparação histórica está em Antes e depois da interface.

A verdade de produto continua em `dashboard/PRODUCT.md`. Tokens executáveis moram em `dashboard/app/globals.css`; a forma das camadas (canto, placas, fio, fenda e pincelada) mora em `dashboard/app/camadas.css`; a LP tem seu escopo em `dashboard/app/lp.css`. Os valores abaixo documentam o código, não constituem uma segunda configuração.

### 4.3.1 Direção

**A marca tem matéria; a ferramenta tem clareza.** A LP combina o monograma FR, camadas com profundidade, tipografia angular e pinceladas diagonais. A dashboard usa a mesma família tipográfica e as mesmas vozes de cor, mas não recebe uma cena atrás dos dados. A fenda do monograma reaparece como pequeno detalhe de identidade, não como recorte que prejudica texto ou alvo de clique.

- Grafite, branco mineral e ouro fosco na chegada.
- Cabeçalhos e navegação em caixa normal; sem telemetria numerada decorativa.
- Números como texto legível, não polígonos de sete segmentos.
- Controles e regiões com **canto cortado** na diagonal da fenda, superfície opaca e borda simples que continua nas duas diagonais. Na LP a peça entra inteira (placas que se abrem); na dashboard, contida (placas só no botão primário).
- Uma região dominante por tela; apoio não disputa o mesmo peso visual.

#### CAMADAS DO FR

Botões, cards e títulos repetem o monograma: placas empilhadas como a extrusão do FR e o canto cortado da fenda. **Uma fonte só:** o polígono do canto e as cores das placas existem apenas em `app/camadas.css`, em `@layer camadas` (acima dos utilitários, para `rounded-*`, `bg-*` e `outline-none` não apagarem o canto nem o foco). A API

pública são classes e variáveis; nenhuma classe define as variáveis PÚBLICAS (`--chanfro`, `--placa-*`, `--cor-*`), só lê com valor de reserva; as internas (`--camada-*`, `--traco-cor`) são definidas pelas classes.

| CLASSE | O QUE DESENHA |
|---|---|
| `.chanfro` | canto cortado no topo-esquerdo e no inferior-direito, com o traço de 1px da borda nas diagonais (`--cor-borda-chanfro`; elemento sem borda usa `transparent`) |
| `.placas` | face no `::before`, placa 2 no `::after`, placa 3 no `drop-shadow`; no hover a face sobe e as placas abrem |
| `.placas-so-hover` | as mesmas placas, planas em repouso |
| `.fio` | 2px de ouro no topo à direita, 40% da largura |
| `.fio-hover` | sublinhado de 2px que cresce no hover e no foco |
| `.fenda` | risco diagonal pequeno no canto superior direito |
| `.pincel` | pincelada inclinada (`skewX(-24deg)`) atrás da palavra-chave |
| `.chanfro-marco` | o canto no `::before` de um marcador (o ponto do `.step`) |

- Fio, fenda e o traço do canto **compõem** por variáveis numa única `background-image`: `superficie fio`, `stat placas fio` `fenda` funcionam juntos. Por isso `.superficie` usa `background-color`, nunca o atalho `background`.
- **Placas são pseudo-elementos, nunca filhos:** `data-embaralhar` reescreve o `textContent` do botão da LP. A camada anula `border`, `ring`, `shadow`, `bg` e `rounded` do próprio elemento, que deve tirar essas classes; a borda da face vem de `--placa-borda`. A transição das placas não inclui `opacity`: o GSAP lê o valor final de `.rv` e o botão nasceria invisível.
- **Foco visível por dentro.** O `clip-path` corta anel externo: em `.chanfro` e `.superficie` o anel é `outline` com deslocamento −2px; em `.placas`, sem clip-path, o anel fica a 6px, longe da face que sobe. CSS sem camada com `outline-offset` positivo vence essa regra; a LP repete o −2px em `.lp .chanfro:focus-visible`.
- `.superficie` tem `clip-path`: conteúdo absoluto que não seja portal e transborde a região é cortado.
- `filter` em `.placas` cria bloco de contenção para `position: fixed`: sobreposição dentro de algo com placas vai por portal.
- **Tokens das placas:** `--placa-2/3` e `--placa-card-2/3` seguem o tema (`#a8904f`, `#6f5f36`, `#34302a`, `#2a2621` no escuro; no claro, derivados dos tokens do claro por `color-mix`, sem hexadecimal do escuro). A LP é sempre escura e lê `--placa-escuro-*`; o fecho dourado lê `--placa-fecho-*` (face grafite `#171819`, placas `#3a3326` e `#7a6532`, texto ouro).
- Ouro continua sendo ação e marca: o fio, a fenda e a pincelada são identidade, não valor.

### 4.3.2 Cor e significado

| PAPEL | ESCURO | CLARO |
|---|---|---|
| Fundo | `#171819` | `#F3F4F5` |
| Superfície / popover / sidebar | `#222426` | `#FFFFFF` |
| Texto principal | `#F3F4F5` | `#202326` |
| Texto secundário | `#B3B9C0` | `#565D65` |
| Ação e foco | `#D6BA72` | `#735713` |
| Dito | `#EDB466` | `#8A4B00` |
| Medido — marcas gráficas | `#8DC8F5` | `#165E9D` |
| Texto de medidas | `#F3F4F5` | `#202326` |
| Borda | `#454B52` | `#D5D9DE` |

**Âmbar é dito; azul identifica as marcas gráficas do medido.** Fala, texto e emoji pertencem ao primeiro; probabilidade, nota, tempo e tendência, ao segundo. Por decisão de 22/10/2026, os textos antes azuis usam branco mineral no escuro e grafite no claro (`--medido-texto` acompanha `--foreground`). Na LP, sempre escura, o token acompanha `--texto`. Números, títulos e legendas ficam neutros; séries, bordas e marcadores continuam com suas cores semânticas. Uma régua entre dito e medido só aparece quando existe essa divisão real; NPS × latência são duas medidas, não uma fala confrontada com uma medida.

**Ouro não é dado.** Pertence a marca, ação, foco e seleção. Não colore série, barra, categoria ou valor de indicador para parecer bom resultado.

Categorias, famílias, emoções e tipos de nó conservam seus significados. No tema claro, a luminância se adapta ao fundo, não se troca a família de matiz. Toda categoria leva rótulo; cor não é o único canal. Os validadores medem texto e marcas nos dois temas, inclusive as cores do grafo.

### 4.3.3 Tipografia e ícones

**Chakra Petch local**, pesos 400, 500 e 600, com licença SIL OFL em `dashboard/public/identidade/`. `next/font/local` serve os arquivos do próprio produto; não há busca de fonte no Google no build ou no navegador.

- Texto, títulos, controles e valores usam a família quadrada.
- Valores comparáveis usam algarismos tabulares. Código, `pre` e `kbd` mantêm pilha monoespaçada de sistema.
- Piso de texto: 11px. Não diminuir procedência ou ressalva para fazê-la caber.
- Títulos grandes fecham o tracking; não aplicar caixa alta e tracking de telemetria a toda a interface.
- Lucide com ícones contextuais. Ação usa o desenho do verbo; seleção é identificada pelo estado, não por um ícone arbitrário.
- Ícone decorativo leva `aria-hidden`; texto ou nome acessível diz a ação.

## 4.3.4 LP

A LP segue o mockup GSAP + 3D aprovado em 19/10/2026, que **substituiu** as regras de movimento anteriores (palco sticky, troca de pose em 650ms, sem partículas nem laço). O conteúdo continua SSR e completo sem JS; a coreografia inteira mora em `components/lp-nova/Coreografia.tsx`, o único componente cliente.

### ESTRUTURA E CONTEÚDO

1. **Abertura:** “O atendimento termina. A análise começa.”, ação para conhecer o método e quatro fatos do motor: 7 famílias, 39 features, 3 BERTimbau, 0 LLM. Vêm de `FATOS_LP`; `tests/test_lp_fatos.py` falha se o fusor divergir. O servidor já entrega o número final; o contador só anima por cima.
2. **Letreiro** “Contexto · não certeza · Dito · Medido”, decorativo (`aria-hidden`).
3. **Contexto:** “Ok, obrigado.” identificado como **exemplo de linguagem**; pedido, resposta e desfecho dizem o que conferir.
4. **Sinais:** leitura da mensagem, contribuição dos sinais e tempos; as leituras gravadas seguem em expansão nativa logo abaixo.
5. **Faixa:** TEXTO, EMOJIS e TEMPO, fixada e rolada na horizontal.
6. **Uso:** panorama → recorte → conversa, com destino real para `/dashboard`.
7. **Limites:** estimativa, não pesquisa; não avalia atendentes; tempo treinado com dados sintéticos; sem sinal não é zero.
8. **Fecho:** o mundo vira ouro; entrar e criar conta.

O rodapé diz “toda **nota** é inferida”: os contadores do topo são fatos do código, não estimativas. `carregarLeituras` valida o snapshot; sem snapshot válido a LP nomeia a ausência e não publica resultado do dublê. Nenhuma análise é executada ao abrir a LP.

## CÉU

O fundo da LP é o céu da arte do João, em movimento: `<Ceu />` (`.lp-ceu`), fixo atrás de tudo (`z-index: 0`, abaixo do palco WebGL) e sem ponteiro. São 140 pontos dourados (`#e9cf8e`, 8% grandes; 60% piscam em ciclos de 2,5 a 7 s) e 26 riscos finos a −22° da vertical, que descem em 14 a 30 s.

- **Só CSS:** sem canvas e sem script por quadro; anima apenas `opacity` e `transform`/`translate`. As posições saem de um PRNG de semente fixa (`lib/lp-nova/ceu.ts`), renderizadas no servidor: sem `Math.random()` no render, a hidratação bate.
- **Fecho:** com `.lp.inv` o ouro entra por opacidade do `::before` em 0,6 s (gradiente não interpola), os pontos somem e os riscos escurecem, como na arte dourada. Nenhum código da coreografia muda.
- **O fecho dourado só existe com JS e movimento ligados** (`.inv` vem do ScrollTrigger); em reduce ou sem JS o fecho continua escuro.
- **Reduce:** o céu fica parado, com os riscos visíveis e a mesma cara da arte.
- **Sem JS:** o céu é HTML do servidor com animação em CSS; chega completo.
- **Camadas de 29/10/2026** (linguagem do filme “A fenda”): `.lp-nevoa` (placa de névoa gerada no Higgsfield, `public/lp-nova/nevoa.webp`, 3 KB, a 32%), `.lp-grade` (grade a −32,8°, o ângulo da fenda) e `.lp-feixe` (feixe dourado ao longo da fenda). Névoa e feixe **respiram sozinhos** por decisão do João; os três planos de pontos (`.lp-plano`, `--z` 0–2) fazem parallax pela linha do tempo da rolagem em CSS (`animation-timeline: scroll(root)`), sem script. Sem suporte, ficam parados. No fecho dourado as camadas somem.
- **Custo medido** (headless, CPU 4×): o feixe com `mix-blend-mode` custava ~60 ms por quadro, então ele é alfa puro; a grade é um segundo fundo da própria `.lp-nevoa`, não uma camada. No celular (< 900 px) o feixe some e o parallax para. Nada de blend nem de camada de tela inteira nova sem medir.

É a **única animação contínua permitida** fora do palco, e só porque é CSS e fica atrás dele. A dashboard não recebe o céu.

## WIDGETS DA LP

Botão primário `btn placas` com três placas (face ouro, `#a8904f` e `#6f5f36`), canto de 12px; no hover a face sobe 3px e a placa de trás desce. Botão secundário `btn o chanfro fio-hover`: contorno e fio que cresce embaixo. `.card` e `.stat` têm face com canto de 18px, duas placas que se abrem no hover e fio no topo; a `.stat` leva a fenda no canto e o rótulo reserva espaço para ela. `.ck li`, abas e o resumo das leituras têm canto e contorno, sem placas; `.tag` tem canto de 6px; o `.step` muda só o marcador, um quadrado cortado. A palavra “análise” do título da abertura leva a pincelada, dentro de uma máscara com folga para não sair cortada. No fecho as placas viram grafite com texto ouro. Os espaços de `.stats`, `.grid` e `.card` são de 16px, para as placas abertas não encostarem na vizinha.

## TIPOGRAFIA DA LP

Unbounded (títulos, letreiro, preloader) e Space Grotesk (corpo), variáveis, locais em `public/identidade/` com SIL OFL, carregadas por `next/font/local` **só em** `app/page.tsx`: a dashboard não baixa nenhuma delas. Rótulos, botões e números seguem em Chakra Petch.

### COREOGRAFIA

- **Preloader** dourado com contador, **uma vez por sessão**: o script do `<head>` (`SCRIPT_PRELOADER`) marca `data-lp-pre="visto"` antes do paint; quem volta de `/entrar` não espera de novo.
- **Lenis** no ticker do GSAP, âncoras pelo próprio Lenis. Títulos sobem palavra a palavra (quebradas no SSR, nunca `innerHTML`), revelações em lote, frase letra a letra com o texto inteiro para leitor de tela, régua e linha dos passos por scrub, botões magnéticos, cursor próprio só com `hover: hover`, texto embaralhado com nome acessível fixo, índice lateral `01 / 07`.
- O letreiro só anima dentro da viewport; ao voltar de baixo ele retoma em reverso sem parar no início da timeline. Cursor e magnetismo só registram ponteiro com `hover: hover` e `pointer: fine`.
- **Desmontagem**: cada listener, tween, ticker e o Lenis são revertidos; o QA confere que nada sobra em `/entrar`.

### FR E PALCO

O **modelo é intocável**: contornos de `escultura.ts` (do SVG oficial, com a fenda) e o material do shader. Câmera, poses e coreografia são livres. As cinco placas **não representam** famílias, features ou contribuições do modelo.

- Um canvas WebGL fixo de tela cheia (`lib/lp-nova/palco-fr.ts`): câmera de 35°, dois programas (FR e 700 partículas em disco, PRNG fixo), dois draws por quadro, DPR até 1,5 e no máximo 2,2 megapixels.
- Geometria e shaders carregam no primeiro respiro do navegador, não no bundle inicial da coreografia. O WebP SSR permanece disponível enquanto o palco prepara; formato, material, partículas e poses não foram simplificados.
- Cada quadro reutiliza o mesmo estado numérico. Perder o contexto destrói a instância anterior antes de restaurar; listeners não se acumulam a cada ciclo.
- Cada pose de `POSES_POR_SECAO` chega com a própria seção (`fromTo` com scrub). Uma timeline única sobre o `<main>` desalinhava por causa da faixa.
- Pulso entre placas, giro e letreiro leem a velocidade do Lenis **suavizada** a cada quadro. A velocidade crua chega em degraus pela roda do mouse e fazia o FR pular. Passos usam `deltaRatio`, então 120 Hz não gira em dobro.
- No fecho o FR vira relevo do próprio ouro, um tom abaixo. Grafite apagava o título escuro por cima.
- Celular: o mesmo palco, centralizado, escala 0,55 e opacidade 0,38.
- Reduce: um quadro parado, opacidade 0,3, sem preloader, pin, Lenis ou letreiro.
- Backend de software (SwiftShader, llvmpipe...), contexto recusado ou perdido: `fr-lp-0.webp` (ou a variante compacta) no lugar. O WebP é captura do mesmo modelo; o exportador antigo saiu com a narrativa sticky.

## 4.3.5 Dashboard

Sidebar de **224px**, recolhida em um rail de **64px**, com ícones e nomes em caixa normal. Dois grupos organizam o dia a dia: **Acompanhar** (visão geral e atendimentos) e **Investigar** (analisar, grafo e modelo, este só para `dev`). Gestão e configuração moram no **menu da conta** (`MenuDoUsuario`, `Menu` do Base UI), aberto pelo nome no rodapé: Operação, Integrações e Configurações (as duas últimas só `dev`), documentação, tema claro/escuro e

sair. Destinos, ícones, grupos e visibilidade têm uma fonte única em `lib/navegacao.ts` (`gruposDoPapel` para a lateral, `telasDaConta` para o menu).

- Página atual usa `aria-current="page"`; sua seção ancestral usa `"location"`. Links recolhidos mantêm nome acessível e tooltip por foco ou ponteiro.
- Somente `de` e `ate` acompanham a troca de tarefa. Busca, paginação e outros parâmetros locais não vazam para a próxima seção.
- Recolhimento é restaurado no servidor pelo cookie de preferência. Ctrl/Meta+B alterna a navegação sem interceptar input, textarea, select ou contenteditable.
- Ação de recolher/expandir fica junto à marca no desktop. No celular, o cabeçalho abre a gaveta; fechamento explícito e Escape devolvem o foco. Navegação comum fecha a gaveta; clique modificado para nova aba não a fecha.
- Marca interativa e link para a visão geral são controles separados. Tema, documentação, sessão e saída permanecem disponíveis; o rodapé compacto não trunca o aviso real de motor sintético.

Permissões continuam sendo contratos do servidor, não consequência de mostrar ou esconder links. Não converter ausência de papel em permissão administrativa.

A visão geral abre com título e filtro, faixa de quatro indicadores e NPS × latência como região dominante. A faixa são **quatro cards separados** (`.superficie`, canto de 12px, espaço entre eles), em quatro colunas em telas largas e duas em larguras menores; o NPS leva o fio dourado. O resumo do atendimento e o esqueleto de carregamento usam o mesmo card, sem fio: uma linguagem só de indicador. Informação de amostra e incerteza continua próxima do NPS, sem transformar ressalva em rodapé invisível.

A dashboard recebe as camadas **contidas**:

- **Canto cortado** em botões (9px; `sm`/`xs` 7px), campos, `select`, `textarea`, gatilhos de abas (7px), badges (6px) e regiões com caixa própria (`.superficie` e `alert`, 12px). `.superficie` já é cortada; o tooltip usa `superficie-flutuante`, e a gaveta móvel da sidebar não leva `.superficie`.
- **Placas só onde se clica:** o botão `default` tem as placas da LP, menores. `outline` e `secondary` têm canto, contorno e fio no hover; `destructive`, só canto (ouro é ação, não alerta). `ghost`, `link` e botões de ícone não têm canto nem placa: num quadrado de 24 a 36px o corte vira ruído. Card que não é clicável não ganha placa, porque a placa avisa “isso é clicável”.
- **Fio dourado** no NPS da faixa e no painel `dominante` (`superficie fio`). O `apoio` continua só com a linha no topo, sem caixa.
- Sobreposições (tooltip, popover, sheet e menus) não mudam. O tema claro usa as mesmas formas com placas derivadas dos tokens do claro.

`Painel` preserva dominante/apoio, erros locais, vazios e aparato metodológico. Superfície opaca, borda simples; nenhum ateliê, brilho especular, vidro ou fundo animado interfere nos gráficos. `EstadoVazio` nomeia o que falta e o próximo passo; o endpoint de origem fica em **Método e ressalvas**, não na face do painel.

## LEITURA PROGRESSIVA — 22/10/2026

- **Analisar:** mantém “arrastar e pronto”. Depois da leitura, a confirmação de gravação e o resumo vêm primeiro; a conversa fica legível sem marcações. Probabilidades, sinais por mensagem e contribuições só aparecem quando pedidos. Apenas o primeiro resumo do lote é dominante; os demais são apoio.
- **Vazios confirmados:** sem histórico disponível, orientar ao primeiro arquivo e, com permissão, à integração. Com histórico fora do filtro, oferecer limpar período. Falha de leitura da API não pode virar convite para importar.
- **Período:** datas manuais e atalhos de 7 dias, 30 dias e todo o período compartilham `de`/`ate` na URL. O fim dos atalhos é o último dia com dados, explicitado junto aos controles; não é a data de hoje.
- **Apoio legível:** descrições de indicadores, períodos, vazios, método e drivers usam 14px (`text-sm`) com entrelinha confortável. Eixos e outras marcas densas preservam a escala própria, sem aumentar toda a interface indiscriminadamente.
- **Honestidade não recolhe:** estimativa, sem sinal, limitações do tempo sintético e avisos sobre o arquivo permanecem visíveis. Endpoints, definições técnicas e escala logit entram no aparato existente, sem um segundo sistema de notas explicativas.

## ATIVIDADE E CARREGAMENTO — 22/10/2026

- A conferência de colunas carrega apenas quando solicitada; a espera tem `role="status"`. Não adicionar uma segunda chamada de leitura ou inferência para preparar os seletores.
- A sonda de saúde fica suspensa com a aba oculta e consulta ao voltar. Polling e tentativa manual compartilham somente a Promise em curso na mesma instância do provider. Isso não é cache de indicadores, sessão ou credencial; a cadência visível e os estados real/dublê/frio/erro permanecem os mesmos.

## NÚMEROS, AUSÊNCIA E PROVENIÊNCIA

`ValorNumerico` recebe uma string já formatada, rótulo e ausência explícita. Não calcula score, nota ou categoria. `NotaInferida` apresenta a nota recebida. Zero observado continua zero; `null` continua ausência.

- NPS e CSAT carregam **estimativa**. Contenção e latência são observadas.
- Amostra insuficiente não ganha ponto de NPS; o intervalo amostral não mede o erro do modelo.
- `sem sinal`, `sem estimativa` e `sem dados` não são sinônimos; a região nomeia a condição correta.
- Categoria medida: cabeça cheia; evidência fraca: tracejada com motivo; ausência: cabeça vazada cinza e texto. Cinza fica fora da escala.
- Erro de uma consulta não apaga indicadores que responderam.
- O segredo da marca nunca modifica indicadores, gráficos ou transcrições.

## FRAUS-CHAN ATRÁS DA LOGO

Cinco cliques consecutivos na marca, com até dois segundos entre eles, revelam a Fraus-chan. Quatro sprites novos compõem a entrada: espiar, puxar a fenda, dar um passo e acenar. A coreografia dura cerca de três segundos, com deslocamento do corpo, peso no passo e uma cortina diagonal que se abre. Cada fase avança pelo

`animationend` da personagem, sem temporizadores paralelos ao CSS. A personagem e “Dar um oi” percorrem as mesmas três reações: saudação, piscadinha cúmplice e comentário sobre a curiosidade de quem clicou. “Rever entrada” remonta o palco e reinicia a coreografia mesmo na mesma fase.

As quatro poses de entrada foram geradas pelo gerador nativo do Codex, solicitando `gpt-image-2` e usando a folha da Fraus-chan do `fraus-video` como referência. Vieram numa folha única, recortada em quatro WebP transparentes de 480×720, com escala e base dos pés alinhadas. Os prompts acompanham as artes nos arquivos `.webp.json`; `face` e `wink` preservam as reações anteriores. A pose antiga de puxar foi removida. Há seis imagens em `public/fraus-chan/`. Componente e imagens carregam só depois de descobrir o segredo; a entrada espera todas as imagens. Falha de imagem é explícita. Não há geração de imagem em runtime, LLM, leitura de conversas, áudio, canvas, persistência ou chamada de API dentro da brincadeira. As falas são fixas; não são respostas de um assistente nem resultados do modelo.

Grafite, ouro e Chakra Petch seguem a ferramenta; o tema claro também é respeitado. No celular, a personagem fica acima da fala e dos controles. `prefers-reduced-motion` pula a entrada espacial e mantém todas as reações em corte seco, inclusive ao ativar a preferência durante a animação. Não há movimento em loop nem fechamento automático. O diálogo Base UI contém foco, fecha por Escape/botão/clique fora e devolve foco à marca. No celular, fechar o segredo não fecha a gaveta que o abriu. O quebra-cabeça de fragmentos foi substituído, não conservado como outro modo.

#### TEMAS

**Escuro** é o padrão; **Claro** é alternativa da dashboard. Preferência fica no navegador e acompanha abas e navegação. Valores legados Instrumento/Chuva não são aliases: retornam ao padrão. LP e acesso ficam escuros, sem apagar a preferência da ferramenta; voltar à dashboard restaura a escolha.

O script inicial evita flash; `TemaDaRota` trata transições do App Router. O canvas do grafo invalida a paleta ao trocar tema sem recriar dados ou física.

### 4.3.6 Acesso, acessibilidade e estados

O acesso mantém o FR pré-renderizado de luz de corte: uma passagem curta, sem loop, interrompida ao operar o formulário. Não carrega motor 3D no navegador. Convite, código administrativo, erros, disponibilidade de login e navegação são contratos do servidor, não efeitos visuais.

- Teclado: pular conteúdo, foco visível, abas com setas/Home/End e `details` nativo com Enter. Controles da LP têm alvo mínimo de 44px.
- Carregamento: esqueleto com forma do resultado e estado ocupado, nunca zero ou “sem sinal” antes da resposta.
- Erro: causa visível e ação real, sem ocultar os vizinhos que funcionam.
- Movimento reduzido remove deslocamento; não apenas o torna mais lento.
- Não pontuar atendentes, não alegar leitura de pensamentos e não apresentar números de demonstração como resultados de clientes reais.

### 4.3.7 Verificação e manutenção

`npm test`, `npm run lint`, `npm run build`, `npm run contraste` e `npm run pisos` cobrem contratos diferentes. Nenhum substitui olhar e operar o produto.

`npm run test:lp` exercita sem JS, reduce, preloader por sessão, pin, índice, inversão do fecho, ida e volta para `/entrar` sem sobras, sem WebGL, celular sem rolagem lateral e o redirect de `/leitura`. `LP_EVIDENCIAS` guarda capturas. Sem GPU o Chrome headless usa SwiftShader e o palco cai no WebP, de propósito; `LP_GPU=1` pede GPU real para exercitar o WebGL. O vídeo do acesso derruba a GPU do headless com Vulkan, por isso o QA bloqueia mídia: é artefato do teste.

`DASHBOARD_URL=http://127.0.0.1:3017 npm run test:atividade` verifica pausa de sondas ocultas, reconsulta ao voltar, concorrência com tentativa manual e recuperação da disponibilidade. Só aceita host local; controla visibilidade e falhas HTTP no navegador, sem gravar dados nem provar inferência real. O QA da LP também confere o letreiro ao voltar do fecho e sua pausa fora da tela; com `LP_GPU=1`, inclui ciclos de perda/restauração sem acumular listeners.

`node scripts/validar-navegacao-playwright.mjs` opera rotas reais, período, recolhimento/reload, atalhos durante edição, tooltip por foco, tema, marca e gaveta móvel. `DASHBOARD_URL` aponta para o produto já iniciado; exige sessão válida ou o modo local aberto. Também conferir dashboard vazia, atendimento sem sinal, temas, grafo e navegação LP ↔ ferramenta.

`node scripts/validar-segredo-playwright.mjs` opera a entrada, as reações por teclado/toque, replay na mesma fase, reabertura, foco e modais aninhados. Confere mudança real dos pixels nas reações, carregamento sob demanda, indicadores intactos e ativação de reduce durante a entrada, além de desktop/celular/paisagem e temas claro/escuro. `SEGREDO_EVIDENCIAS` guarda as capturas opcionais; `DASHBOARD_URL` aceita só host local.

A documentação MkDocs mantém sua aparência própria nesta entrega; não presumir que o CSS do site foi redesenhado junto com o produto.

# 4.4 LP E DASHBOARD — ANTES E DEPOIS

**16/10/2026 · direção FR aprovada no protótipo v3 e aplicada ao produto.**

A versão anterior usava o vocabulário do **Instrumento**: orbe, fósforo de osciloscópio, rótulos monoespaçados e números em sete segmentos. A nova direção faz do **FR oficial** a assinatura visual: matéria clara e champanhe sobre grafite, letras quadradas e pinceladas diagonais. Na ferramenta, a expressão da marca fica no chassi; não há uma cena competindo com os dados.

Este registro compara implementação e composição. **Não é um estudo de usabilidade, uma medição de acurácia ou uma prova de publicação em produção.** As regras vigentes estão em Design da interface.

## 4.4.1 Condições da comparação

- **Antes:** produto exportado da revisão `e5aa19d859e1f5cc402958a16ae43450abc9a905`, sem alterar seu desenho. Capturas em servidor de desenvolvimento; o pequeno indicador circular do Next não faz parte da identidade do produto.
- **Depois:** build de produção local do redesign, na mesma máquina e Chromium. Não houve deploy nesta entrega.
- **Enquadramento equivalente:** desktop de 1440 × 900 e celular de 390 × 844 pixels CSS, escala de captura 1,25. Cada par mostra o mesmo tamanho de viewport.
- **Dados iguais:** snapshot de demonstração versionado, 64 conversas sintéticas, sendo 59 com sinal e cinco sem sinal; período completo de 01/08 a 22/08/2026. O banco foi copiado para uma instância temporária compartilhada pelas duas interfaces, sem reanalisar ou modificar as conversas.
- **Proveniência do snapshot:** SHA-256 `a18a4fa04e0a4ba74f145625727ba6812893327f7cf9c87a22849f08e2fbe9ef`.
- A API de demonstração identifica explicitamente **motor dublê — números sintéticos**. Esses números servem para conferir a interface, não são resultados de clientes reais nem validação do classificador.
- A LP estava **sem snapshot válido de leituras gravadas**. As duas versões mantêm ausência explícita; nenhum resultado foi criado para preencher a tela.

Clique nas imagens para abrir a captura em tamanho original.

## 4.4.2 Landing page


Fraus
Um obrigac
não conta t
ok, obrigado 🙂
CASO SINTÉTICO · COBRANÇA SEM SOLUÇÃO
O Fraus estima satisfação em atendimentos de chatbot, s
pesquisa. Inspecione o contexto antes de agir.
Entender como funciona
Entrar